Section 1: Introduction, purpose, hardware, design.... 4
SAIL Hardware.... 4
SAIL Storage Design.... 4
SAIL Network Design.... 5
SAIL Operational Goals.... 5
Section 2: Software choices and architecture decisions.... 6
Architecture Overview.... 6
Section 3: Operating system setup.... 6
Operating System Installation.... 7
Section 4: Physical setup and wiring.... 11
Network Topology Diagram.... 11
Hardware Layout.... 12
Hardware Wiring.... 12
Ethernet Cable Construction.... 13
Physical Environment Assembly.... 15
Section 5: Network Configuration.... 16
Network Planning and IP Address Allocation.... 16
Planned DHCP Reservations.... 18
IP Address Verification.... 19
Shared Storage Configuration.... 19
Step 1: Identify the SSD on pi-node-1.... 20
Step 2: Prepare the SSD Filesystem.... 20
Step 3: Create the Permanent SSD Mount Point.... 21
Step 4: Create the SAIL NFS Storage Directory.... 21
Step 5: Install the NFS Server on pi-node-1.... 22
Step 6: Export the SAIL Storage Directory.... 22
Step 7: Configure and Verify NFS Client Access.... 22
Step 8: Verify Shared Storage from pi-node-1.... 23
Completion Criteria -> Completed.... 23
Environment Shutdown Procedure.... 24
Section 6: Kubernetes setup.... 24
Kubernetes Installation.... 25
Step 1: Verify Network Connectivity.... 25
Step 2: Verify and Configure Required Raspberry Pi cgroups.... 25
Step 3: Install and Assemble the k3s Cluster.... 25
Step 4: Verify Cluster Information.... 26
Step 5: Verify Kubernetes System Pods.... 26
Step 6: Install the Kubernetes NFS CSI Driver.... 26
Step 7: Configure Kubernetes Administration from the Computer.... 27
Step 8: Create the SAIL NFS StorageClass.... 28

Step 9: Verify Kubernetes Persistent Storage....28
Step 10: Deploy and Verify a Test Workload and Service....29
Step 11: Remove the Test Workload....30
Section 7: Workload Design....30
Application Workload....31
Security Workload....32
Automated/AI Workload....32
Operational Workload....33
Integrated Workload Flow....33
Section 8: Application Development and Deployment....34
Application Design....34
Application Structure....36
Application Behavior....37
Application Deployment....37
Create the Application Environment....38
Step 1: Create the application folder and virtual environment....38
Step 2: Configure VS Code to Use the Environment....38
Step 3: Configure Git for the Application....39
Step 4: Install Application and Development Dependencies....40
Step 5: Create the Application Project Structure....41
Step 6: Configure Application Settings and Environment Variables....42
Step 7: Configure PostgreSQL for Development....43
Step 8: Configure SQLAlchemy and database connectivity....44
Step 9: Configure Alembic database migrations....45
Step 10: Implement the user data model and database schema....46
Step 11: Implement user registration, authentication, and password security....50
Step 12: Implement user profiles, roles, and permissions....56
Step 13: Run and manually verify the current backend locally....61
Step 14: Create the Orbit frontend structure and connect it to the FastAPI backend....66
Step 15: Build registration and login screens using the existing authentication API....70
Step 16: GitHub Actions for Black, Ruff, and frontend ESLint....73
Step 17: Build user profile and account screens....76
Step 18: Build posts and associated API endpoints, schemas, frontend functionality, and tests....85
Step 19: Add comments and reactions, i.e. data models, API endpoints, schemas, frontend functionality, and tests....85
Section 9: Monitoring, Logging, & Observability....85
Overview....85
Tools....85
Implementation....86
Step 1: Power on the SAIL cluster and verify network connectivity....86

Step 2: Verify SSH access, k3s services, and Kubernetes connectivity....86
Step 3: Verify SSD, NFS, and Kubernetes storage....86
Step 4: Verify internet access and install or verify Helm....86
Step 5: Inspect cluster resources, Metrics Server, and existing workloads....86
Step 6: Plan monitoring deployment, storage, and resource allocations....86
Step 7: Deploy Prometheus and configure metrics collection....86
Step 8: Configure Raspberry Pi and Kubernetes metrics....87
Step 9: Deploy Grafana and create monitoring dashboards....87
Step 10: Deploy Loki and centralized log collection....87
Step 11: Review monitoring storage, retention, and persistence....87
Step 12: Deploy OpenTelemetry and a tracing backend....87
Step 13: Instrument Orbit and PostgreSQL....87
Step 14: Configure monitoring alerts....88
Step 15: Establish normal operating baselines....88
Step 16: Verify monitoring end-to-end....88
Section 10: Security & Infrastructure Automation....88
Section 11: AI-Assisted Operations & Deployment Automation....89
Section 12: Event Scenarios and Investigation Workflows....89
Appendix:....91
Section 5 Step 7: sail-nfs-client-setup.sh....91
Section 6 Step 2: sail-k3s-prerequisites.sh....94
Section 6 Step 3: sail-k3s-cluster-setup.sh....97
Section 8: SAIL Orbit: Removal of display_name....100
Section 8 Step 17i: Deferred Issues: Frontend Dependencies and Security....103

Section 1: Introduction, purpose, hardware, design

SAIL, which stands for Security and AI Infrastructure Lab, is a self hosted infrastructure and automation environment designed to support hands-on learning in DevOps, cybersecurity, artificial intelligence, and systems operations. The environment uses Raspberry Pi systems and an Apple Silicon computer connected through a local home network to create a small scale computing environment where infrastructure, monitoring, automation, and AI related systems can be configured and operated together.

The purpose of the environment is to provide hands-on experience with deployment, monitoring, troubleshooting, automation, logging, security tooling, and operational maintenance through the use of real hardware and locally hosted services. Multiple systems and services run across the shared network, including containerized applications, monitoring tools, log collection systems, AI related services, and automation workflows. The environment also allows operational failures, alert generation, troubleshooting activities, and recovery procedures to be tested and documented over time. All systems connect through a local wired home network using Ethernet connections. The environment runs on existing hardware using free and locally hosted software and services.

SAIL Hardware

SAIL uses the following hardware:

Device	Purpose
Raspberry Pi 4 8GB x3	Kubernetes cluster nodes
MacBook	Administration workstation, AI services, documentation, and automation tools
1TB External SSD	Persistent storage for cluster data, logs, metrics, backups, and documentation assets
Ethernet cables	Network connectivity between systems
Network switch	Local wired network connection between devices

SAIL Storage Design

SAIL uses a single 1TB external SSD as centralized storage for the environment. The storage drive connects to pi-node-1 and stores long term operational and project data generated by the environment.

The centralized storage supports:

- Monitoring data
- Logs
- Kubernetes persistent storage
- AI model files
- Documentation

- Screenshots and diagrams
- Backup files
- Project assets

The Raspberry Pi systems maintain their own operating system storage while the external SSD attached to pi-node-1 stores larger persistent datasets and operational records generated by the environment.

SAIL Network Design

All SAIL systems operate on a local wired network inside the home environment. Each device connects through Ethernet to improve stability and simplify troubleshooting.

The systems use the following names:

System	Role
MacBook	Admin workstation, AI host, documentation, kubectl, Ansible, Git
pi-node-1	k3s control plane, schedulable worker node, and external storage host
pi-node-2	k3s worker
pi-node-3	k3s worker

SAIL Operational Goals

The primary purpose of SAIL is to provide a controlled environment for developing and evaluating operational, security, and AI-assisted investigation workflows.

The environment is intended to support:

- Kubernetes administration and operations
- Infrastructure monitoring and observability
- Centralized logging and event collection
- Runtime security monitoring
- Vulnerability assessment
- Configuration automation
- Failure testing and recovery exercises
- AI-assisted analysis of operational and security events
- Hands-on DevOps and cybersecurity learning

Rather than serving only as a Kubernetes cluster, SAIL functions as a platform for generating, detecting, investigating, and responding to infrastructure and security events. The environment allows operational failures, configuration issues, security alerts, and other events to be intentionally introduced and analyzed using monitoring, logging, security, and AI tooling.

The long-term objective is to develop repeatable workflows that demonstrate how modern infrastructure, security operations, and AI-assisted analysis can be combined within a self-hosted environment.

Section 2: Software choices and architecture decisions

Technology	Purpose	Reason for Selection
k3s/containerd	Kubernetes orchestration and container runtime	Lightweight Kubernetes platform with an integrated container runtime, well suited for Raspberry Pi systems
Ollama	Local AI model hosting	Runs AI models locally on Apple Silicon hardware
Open WebUI	AI web interface	Browser based access to local AI models
Prometheus	Metrics collection	Tracks system and service health
Grafana	Monitoring dashboards	Visualizes metrics and operational data
Loki	Log collection	Centralized log storage and searching
Falco	Runtime security monitoring	Detects suspicious activity in containers and Linux systems
Trivy	Vulnerability scanning	Scans containers and configurations for known issues
Ansible	System configuration automation	Reduces repetitive setup work across systems
Docker	Container image development and testing	Building/testing the Orbit image during development.
GitHub	Documentation and configuration tracking	Stores notes, configurations, diagrams, and troubleshooting records

Architecture Overview

The environment is organized around a Kubernetes cluster running on three Raspberry Pi systems. One Raspberry Pi serves as the k3s control plane and also participates as a worker node, while the remaining Raspberry Pi systems serve as worker nodes. Kubernetes schedules workloads across the available cluster nodes based on resource availability and configuration. The MacBook functions as an administration workstation and hosts AI related services, documentation assets, and management tools used to operate the environment. The environment uses a local wired network and centralized storage provided through an external SSD connected to pi-node-1. All services operate locally using existing hardware and free software.

Section 3: Operating system setup

The SAIL environment uses Raspberry Pi OS Lite (64-bit) on all Raspberry Pi systems. Raspberry Pi OS was selected because it is the operating system maintained specifically for Raspberry Pi hardware and provides broad compatibility with the tools planned for the environment, including k3s, containerd, Prometheus, Grafana, Loki, Falco, Trivy, Docker, and

Ansible. During the planning phase, alternative operating systems were evaluated, including Ubuntu Server. Both options appeared capable of supporting the planned architecture.

Raspberry Pi OS was ultimately selected because the primary objective of SAIL is to provide a stable platform for infrastructure, security, monitoring, automation, and AI-assisted investigation workflows. As the operating system designed specifically for the hardware used in the project, Raspberry Pi OS offers strong hardware support, extensive community documentation, lower resource overhead, and broad compatibility with the planned software stack.

Operating System Installation

The following are performed from a computer connected to the SAIL environment.

Step 1: Download Raspberry Pi Imager

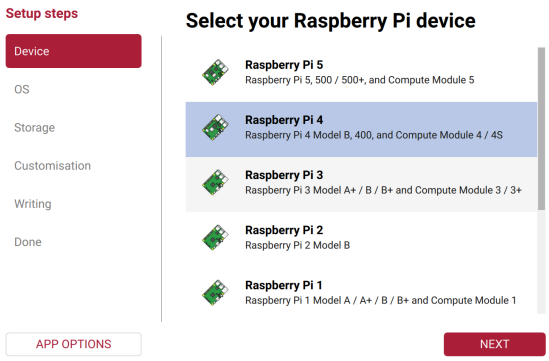
1. Open a web browser. Navigate to: https://www.raspberrypi.com/software/
2. Download Raspberry Pi Imager.
3. Install the application.
4. Launch Raspberry Pi Imager.

Step 2: Prepare the First MicroSD Card

1. Insert a MicroSD card into the computer.
2. Verify the operating system detects the card before proceeding.

Step 3: Select the Raspberry Pi Device

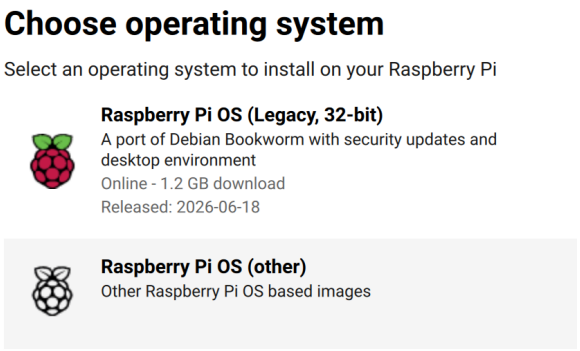
1. Within Raspberry Pi Imager:
    a. Select **Device**
    b. Select **Raspberry Pi 4**



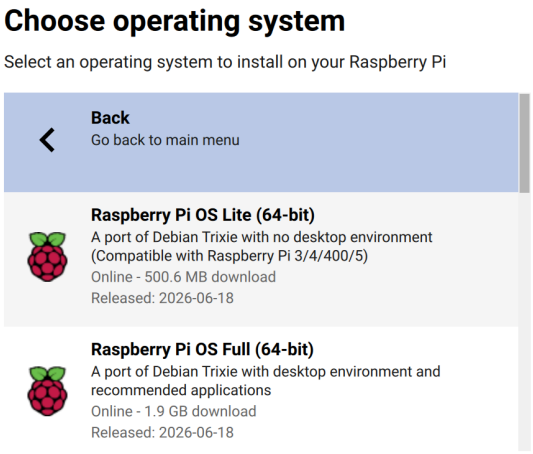
    c. Choose **NEXT**

Step 4: Select the Operating System

1. Select **Raspberry Pi OS (other)**

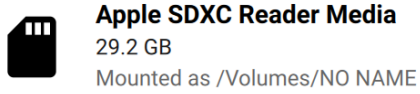


2. Select **Raspberry Pi OS Lite (64-bit)**



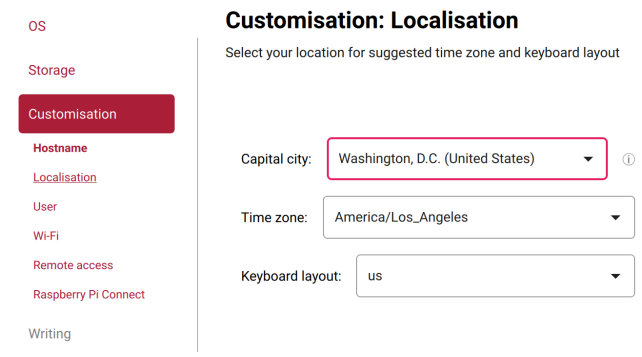
Step 5: Select Storage

1. Select the inserted MicroSD card

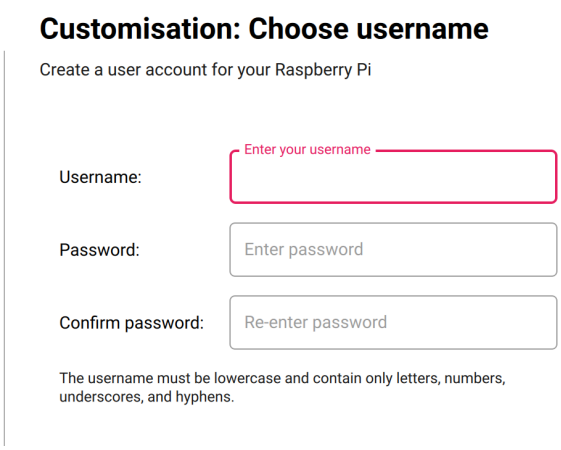


Step 6: Configure Operating System Customization

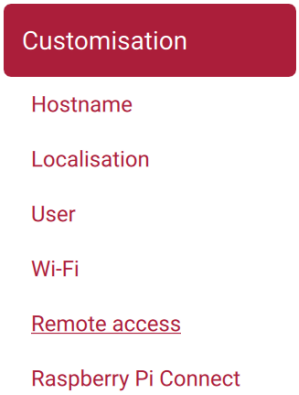
1. Enter Hostname: pi-node-1
2. Enter Capital city, Time zone, and keyboard layout



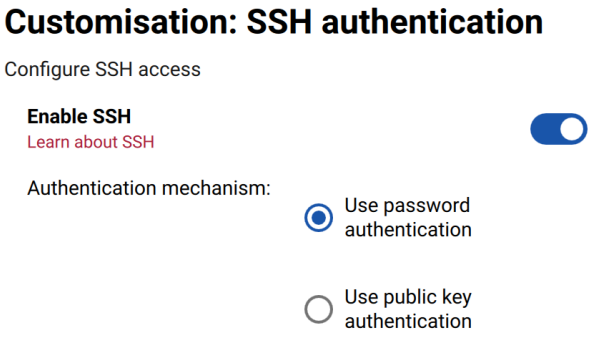
3. Username: <your username>
4. Password: <your password>



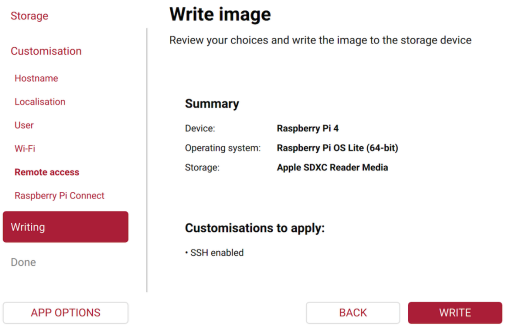
5. Click on Remote Access



6. Enable: SSH



7. Click writing, confirm settings and click **WRITE**



Step 7: Write the Image

1. Wait for image creation and verification to complete.
2. Label the MicroSD card: pi-node-1

Step 8: Create Remaining Node Images

1. Repeat Steps 2 through 7.
2. For the second card use: Hostname: pi-node-2
3. For the third card use: Hostname: pi-node-3
4. Label each completed MicroSD card.

Step 9: Install the MicroSD Cards

1. Insert the appropriate MicroSD card into each Raspberry Pi.
2. Verify that each Raspberry Pi receives the card matching its hostname.

Step 10: Boot the Cluster

1. Verify:
    a. Ethernet cables are connected
    b. The SSD is connected to pi-node-1
    c. Power supplies are connected
2. Apply power to the Raspberry Pi systems.
3. Allow several minutes for the initial boot process to complete.

Step 11: Identify Assigned IP Addresses

1. Log in to the router or gateway administration interface.
2. Locate the Raspberry Pi systems in the connected devices list.
3. Record for each node:
    a. Hostname
    b. MAC Address
    c. IP Address

Step 12: Verify SSH Connectivity

1. Open a terminal on the computer.
2. Connect to pi-node-1: ssh <username>@<ip-address>
    a. Example: ssh sailadmin@10.0.0.20
3. When prompted: Are you sure you want to continue connecting (yes/no/[fingerprint])?
    a. Type: yes
4. Enter the password configured during imaging.
5. Verify a successful login.
6. Repeat for pi-node-2 and pi-node-3.

Step 13: Update Raspberry Pi OS

1. Run: sudo apt update
   sudo apt full-upgrade -y
2. Wait for updates to complete.

Step 14: Reboot the Node

1. Run: sudo reboot
2. Wait approximately one minute.
3. Reconnect using SSH.
4. Repeat Steps 13 and 14 on all three nodes.

Step 15: Verify Hostname Configuration

1. Run: hostname
2. Verify the hostname matches the intended node name.

Step 16: Verify Operating System Installation

1. Run: cat /etc/os-release
2. Verify that Raspberry Pi OS Lite (64-bit) is installed.

Step 17: Record Baseline System Information

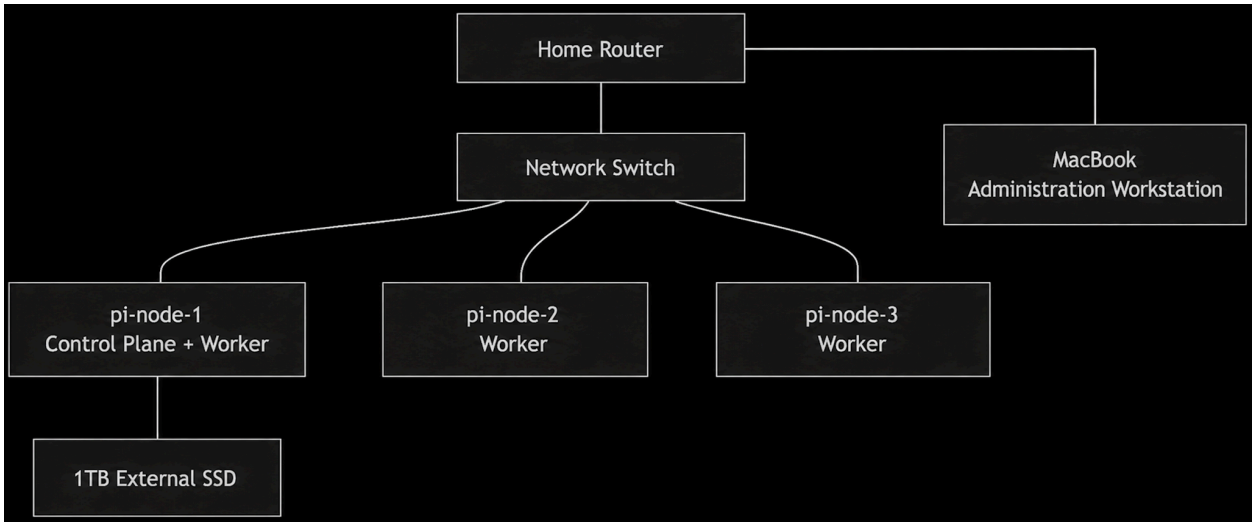
1. Run: free -h
2. Record available memory.
3. Run: df -h
4. Record available storage.This information is used later when evaluating cluster resource utilization and monitoring data.

Operating system installation is complete when:

- All three Raspberry Pi systems boot successfully
- SSH access is verified
- Operating system updates are installed
- Hostnames are configured correctly
- Network connectivity is verified
- Baseline system information has been recorded

Section 4: Physical setup and wiring

Network Topology Diagram



Hardware Layout

Physical Tower Layout

Top

Netgear GS105E

```
┌──────────────────────────┐
│      pi-node-1           │
├──────────────────────────┤
│      pi-node-2           │
├──────────────────────────┤
│      pi-node-3           │
├──────────────────────────┤
│   SSD hard drive         │
│   Anker Charger          │
└──────────────────────────┘
```

Bottom

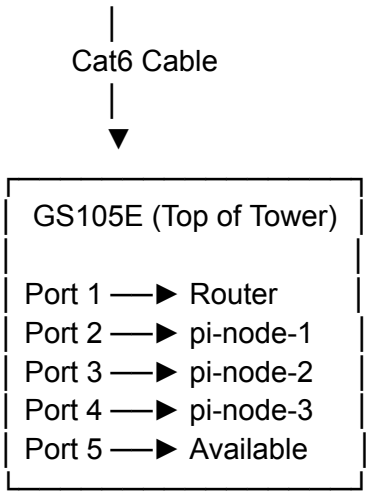
Rear

```
┌──────────────────────────┐
│      Fan 1               │
├──────────────────────────┤
│      Fan 2               │
└──────────────────────────┘
```

Hardware Wiring

Ethernet Wiring

Home Router



Power Wiring

Anker Charger

USB-C #1 ──────────► pi-node-1

USB-C #2 ──────────► pi-node-2

USB-C #3 ──────────► pi-node-3

pi-node-1 USB 3.0 ──► 1TB SSD

Fan 1

Red ──────────► pi-node-1 GPIO Pin 4

Black──────────► pi-node-1 GPIO Pin 6

Fan 2

Red ──────────► pi-node-3 GPIO Pin 4

Black──────────► pi-node-3 GPIO Pin 6

Ethernet Cable Construction

The SAIL environment uses custom Ethernet cables to connect the Raspberry Pi cluster nodes, network switch, and administrative workstation. Building custom cables allows cable lengths to be matched to the physical layout of the environment while providing hands-on experience with basic network infrastructure.

Materials

- Cat5e or Cat6 Ethernet cable

- RJ45 connectors
- RJ45 crimping tool
- Cable stripper
- Cable tester

Step 1: Measure Cable Length

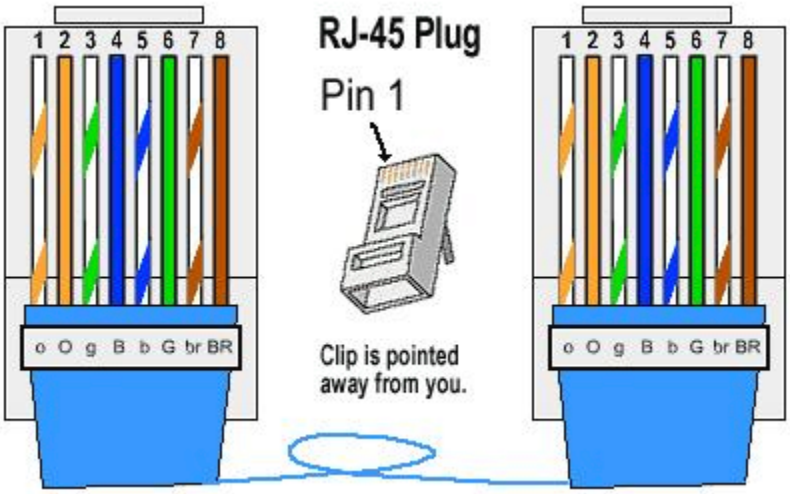
1. Measure the distance between the devices to be connected.
2. Add approximately 12 inches (30 cm) of extra cable to allow for routing and future adjustments.
3. Cut the cable to the desired length.

Step 2: Strip the Cable Jacket

1. Using the cable stripper, remove approximately 1 inch (2.5 cm) of the outer cable jacket from the end of the cable.
2. Take care to avoid damaging the internal conductors.

Step 3: Arrange Conductors

1. Untwist the wire pairs and arrange them according to the T568B standard:
    a. White/Orange
    b. Orange
    c. White/Green
    d. Blue
    e. White/Blue
    f. Green
    g. White/Brown
    h. Brown



2. Flatten the conductors and place them in the correct order.

Step 4: Trim Conductors

1. Hold the conductors firmly and trim them to an equal length approximately 0.5 inches (1.3 cm) from the cable jacket.

Step 5: Insert RJ45 Connector

1. Insert the conductors into the RJ45 connector while maintaining the T568B wire order.

2. Verify that all conductors reach the end of the connector and remain in the correct sequence.
3. Verify that the cable jacket extends into the connector body.

Step 6: Crimp Connector

1. Insert the connector into the crimping tool.
2. Fully compress the crimping tool to secure the conductors and strain relief.

Step 7: Repeat for Opposite End

1. Repeat Steps 2 through 6 using the same T568B wiring order.

Step 8: Test Cable

1. Connect both ends of the cable to a cable tester.
2. Verify continuity on all eight conductors.
3. Replace and recrimp any connector that fails testing.

Verification

1. Connect the completed cable between two network devices.
2. Verify link lights are present on both devices and confirm network connectivity.

Physical Environment Assembly

Materials

- Raspberry Pi 4 (8GB) x3
- Raspberry Pi tower or cluster case
- MicroSD cards
- 1TB external SSD
- Network switch
- Ethernet cable
- RJ45 connectors
- RJ45 crimping tool
- Cable tester
- Raspberry Pi power supplies

Step 1: Prepare the MicroSD Cards

1. Verify that the prepared and labeled MicroSD cards are available for installation.
2. Configure hostname, SSH, username/password, locale, etc.
3. Label each card (`pi-node-1`, `pi-node-2`, `pi-node-3`).

Step 2: Install Storage Media

1. Insert the prepared MicroSD cards into the Raspberry Pi systems.
2. Connect the 1TB external SSD to the node designated as pi-node-1.
3. Verify that all storage devices are seated securely.

Step 3: Assemble the Raspberry Pi Tower

1. Assemble the Raspberry Pi tower according to the Hardware Layout and Hardware Wiring diagrams in this section. Refer to the manufacturer's instructions only for the mechanical assembly of the tower.
2. Install all Raspberry Pi systems into the tower.

3. Install any supplied heatsinks, cooling fans, or mounting hardware.
4. Verify that all components are mounted securely and that cooling components operate without obstruction.

Step 4: Construct Ethernet Cables

1. Measure the required cable lengths based on the planned location of the tower and network switch.
2. Add approximately 12 inches (30 cm) to each measurement to allow for cable routing and future adjustments.
3. Construct and test Ethernet cables using the Ethernet Cable Construction procedure documented earlier in this section.
4. Label cables as desired.

Step 5: Install Network Infrastructure

1. Place the Raspberry Pi tower and network switch in their intended locations.
2. Connect all Raspberry Pi systems to the network switch.
3. Connect the network switch to the local network.
4. Verify that all network connections are fully seated.

Step 6: Connect Power

1. Connect power supplies to the Raspberry Pi systems and network switch.
2. Route cables to minimize strain on connectors and maintain airflow around equipment.

Step 7: Power On the Environment

1. Apply power to the network switch and Raspberry Pi systems.
2. Observe power indicators, network link lights, and cooling operation.
3. Allow the systems to complete their initial startup sequence.

Step 8: Verify Physical Infrastructure

Verify:

- Raspberry Pi tower is assembled correctly
- Storage devices are connected
- Ethernet cables pass testing
- Network link lights are active
- Cooling components are operating
- All systems receive power

Completion Criteria

Physical environment assembly is complete when:

- The Raspberry Pi tower is assembled
- Storage devices are installed
- Network cabling is connected and tested
- Network connectivity is established
- All systems power on successfully
- The environment is ready for operating system installation

Section 5: Network Configuration

Network Planning and IP Address Allocation

Before configuring persistent IP addresses, the local network configuration was reviewed to determine how the cluster nodes would be addressed and managed. Establishing an addressing plan early helps avoid conflicts with existing devices and simplifies SSH access, and cluster administration.

The first step was identifying the address currently assigned to the system. A quick review of assigned addresses was performed using:

```
ifconfig | grep "inet"
```

Because multiple addresses were returned, the active network adapter was identified using:

```
networksetup -listallhardwareports
```

After identifying the active network interface, its assigned address was obtained using:

```
ipconfig getifaddr en0
```

The resulting address was used to determine the address range used by the local network. Readers following this process should note the first three portions of the address, which typically indicate the local network range.

Next, the default gateway was identified using:

```
netstat -nr | grep default
```



The gateway address was entered into a web browser to access the router or gateway administration interface.

Within the administration interface, the local network settings were reviewed to identify:

- The gateway address
- The subnet mask
- The DHCP address range
- Available IP reservation features

The review confirmed that the gateway supported DHCP reservations. DHCP reservations allow specific devices to consistently receive the same IP address while still using DHCP for network configuration. This approach provides the convenience of DHCP while maintaining predictable addresses for cluster infrastructure. Based on the available address space, three addresses were selected and reserved for the Kubernetes nodes. Assigning predictable addresses simplifies administration, automation, and troubleshooting.

Planned DHCP Reservations

To provide predictable node addresses for SSH access, cluster administration, Ansible automation, and troubleshooting, DHCP reservations is configured on the local gateway.

System	Role	Reserved IP
pi-node-1	k3s control plane, worker, SSD host	10.0.0.20
pi-node-2	k3s worker	10.0.0.21
pi-node-3	k3s worker	10.0.0.22

The reservations can be created through the gateway administration interface after the Raspberry Pi systems have been connected to the network and obtained initial DHCP leases.

The reservation process consists of:

1. Log in to the gateway administration interface.
2. Navigate to Connected Devices.
3. Select Add Device With Reserved IP.
4. Identify the Raspberry Pi by hostname, MAC address, or current DHCP address and select it from the device list.
5. Assign the planned IP address.
6. Save the configuration.
7. Repeat for the remaining nodes.
8. Reboot the Raspberry Pi systems or renew their DHCP leases.

Once completed, the cluster nodes consistently receives the same addresses whenever they reconnect to the network.

If there is no direct access to the gateway, have the nodes request the addresses. Check what the automatically assigned addresses are with:

Node	Command	Result IP
pi-node-1	`ping pi-node-1.local`	10.0.0.155
pi-node-2	`ping pi-node-2.local`	10.0.0.119
pi-node-3	`ping pi-node-3.local`	10.0.0.149

To change them to the planned static IPs, start with node 1:

1. SSH into the node: `ssh sailadmin@pi-node-1.local`
2. ID the ethernet connection: nmcli connection show
    a. For all nodes, that connection is `Wired connection 1` with device `eth0`
3. Configure the static IP:

```
sudo nmcli connection modify "Wired connection 1" \
      ipv4.method manual \
      ipv4.addresses 10.0.0.20/24 \
      ipv4.gateway 10.0.0.1 \
      ipv4.dns "10.0.0.1"
```

4. Verify the saved configuration before activating it:

```
nmcli -f ipv4.method,ipv4.addresses,ipv4.gateway,ipv4.dns
connection show "Wired connection 1"
```

   Expected result for node 1:

```
ipv4.method:       manual
ipv4.addresses:    10.0.0.20/24
ipv4.gateway:      10.0.0.1
ipv4.dns:          10.0.0.1
```

5. Activate the new config: `sudo nmcli connection up "Wired connection 1"`
6. Verify the node's new address: `ip -4 addr show eth0`

```
inet 10.0.0.20/24
valid_lft forever preferred_lft forever
```

7. Repeat these steps with nodes 2 & 3
8. Exit ssh and verify all three from the computer:

```
ping -c 2 10.0.0.20
ping -c 2 10.0.0.21
ping -c 2 10.0.0.22
```

IP Address Verification

After configuring the static IP addresses, verify that each node is reachable at its assigned address using the ping commands above. All three nodes should respond from their planned addresses: `10.0.0.20`, `10.0.0.21`, and `10.0.0.22`.

Shared Storage Configuration

The 1TB external SSD is physically connected to **pi-node-1** and provides centralized persistent storage for the SAIL environment. The SSD is shared with the other Raspberry Pi nodes using NFS so that Kubernetes workloads running on any node can access persistent storage over the local network.

Step 1: Identify the SSD on pi-node-1

a. Connect to **pi-node-1**: `ssh sailadmin@10.0.0.20` then `lsusb.` Note the Samsung HD is listed.

```
[sailadmin@pi-node-1:~ $ lsusb
 Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
 Bus 001 Device 002: ID 2109:3431 VIA Labs, Inc. Hub
 Bus 001 Device 007: ID 04e8:4001 Samsung Electronics Co., Ltd PSSD T7
 Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
```

b. List attached block devices and filesystems: `lsblk -f`

```
sailadmin@pi-node-1:~ $ lsblk -f
NAME        FSTYPE FSVER LABEL  UUID                                 FSAVAIL FSUSE% MOUNTPOINTS
loop0       swap   1
sda
└─sda1      exfat  1.0   T7     56F6-5C57
mmcblk0
├─mmcblk0p1 vfat   FAT32 bootfs B2F0-82D2                             428.7M    15% /boot/firmware
└─mmcblk0p2 ext4   1.0   rootfs 15f4c6be-1102-4331-9904-f78e78afd1fd   23.1G    14% /
zram0       swap   1     zram0  3671c3ec-0be0-42a5-92d1-cb35a031fb42                [SWAP]
sailadmin@pi-node-1:~ $
```

c. Identify the 1TB external SSD by its size and device name. It is:

sda
└─sda1 exfat 1.0 T7 56F6-5C57

d. Verify the selected device is the external SSD; not the MicroSD card containing Pi OS.

```
[sailadmin@pi-node-1:~ $ ls -l /dev/sda /dev/sda1
brw-rw---- 1 root disk 8, 0 Aug 25 19:46 /dev/sda
brw-rw---- 1 root disk 8, 1 Aug 25 19:46 /dev/sda1
sailadmin@pi-node-1:~ $
```

e. Record the SSD partition name. It will typically resemble: `/dev/sda1` -**Confirmed**

`/dev/sda` → whole Samsung T7 device
`/dev/sda1` → partition 1 on the T7

Step 2: Prepare the SSD Filesystem

a. Check whether the SSD partition already contains a filesystem:

`sudo blkid /dev/sda1`

```
[sailadmin@pi-node-1:~ $ sudo blkid /dev/sda1
/dev/sda1: LABEL="T7" UUID="56F6-5C57" BLOCK_SIZE="512" TYPE="exfat" PARTUUID="3ee6e1cc-01"
sailadmin@pi-node-1:~ $
```

b. Format the SSD partition as `ext4`:

`sudo mkfs.ext4 -L SAIL_STORAGE /dev/sda1`

```
[sailadmin@pi-node-1:~ $ sudo mkfs.ext4 -L SAIL_STORAGE /dev/sda1
 mke2fs 1.47.2 (1-Jan-2025)
 /dev/sda1 contains a exfat file system labelled 'T7'
[Proceed anyway? (y,N) y
 Creating filesystem with 244190008 4k blocks and 61054976 inodes
 Filesystem UUID: bebf7baf-d72d-4e9a-9d56-3f8cad05fd9a
 Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208,
        4096000, 7962624, 11239424, 20480000, 23887872, 71663616, 78675968,
        102400000, 214990848

 Allocating group tables: done
 Writing inode tables: done
 Creating journal (262144 blocks): done
 Writing superblocks and filesystem accounting information: done

 sailadmin@pi-node-1:~ $
```

c. Verify the filesystem: `lsblk -f`

```
[sailadmin@pi-node-1:~ $ lsblk -f
NAME        FSTYPE FSVER LABEL       UUID                                 FSAVAIL FSUSE% MOUNTPOINTS
loop0       swap   1
sda
└─sda1      ext4   1.0   SAIL_STORAGE bebf7baf-d72d-4e9a-9d56-3f8cad05fd9a
mmcblk0
├─mmcblk0p1 vfat   FAT32 bootfs       B2F0-82D2                            428.7M    15% /boot/firmware
└─mmcblk0p2 ext4   1.0   rootfs       15f4c6be-1102-4331-9904-f78e78afd1fd   23.1G    14% /
zram0       swap   1     zram0        3671c3ec-0be0-42a5-92d1-cb35a031fb42                [SWAP]
sailadmin@pi-node-1:~ $
```

`ext4` provides a native Linux filesystem for the NFS storage.

verified:

- Partition: `/dev/sda1`
- Filesystem: **ext4**
- Label: **SAIL_STORAGE**
- New UUID: `bebf7baf-d72d-4e9a-9d56-3f8cad05fd9a`

Step 3: Create the Permanent SSD Mount Point

a. Create the mount location: `sudo mkdir -p /mnt/sail-storage`

b. Obtain the SSD filesystem UUID: `sudo blkid /dev/sda1`

c. Record the value following `UUID=`bebf7baf-d72d-4e9a-9d56-3f8cad05fd9a

d. Open `/etc/fstab`: `sudo nano /etc/fstab`

e. Add: `UUID=<ssd-uuid> /mnt/sail-storage ext4 defaults,nofail 0 2`

f. Replace `<ssd-uuid>` with the UUID recorded above.

g. Save and exit the file.

h. Mount the filesystems defined in `/etc/fstab`: `sudo mount -a`

```
[sailadmin@pi-node-1:~ $ sudo systemctl daemon-reload
[sailadmin@pi-node-1:~ $ sudo mount -a
 sailadmin@pi-node-1:~ $
```

i. Verify the SSD is mounted: `findmnt /mnt/sail-storage`

```
[sailadmin@pi-node-1:~ $ findmnt /mnt/sail-storage
TARGET            SOURCE    FSTYPE OPTIONS
/mnt/sail-storage /dev/sda1 ext4   rw,relatime,stripe=8191
sailadmin@pi-node-1:~ $
```

j. Verify its available capacity: `df -h /mnt/sail-storage`

```
[sailadmin@pi-node-1:~ $ df -h /mnt/sail-storage
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1       916G  2.1M  870G   1% /mnt/sail-storage
```

Using the UUID rather than `/dev/sda1` prevents the mount from depending on the SSD receiving the same `/dev/sdX` name after every reboot. Confirmed: `/dev/sda1` is mounted at `/mnt/sail-storage`.

Step 4: Create the SAIL NFS Storage Directory

a. Create a directory specifically for Kubernetes persistent storage:

```
sudo mkdir -p /mnt/sail-storage/kubernetes
```

b. Set permissions suitable for the SAIL private cluster:

```
sudo chmod 0777 /mnt/sail-storage/kubernetes
```

c. Verify the directory: `ls -ld /mnt/sail-storage/kubernetes`

```
[sailadmin@pi-node-1:~ $ sudo mkdir -p /mnt/sail-storage/kubernetes
[sailadmin@pi-node-1:~ $ sudo chmod 0777 /mnt/sail-storage/kubernetes
[sailadmin@pi-node-1:~ $ ls -ld /mnt/sail-storage/kubernetes
 drwxrwxrwx 2 root root 4096 Aug 25 20:15 /mnt/sail-storage/kubernetes
```

The broad permissions allow Kubernetes workloads using different container UIDs/GIDs to create storage directories in this isolated lab environment.

Step 5: Install the NFS Server on pi-node-1

a. Update the package index: `sudo apt update`

```
Fetched 27.1 MB in 7s (4,082 kB/s)
98 packages can be upgraded. Run 'apt list --upgradable' to see them.
Notice: Repository 'http://deb.debian.org/debian trixie InRelease' changed its 'Version' value from '13.5' to '13.6'
sailadmin@pi-node-1:~ $
```

b. Install the NFS server: `sudo apt install -y nfs-kernel-server`

```
Creating config file /etc/default/nfs-kernel-server with new version
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for libc-bin (2.41-12+rpt1+deb13u3) ...
sailadmin@pi-node-1:~ $
```

c. Enable and start the service: `sudo systemctl enable --now nfs-kernel-server`

```
[sailadmin@pi-node-1:~ $ sudo systemctl enable --now nfs-kernel-server
Synchronizing state of nfs-kernel-server.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable nfs-kernel-server
```

d. Verify the service: `systemctl is-active nfs-kernel-server`

e. Confirm the result is: `active` - **Confirmed**

Step 6: Export the SAIL Storage Directory

a. Open the NFS exports configuration: `sudo nano /etc/exports`

b. Add: `/mnt/sail-storage/kubernetes 10.0.0.0/24(rw,sync,no_subtree_check)`

c. Save and exit the file. This means

- `/mnt/sail-storage/kubernetes` → directory on the T7 being shared.
- `10.0.0.0/24` → permits clients on the SAIL `10.0.0.x` subnet.
- `rw` → permits reads and writes.
- `sync` → requires writes to be committed before the server reports them complete.
- `no_subtree_check` → disables subtree verification, appropriate here because the exported directory is being used as persistent shared storage.

d. Apply the export configuration: `sudo exportfs -rav`

e. Verify the active export: `sudo exportfs -v`

f. Confirm `/mnt/sail-storage/kubernetes` is exported to `10.0.0.0/24`.

```
[sailadmin@pi-node-1:~ $ sudo nano /etc/exports
[sailadmin@pi-node-1:~ $ sudo exportfs -rav
exporting 10.0.0.0/24:/mnt/sail-storage/kubernetes
[sailadmin@pi-node-1:~ $ sudo exportfs -v
/mnt/sail-storage/kubernetes
                10.0.0.0/24(sync,wdelay,hide,no_subtree_check,sec=sys,rw,secure,root_squash,no_all_squash)
sailadmin@pi-node-1:~ $
```

This restricts access to systems on the SAIL local subnet rather than exposing the storage to arbitrary hosts. **Completed**

Step 7: Configure and Verify NFS Client Access

The `sail-nfs-client-setup.sh` script automates NFS client configuration across all three SAIL nodes by connecting to each over SSH, updating its package index, and installing `nfs-common` so Kubernetes can later mount NFS-backed persistent volumes. The script then temporarily mounts the SSD-backed NFS share on `pi-node-2` and `pi-node-3`, testing write access and cross-node visibility by creating a file on node 2, confirming node 3 can access it, and creating a second file from node 3. This provides a repeatable verification that the

shared-storage path from the cluster nodes through the NFS server on `pi-node-1` to the SSD is functioning as designed.

a. From the VSCode, navigate to the SAIL project directory containing the script:
```
cd <SAIL-project-directory>
```
b. Verify that the script is present: `ls -l sail-nfs-client-setup.sh`
c. Make the script executable: `chmod +x sail-nfs-client-setup.sh`
d. Run the script: `./sail-nfs-client-setup.sh`
e. Enter the `sailadmin` password when authentication requested.
f. Allow the script to complete for all three nodes. Pause if the script reports an error.
g. Confirm the final output reports successful verification of:
- `nfs-common` installation on all three nodes.
- NFS mounting and write access from `pi-node-2`.
- NFS mounting and write access from `pi-node-3`.
- Visibility from `pi-node-3` of the test file created by `pi-node-2`.

```
========================================
NFS client setup and verification complete
========================================
Verified:
  - nfs-common installed on all three nodes
  - pi-node-2 can mount the NFS share
  - pi-node-2 can write to shared storage
  - pi-node-3 can mount the NFS share
  - pi-node-3 can read data written by pi-node-2
  - pi-node-3 can write to shared storage
```

Step 8: Verify Shared Storage from pi-node-1

a. Connect to **pi-node-1**: `ssh sailadmin@10.0.0.20`
b. List the contents of the shared storage directory: `ls -l /mnt/sail-storage/kubernetes`
c. Confirm both test files exist:
```
pi-node-2-test
pi-node-3-test
```

```
[sailadmin@pi-node-1:~ $ ls -l /mnt/sail-storage/kubernetes
total 0
-rw-rw-r-- 1 sailadmin sailadmin 0 Aug 25 20:50 pi-node-2-test
-rw-rw-r-- 1 sailadmin sailadmin 0 Aug 25 20:50 pi-node-3-test
sailadmin@pi-node-1:~ $
```

d. Remove the test files:
```
sudo rm /mnt/sail-storage/kubernetes/pi-node-2-test
sudo rm /mnt/sail-storage/kubernetes/pi-node-3-test
```

Completion Criteria -> Completed

The shared storage configuration is complete when:

- The 1TB SSD is permanently mounted at `/mnt/sail-storage` on `pi-node-1`.
- The Kubernetes storage directory exists at `/mnt/sail-storage/kubernetes`.
- `pi-node-1` exports that directory using NFS.
- The NFS export is restricted to the SAIL `10.0.0.0/24` network.
- `pi-node-2` can mount and write to the NFS share.
- `pi-node-3` can mount and write to the NFS share.
- Files written through one node are visible through the other nodes.

- Test files and temporary mounts have been removed.

Environment Shutdown Procedure

The SAIL environment should be shut down cleanly before disconnecting power. This allows the Raspberry Pi systems to complete disk operations and shut down the operating system before power is removed.

Step 1: Shut Down pi-node-1

1. Open a terminal on the computer.
2. Connect to **pi-node-1**: `ssh sailadmin@10.0.0.20`
3. Shut down the node: `sudo shutdown -h now`
4. Wa it for the SSH connection to close.

Step 2: Shut Down pi-node-2

1. Connect to **pi-node-2**: `ssh sailadmin@10.0.0.21`
2. Shut down the node: `sudo shutdown -h now`
3. Wait for the SSH connection to close.

Step 3: Shut Down pi-node-3

1. Connect to **pi-node-3**: `ssh sailadmin@10.0.0.22`
2. Shut down the node: `sudo shutdown -h now`
3. Wait for the SSH connection to close.

Step 4: Disconnect Tower Power

1. Wait approximately 30 seconds after shutting down the final Raspberry Pi.
2. Unplug the tower from its power source.
3. Verify that the Raspberry Pi systems, cooling fans, and network switch are no longer receiving power.

Section 6: Kubernetes setup

The SAIL environment uses k3s to provide Kubernetes orchestration across the Raspberry Pi cluster. k3s was selected because it is a lightweight Kubernetes distribution designed for environments with limited computing resources while maintaining compatibility with the standard Kubernetes APIs and tooling. This allows SAIL to provide a realistic Kubernetes environment using Raspberry Pi hardware without the overhead of a full Kubernetes installation.

The cluster consists of one control plane node and two worker nodes. The control plane is hosted on pi-node-1, which also participates as a schedulable worker node. The remaining Raspberry Pi systems function as worker nodes that host workloads distributed by Kubernetes. This configuration provides a balance between efficient hardware utilization and a realistic multi-node cluster suitable for infrastructure management, monitoring, security testing, and AI-assisted operational analysis.

Kubernetes Installation

The following procedures prepare the Raspberry Pi systems for k3s, install k3s on the Raspberry Pi cluster, configure Kubernetes administration from a computer, configure persistent storage using the external SSD, and verify that the Kubernetes environment is functioning correctly.

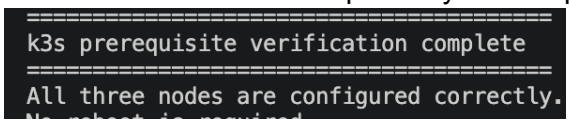
Step 1: Verify Network Connectivity

a. Open a terminal on the computer.
b. Verify that each Raspberry Pi is accessible using SSH: `ssh <username>@10.0.0.20`
c. After successfully connecting, run: exit
d. Repeat for: `ssh <username>@10.0.0.21` and `ssh <username>@10.0.0.22`
e. Resolve any connectivity issues before continuing.
All working - completed

Step 2: Verify and Configure Required Raspberry Pi cgroups

The `sail-k3s-prerequisites.sh` script verifies that all three Raspberry Pi nodes have the memory cgroup configuration required by k3s. It checks each node over SSH, adds only missing parameters, verifies the resulting configuration, and identifies any nodes that must be rebooted before k3s installation.

a. From Terminal, navigate to the SAIL project directory containing the setup scripts:
`cd <SAIL-project-directory>/setup_scripts`
b. Make the prerequisite script executable: `chmod +x sail-k3s-prerequisites.sh`
c. Run the script: `./sail-k3s-prerequisites.sh`
d. Enter the `sailadmin` password when SSH or `sudo` requests authentication.
e. If the script reports that one or more nodes were modified, reboot **only the nodes identified by the script**: `ssh sailadmin@<node-ip> "sudo reboot"`
f. Wait for each rebooted node to become reachable again.
g. Run the prerequisite script again: `./sail-k3s-prerequisites.sh`
h. Confirm the final run reports:
All three nodes were already configured correctly.
No reboot is required by this script.



Step 3: Install and Assemble the k3s Cluster

The `sail-k3s-cluster-setup.sh` script installs the k3s control plane on `pi-node-1`, retrieves the cluster join token, installs the k3s agents on `pi-node-2` and `pi-node-3`, and verifies that all three systems successfully join the Kubernetes cluster. The script preserves the different server and worker installation roles while automating the repeated SSH and installation work, and it stops if installation or cluster readiness verification fails.

a. From the MacBook Terminal, navigate to the SAIL setup-scripts directory:
`cd <SAIL-project-directory>/setup_scripts`
b. Make the cluster setup script executable: `chmod +x sail-k3s-cluster-setup.sh`
c. Run the script: `./sail-k3s-cluster-setup.sh`
d. Enter the `sailadmin` password when SSH or `sudo` requests authentication.

e. Allow the script to install the k3s server on `pi-node-1`, retrieve the cluster join token, and install the worker agents on `pi-node-2` and `pi-node-3`.

f. Confirm the script reaches: `k3s cluster setup complete`

g. Confirm the final output reports that all three nodes joined the cluster and report **Ready**.

```
========================================
Verifying cluster
========================================
NAME        STATUS   ROLES           AGE     VERSION
pi-node-1   Ready    control-plane   14m     v1.36.3+k3s1
pi-node-2   Ready    <none>          7m49s   v1.36.3+k3s1
pi-node-3   Ready    <none>          6m51s   v1.36.3+k3s1

========================================
k3s cluster setup complete
========================================
Verified:
  - k3s server installed on pi-node-1
  - k3s agent installed on pi-node-2
  - k3s agent installed on pi-node-3
  - all three nodes joined the cluster
  - all three nodes report Ready
```

Step 4: Verify Cluster Information

a. Run: `sudo kubectl cluster-info`

b. Verify that the Kubernetes API server is accessible.

```
sailadmin@pi-node-1:~ $ sudo kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:6443
CoreDNS is running at https://127.0.0.1:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
Metrics-server is running at https://127.0.0.1:6443/api/v1/namespaces/kube-system/services/https:metrics-server:https/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

Step 5: Verify Kubernetes System Pods

a. Run: `sudo kubectl get pods --all-namespaces`

b. Verify that all Kubernetes system pods report a status of **Running** or **Completed**.

```
sailadmin@pi-node-1:~ $ sudo kubectl get pods --all-namespaces
NAMESPACE     NAME                                      READY   STATUS      RESTARTS      AGE
kube-system   coredns-54996dc9b4-cmvhx                  1/1     Running     0             20m
kube-system   helm-install-traefik-crd-m5j9c            0/1     Completed   0             19m
kube-system   helm-install-traefik-fq8fn                0/1     Completed   1 (19m ago)   19m
kube-system   local-path-provisioner-58d557dc48-s7jg8   1/1     Running     0             20m
kube-system   metrics-server-6dc596dfb8-726qs           1/1     Running     0             20m
kube-system   svclb-traefik-a496dd55-cvjbp              2/2     Running     0             13m
kube-system   svclb-traefik-a496dd55-hp67j              2/2     Running     0             12m
kube-system   svclb-traefik-a496dd55-s66hl              2/2     Running     0             19m
kube-system   traefik-59b7647586-z6xsr                  1/1     Running     0             19m
```

Step 6: Install the Kubernetes NFS CSI Driver

a. While connected to **pi-node-1**, install the NFS CSI driver:

```
curl -skSL
https://raw.githubusercontent.com/kubernetes-csi/csi-driver-nfs/v4.13.4/depl
oy/install-driver.sh | sudo bash -s v4.13.4 --
```

b. Then verify:

```
sudo kubectl -n kube-system get pods -o wide -l app=csi-nfs-controller
sudo kubectl -n kube-system get pods -o wide -l app=csi-nfs-node
```

```
sailadmin@pi-node-1:~ $ curl -skSL https://raw.githubusercontent.com/kubernetes-csi/csi-driver-nfs/v4.13.4/deploy/install-driver.sh | sudo bash -s v4.13.4 --
Installing NFS CSI driver, version: v4.13.4 ...
serviceaccount/csi-nfs-controller-sa created
serviceaccount/csi-nfs-node-sa created
clusterrole.rbac.authorization.k8s.io/nfs-external-provisioner-role created
clusterrolebinding.rbac.authorization.k8s.io/nfs-csi-provisioner-binding created
clusterrole.rbac.authorization.k8s.io/nfs-external-resizer-role created
clusterrolebinding.rbac.authorization.k8s.io/nfs-csi-resizer-role created
csidriver.storage.k8s.io/nfs.csi.k8s.io created
deployment.apps/csi-nfs-controller created
daemonset.apps/csi-nfs-node created
NFS CSI driver installed successfully.
sailadmin@pi-node-1:~ $ sudo kubectl -n kube-system get pods -o wide -l app=csi-nfs-controller
NAME                                 READY   STATUS              RESTARTS   AGE   IP          NODE        NOMINATED NODE   READINESS GATES
csi-nfs-controller-65b4f9877-5qbjz   0/5     ContainerCreating   0          9s    10.0.0.22   pi-node-3   <none>           <none>
sailadmin@pi-node-1:~ $ sudo kubectl -n kube-system get pods -o wide -l app=csi-nfs-node
NAME                 READY   STATUS              RESTARTS   AGE   IP          NODE        NOMINATED NODE   READINESS GATES
csi-nfs-node-27nvf   0/3     ContainerCreating   0          15s   10.0.0.22   pi-node-3   <none>           <none>
csi-nfs-node-f76m8   0/3     ContainerCreating   0          15s   10.0.0.21   pi-node-2   <none>           <none>
csi-nfs-node-zccfd   0/3     ContainerCreating   0          15s   10.0.0.20   pi-node-1   <none>           <none>
```

c. Wait for the driver components to start.

d. Verify the NFS CSI pods:

```
sudo kubectl -n kube-system get pods -o wide | grep csi-nfs
```

e. Confirm the NFS CSI controller and node components report `Running`.

```
sailadmin@pi-node-1:~ $ sudo kubectl -n kube-system get pods -o wide | grep csi-nfs
csi-nfs-controller-65b4f9877-5qbjz       5/5     Running     0             86s   10.0.0.22   pi-node-3   <none>           <none>
csi-nfs-node-27nvf                       3/3     Running     0             85s   10.0.0.22   pi-node-3   <none>           <none>
csi-nfs-node-f76m8                       3/3     Running     0             85s   10.0.0.21   pi-node-2   <none>           <none>
csi-nfs-node-zccfd                       3/3     Running     0             85s   10.0.0.20   pi-node-1   <none>           <none>
```

The NFS CSI driver allows Kubernetes to dynamically provision persistent volumes from NFS storage.

Step 7: Configure Kubernetes Administration from the Computer

a. Connected to **pi-node-1**, make a temp copy of Kubernetes configuration for `sailadmin`:

```
sudo cp /etc/rancher/k3s/k3s.yaml /home/sailadmin/k3s.yaml
sudo chown sailadmin:sailadmin /home/sailadmin/k3s.yaml
```

b. Disconnect from **pi-node-1**: `exit`

c. On the computer terminal, verify that `kubectl` is installed: `kubectl version --client`

d. If `kubectl` is not installed, install it using the appropriate package manager for the computer's operating system.

e. Create the Kubernetes configuration directory: `mkdir -p ~/.kube`

f. If an existing kubeconfig exists, back it up before continuing:

```
if [ -f ~/.kube/config ]; then
    cp ~/.kube/config ~/.kube/config.backup
fi
```

g. Copy the k3s configuration from **pi-node-1**:

```
scp sailadmin@10.0.0.20:/home/sailadmin/k3s.yaml ~/.kube/config
```

h. Restrict access to the configuration file: `chmod 600 ~/.kube/config`

i. Open `~/.kube/config` in a text editor and change the Kubernetes server address from:

```
https://127.0.0.1:6443
```

to:

```
https://10.0.0.20:6443
```

j. Verify Kubernetes administration from the computer: `kubectl get nodes`

```
NAME        STATUS   ROLES           AGE   VERSION
pi-node-1   Ready    control-plane   45m   v1.36.3+k3s1
pi-node-2   Ready    <none>          38m   v1.36.3+k3s1
pi-node-3   Ready    <none>          37m   v1.36.3+k3s1
```

k. Confirm all three nodes report **Ready**.

l. Remove the temporary configuration copy from **pi-node-1**:

```
ssh sailadmin@10.0.0.20 "rm /home/sailadmin/k3s.yaml"
```

Going forward, Kubernetes administration can be performed directly from the computer, no ssh.

Step 8: Create the SAIL NFS StorageClass

a. Create the StorageClass definition in `sail-nfs-storageclass.yaml`:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: sail-nfs
provisioner: nfs.csi.k8s.io
parameters:
  server: 10.0.0.20
  share: /mnt/sail-storage/kubernetes
reclaimPolicy: Retain
volumeBindingMode: Immediate
allowVolumeExpansion: true
mountOptions:
  - nfsvers=4.1
```

b. Apply the StorageClass: `kubectl apply -f sail-nfs-storageclass.yaml`

c. Verify it: `kubectl get storageclass`

d. Confirm `sail-nfs` appears and its provisioner is: `nfs.csi.k8s.io`

```
sail-nfs        nfs.csi.k8s.io      Retain          Immediate           true                   21s
```

The StorageClass tells Kubernetes where the shared persistent storage is located and which storage driver should provision volumes from it.

Step 9: Verify Kubernetes Persistent Storage

a. Create a test PersistentVolumeClaim `sail-storage-test-pvc.yaml:`

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: sail-storage-test
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: sail-nfs
  resources:
    requests:
      storage: 1Gi
```

b. Apply it: `kubectl apply -f sail-storage-test-pvc.yaml`

c. Verify the PVC: `kubectl get pvc sail-storage-test`

d. Confirm its status reaches: `Bound`

e. Create a temporary pod using the PVC with `sail-storage-test-pod.yaml`:

```
apiVersion: v1
kind: Pod
metadata:
  name: sail-storage-test
spec:
  containers:
    - name: test
      image: busybox
```

```
      command: ["sh", "-c", "echo SAIL-storage-test > /data/test.txt &&
sleep 3600"]
      volumeMounts:
        - name: sail-storage
          mountPath: /data
  volumes:
    - name: sail-storage
      persistentVolumeClaim:
        claimName: sail-storage-test
```

f. Apply the pod: `kubectl apply -f sail-storage-test-pod.yaml`

g. Verify the pod reaches `Running`: `kubectl get pod sail-storage-test`

h. Write test data to the persistent volume:

```
kubectl exec sail-storage-test -- sh -c 'echo SAIL-storage-test > /data/test.txt'
```

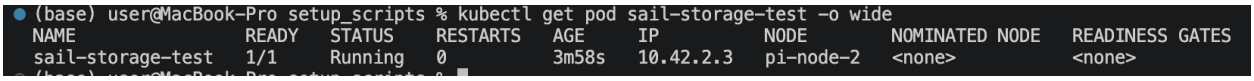
i. Verify the data written to persistent storage:

```
kubectl exec sail-storage-test -- cat /data/test.txt
```

Confirm: `SAIL-storage-test` - **Confirmed**

j. Determine which Kubernetes node is running the pod:

```
kubectl get pod sail-storage-test -o wide
```



k. Delete the pod: `kubectl delete pod sail-storage-test`

l. Recreate it using the same PVC:

```
kubectl apply -f sail-storage-test-pod.yaml
```

m. Wait until the recreated pod reaches `Running`:

```
kubectl get pod sail-storage-test
```

n. Verify the original data remains available:

```
kubectl exec sail-storage-test -- cat /data/test.txt
```

o. Confirm: `SAIL-storage-test` - **Confirmed**

Verifies that the data exists independently of the individual pod and is available after pod deletion and recreation, to support the later persistence requirement.

p. Delete the temporary pod: `kubectl delete pod sail-storage-test`

q. Delete the test PVC: `kubectl delete pvc sail-storage-test`

r. Remove the temporary test YAML files:

```
rm sail-storage-test-pod.yaml
rm sail-storage-test-pvc.yaml
```

s. Keep `sail-nfs-storageclass.yaml` because `sail-nfs` is part of the permanent SAIL Kubernetes configuration.

t. Verify the StorageClass remains available: `sudo kubectl get storageclass`

u. Confirm `sail-nfs` remains listed. **Confirmed**

That confirms:

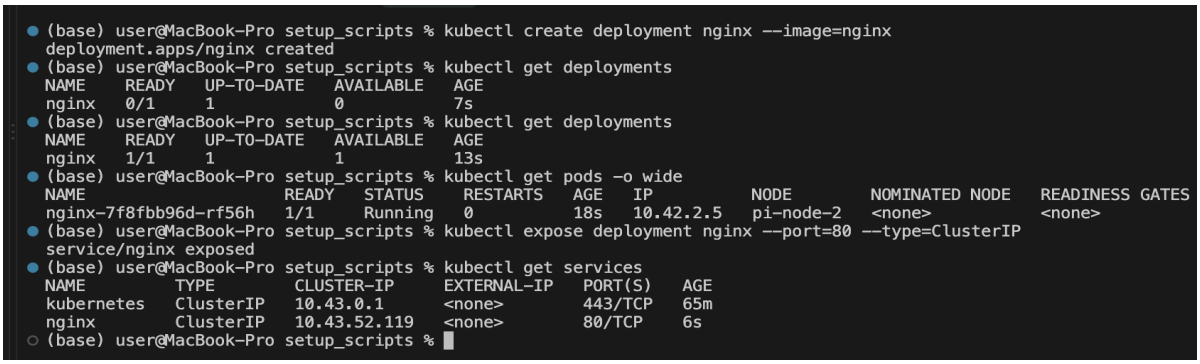
- the `sail-nfs` StorageClass can provision a PVC;
- the test pod can mount and write to the NFS-backed volume;
- the data remained after the pod was deleted and recreated;
- the temporary test resources were removed;
- the permanent `sail-nfs` StorageClass remains available.

Step 10: Deploy and Verify a Test Workload and Service

a. Create an nginx deployment:`kubectl create deployment nginx --image=nginx`

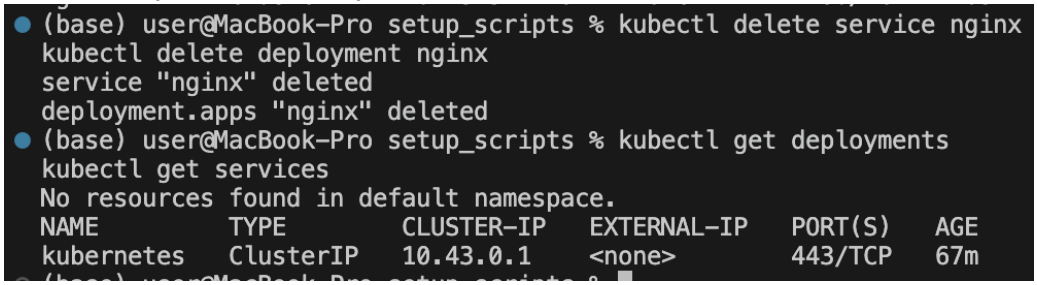
b. Verify the deployment: `kubectl get deployments`

c. Verify the pod and determine which node is running it: `kubectl get pods -o wide`
d. Confirm the nginx pod reaches **Running**.
e. Create a ClusterIP service:
```
kubectl expose deployment nginx --port=80 --type=ClusterIP
```
f. Verify the service: `kubectl get services`
g. Confirm an `nginx` service appears with type **ClusterIP**.



Step 11: Remove the Test Workload

a. Run: `sudo kubectl delete service nginx`
`sudo kubectl delete deployment nginx`
b. Verify that the deployment and service have been removed.



Completion Criteria

The Kubernetes installation is complete when:

- k3s is installed on all three Raspberry Pi systems.
- All three nodes report **Ready**.
- Kubernetes system components are healthy.
- Kubernetes administration from the computer is functioning.
- The NFS CSI controller and node components are running.
- The `sail-nfs` StorageClass is available.
- Kubernetes successfully provisions storage from the SSD-backed NFS share.
- Persistent data remains available after a pod is deleted and recreated.
- A test workload deploys successfully.
- A ClusterIP service is created successfully.
- The test deployment and service are removed successfully.
- Baseline cluster information has been verified.

Section 7: Workload Design

SAIL operates multiple workloads to create an active environment for infrastructure, security, automation, and AI-assisted operations. These workloads generate ongoing activity that can be

monitored and analyzed while also providing opportunities to test failures, security events, investigation, and recovery.
The environment includes four primary workloads:

- **Application workload:** Containerized applications generate API requests, authentication activity, data processing, storage operations, errors, and other application behavior.
- **Security workload:** Security tools assess systems and assets for vulnerabilities and security weaknesses while monitoring the running environment for security-relevant behavior. Security activity produces findings and detections that can be investigated using evidence collected throughout SAIL.
- **Automated/AI workload:** Automated processes and AI agents interact with applications and services while the environment is running. These processes create sustained activity, generate different usage and security patterns, and support investigation and remediation workflows.
- **Operational workload:** Kubernetes, containers, hosts, storage, and networking generate activity through deployments, scheduled processes, configuration changes, resource usage, failures, troubleshooting, and recovery operations.

These workloads create the data used throughout SAIL. Monitoring and security systems collect logs, metrics, traces, vulnerability findings, runtime detections, application records, and system information that can be used during troubleshooting and security investigations.

Application Workload

The application workload provides functioning services for SAIL to host and operate. Applications run as containerized workloads on the Kubernetes cluster and provide real targets for deployment, monitoring, security assessment, automated activity, troubleshooting, and investigation.
The application workload includes:

1. **Interactive application activity:** Applications expose APIs or other interfaces that can receive requests and perform useful processing. Requests can involve authentication, authorization, data access, application logic, and persistent storage.
2. **Persistent application state:** Application data remains available across container and pod restarts. Database activity also provides another service dependency that can be monitored, secured, disrupted, and recovered.
3. **Security-relevant application behavior:** Authentication attempts, access-control decisions, input validation, privileged operations, configuration changes, errors, and other significant actions produce records that can be correlated with infrastructure and security telemetry.
4. **External asset interaction:** Application components can interact with authorized external assets when a workload requires information outside the SAIL cluster. This allows security-oriented services to perform useful work against assets such as owned websites or repositories rather than operating only against simulated targets.

The exact functionality of the initial application is determined after the complete SAIL workload is defined. Its design needs to provide useful application behavior while also supplying enough activity and security context for the other SAIL workloads.

Security Workload

The security workload provides SAIL with active security assessment, detection, and investigation capabilities.

1. **Assessment:** SAIL examines systems and assets for vulnerabilities and security weaknesses. Assessments can cover SAIL infrastructure and applications along with external assets owned or administered by the operator. Assessments can identify vulnerable software, exposed services, insecure configurations, application weaknesses, and other security findings. Results are recorded so findings can be reviewed, investigated, remediated, and compared with later assessments.
2. **Detection:** SAIL monitors running systems and applications for security-relevant behavior. This includes authentication failures, unauthorized access attempts, unexpected processes, container activity, configuration changes, suspicious requests, unusual application activity, and other behaviors identified by the security tooling.
3. **Investigation:** Security findings and detections are supported with evidence collected across the environment. Investigations can use application activity, system state, vulnerability results, logs, metrics, traces, runtime security information, and related records to determine what occurred and assess its significance. AI-assisted investigation capabilities can use this evidence to correlate activity and support analysis.
4. **Verification:** Security assessments can be repeated after remediation to determine whether an identified weakness has been corrected. Detection and monitoring data can also confirm whether affected systems have returned to their expected state.

The security workflow follows:

Assess → Detect → Collect Evidence → Investigate → Remediate → Verify

This workflow provides a common process for handling security findings and events throughout SAIL. It connects the security workload with the application, automated/AI, and operational workloads so activity occurring in one part of the environment can be examined using evidence produced elsewhere in the system.

Automated/AI Workload

The automated/AI workload provides sustained activity and AI-assisted operational capabilities across SAIL. Automation performs repeatable work while agents make bounded decisions based on the state of the environment and the information available to them.

The workload includes three agent roles:

1. **Activity Agent:** Generates sustained interaction with applications and services. The agent can represent different patterns of normal and security-relevant behavior, vary the frequency and sequence of its actions, and create enough activity for monitoring systems to establish patterns and detect changes.
2. **Investigation Agent:** Examines security findings, alerts, and operational problems by retrieving relevant evidence from SAIL. The agent can correlate application records, logs, metrics, traces, runtime detections, vulnerability results, and current system state to develop an evidence-supported assessment.

3. **Remediation Agent:** Uses investigation results to prepare corrective actions. Remediation activity is recorded and followed by verification so the outcome can be compared with the original finding or incident.

Agent activity is also observable. Requests, tool calls, decisions, investigation inputs, outputs, and remediation actions generate records that can be reviewed alongside the systems the agents operate against.

Operational Workload

The operational workload represents the normal administration and behavior of the SAIL infrastructure. It provides the system-level activity required to operate applications and security services across the physical hosts and Kubernetes cluster.

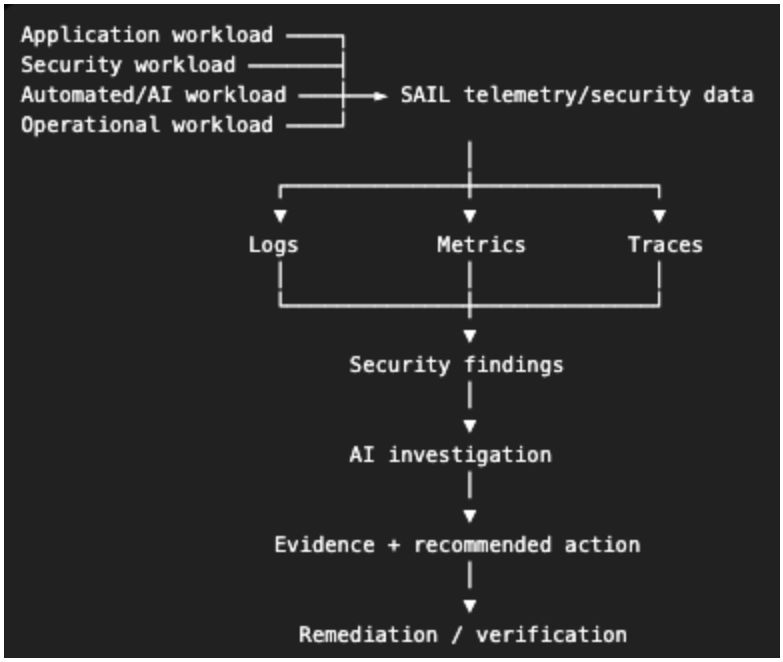
The workload includes:

1. **Cluster operations:** Kubernetes schedules and manages application, security, monitoring, automation, and AI workloads across the available nodes. Deployments, pod creation, restarts, scheduling decisions, and service changes generate operational activity.
2. **System operations:** Hosts generate CPU, memory, storage, network, process, and service activity while SAIL is running. Changes in these resources provide infrastructure-level context for application and security events.
3. **Deployment and configuration activity:** Application releases, container images, infrastructure configuration, Kubernetes resources, and automated configuration changes create a record of how the environment changes over time.
4. **Failure and recovery activity:** Service outages, pod failures, resource pressure, storage problems, network problems, and configuration errors provide scenarios for detection and troubleshooting. Recovery activity provides evidence that corrective actions restored the affected system.

Operational data provides context for determining whether an application or security event is isolated or connected to a broader change in the environment.

Integrated Workload Flow

The four workloads operate as parts of the same environment rather than as separate demonstrations. Application and operational workloads produce normal system activity. Security systems assess assets and monitor the environment. Automated processes and AI agents interact with these systems and generate additional activity.



Section 8: Application Development and Deployment

Application Design

Orbit is a social media application that provides a realistic workload for SAIL while supporting continued development as an independent application. Users can create accounts, authenticate, maintain profiles, publish posts, comment and react to posts, follow other users, and view a chronological feed. User and administrative permissions control access to application functions and data. Orbit exposes its functionality through a REST API and stores persistent data in PostgreSQL.

Automated users can generate substantial activity by exercising authentication, content creation, feed retrieval, social relationships, authorization checks, database operations, and concurrent API requests. The volume and frequency of these interactions can be varied to create different workload levels for monitoring application performance and infrastructure behavior. Orbit also generates security-relevant activity through authentication attempts, permission checks, administrative operations, validation failures, user actions, and application errors.

The SAIL implementation focuses on the functionality needed to provide a useful application workload without requiring the complete feature set of a production social media platform. Application activity can be correlated with monitoring and security data collected elsewhere in SAIL, while the initial architecture provides a foundation for continued development of Orbit after SAIL is complete.

The SAIL implementation of Orbit includes:

- User registration and authentication.
- User profiles and account management.
- User and administrative roles and permissions.
- Creation, editing, deletion, and retrieval of posts.
- Comments and reactions.
- Follow and unfollow relationships.
- A chronological user feed.
- REST API access and request validation.
- Persistent application data.
- Application and security-relevant logging.
- Health and application status endpoints.
- Support for sustained automated user activity.

The application uses:

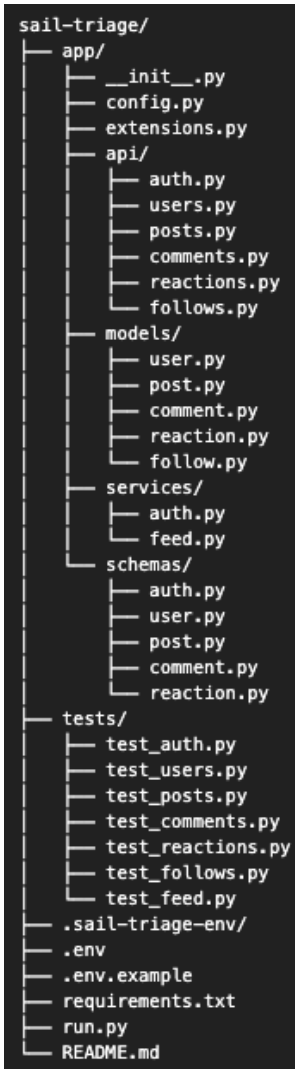
Technology	Purpose
Python	Programming language
FastAPI	Web/API framework
Pydantic	Request/response validation and schemas
Uvicorn	Runs the FastAPI application
PostgreSQL	Application database
SQLAlchemy	ORM/database access
psycopg	Connects the application to PostgreSQL
Alembic	Database schema migrations
PyJWT	Authentication tokens
pwdlib / Argon2	Password hashing
pytest	Automated testing
Black	Code formatting
Ruff	Linting/code quality
dotenv	Loads application configuration from .env files into env variables.

Frontend	Purpose
React	Frontend user interface
Vite	Frontend development server and build tooling
JavaScript	Frontend application logic

HTML	Interface structure
Tailwind CSS	Interface styling and responsive layouts
shadcn/ui	Reusable user interface components

Application Structure

The `api` package handles REST endpoints, `models` contains database models, `services` contains application logic that does not belong directly in an endpoint, `schemas` handles request and response validation, and `tests` contains automated tests for the application.



Development follows standard Python practices, including PEP 8 conventions, type hints where appropriate, automated formatting and linting, separation of application responsibilities, and automated testing.

Application Behavior

At a high level, Orbit supports the following application workflow:

1. **Register and authenticate users** using secure authentication tokens.
2. **Manage accounts and profiles** with user and administrative permissions.
3. **Create and manage posts** including publishing, retrieving, editing, and deleting.
4. **Interact with content** through comments and reactions.
5. **Manage social relationships** by categorizing connections based on relationship type or closeness.
6. **Generate a chronological feed** containing content the authenticated user is authorized to view.
7. **Validate requests and enforce authorization** before accessing protected functions or data.
8. **Store and retrieve application data** through PostgreSQL.
9. **Record security-relevant activity** including authentication, permission checks, administrative actions, validation failures, and errors.
10. **Report application health** for monitoring Orbit and its dependencies.
11. **Support sustained automated activity** at varying workload levels.

Application Deployment

The application completes in the following order:

1. Create the application folder and virtual environment. **Complete**
2. Configure VS Code to use the project environment. **Complete**
3. Configure Git for the application. **Complete**
4. Install the initial application and development dependencies. **Complete**
5. Create the Application Project Structure. **Complete**
6. Configure application settings and environment variables. **Complete**
7. Configure PostgreSQL for local development. **Complete**
8. Configure SQLAlchemy and database connectivity. **Complete**
9. Configure Alembic database migrations. **Complete**
10. Implement the user data model and database schema. **Complete**
11. Implement user registration, authentication, and password security. **Complete**
12. Implement user profiles, roles, and permissions. **Complete**
13. Run and manually verify the current backend locally. **Complete**
14. Create the Orbit frontend structure and connect it to the FastAPI backend. **Complete**
15. Build registration and login screens using the existing authentication API. **Complete**
16. GitHub Actions for Black, Ruff, and frontend ESLint **Complete**
17. Build user profile and account screens **Complete**
18. Build posts and associated API endpoints, schemas, frontend functionality, and tests.
19. Add comments and reactions, i.e. data models, API endpoints, schemas, frontend functionality, and tests.

20. Add follow and unfollow relationships, i.e. data models, API endpoints, schemas, frontend functionality, and tests.
21. Build the chronological feed, including backend feed logic, API endpoints, schemas, frontend functionality, and tests.
22. Complete frontend navigation and remaining user-interface behavior.
23. Add application and security-relevant logging.
24. Add health endpoints and application error handling.
25. Complete remaining automated and integration testing.
26. Run and test the complete application locally, i.e. frontend, FastAPI, and PostgreSQL.
27. Verify FastAPI validation and OpenAPI documentation.
28. Generate sustained automated user activity and verify Orbit under a variable workload.
29. Containerize the completed application.
30. Deploy Orbit and PostgreSQL to the k3s cluster.
31. Verify the deployed application is reachable, persistent, and functioning correctly.

Create the Application Environment

Step 1: Create the application folder and virtual environment

Inside the existing `SAIL` repository, create the application folder:

```
SAIL/
├── README.md
├── docs/
└── sail-orbit/
```

a. In Terminal, from the root of the SAIL repo:

```
mkdir -p sail-orbit
cd sail-orbit
```

b. Create the virtual environment:

```
python3 -m venv .sail-orbit-env
```

c. Activate it:

```
source .sail-orbit-env/bin/activate
```

d. Verify it:

```
which python
python --version
```

The `which python` result should point to something like:

```
.../SAIL/sail-orbit/.sail-orbit-env/bin/python
```

At this point, the only thing created automatically inside `sail-orbit` is:

```
sail-orbit/
└── .sail-orbit-env/
```

Step 2: Configure VS Code to Use the Environment

a. Press `Cmd + Shift + P`.
b. Search for:

```
Python: Select Interpreter
```

c. Select: Enter interpreter path → Find.
d. Select:

```
sail-orbit/.sail-orbit-env/bin/python
```

Step 3: Configure Git for the Application

Since `SAIL` is already a Git repository, do not run `git init` again.

a. From inside `sail-orbit`, confirm are inside the existing repository:

```
git status
```

b. Add the virtual environment to the repository's `.gitignore`.

From the SAIL repository root, open or create `.gitignore` and add:

```
sail-orbit/.sail-orbit-env/
```

c. Verify Git is ignoring it:

```
git status
```

d. Create a development branch for the application work:

```
git switch -c develop
```

e. Verify the branch:

```
git branch
```

Should see:

```
* develop
main
```

f. in a .gitignore at the file base level:

```
# Python virtual environment
sail-orbit/.sail-orbit-env/

# Python generated files
__pycache__/
*.py[cod]

# Testing and tooling caches
.pytest_cache/
.ruff_cache/
.coverage
htmlcov/

# Environment variables and secrets
.env
.env.*
!.env.example

# macOS
.DS_Store

# VS Code local settings
.vscode/

# Logs
*.log
```

g. Make an initial commit for the application folder/setup once the structure wanted tracked exists. For now, if `sail-orbit` contains only the ignored virtual environment, there is nothing useful to commit yet.

Step 4: Install Application and Development Dependencies

With `.sail-orbit-env` activated and the terminal inside `sail-orbit`:

a. Create the requirements file:

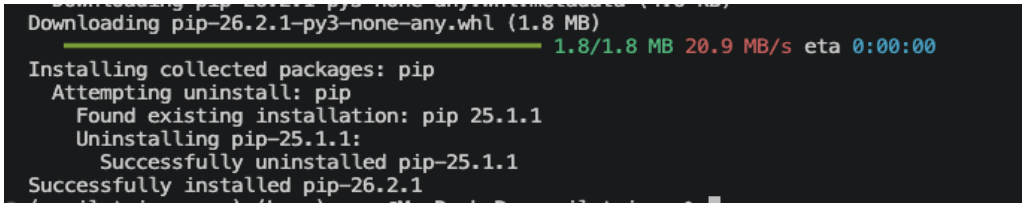
```
touch requirements.txt
```

b. Open `requirements.txt` in VS Code and add:

```
Flask
Flask-RESTX
pytest
black
ruff
```

c. Upgrade `pip`:

```
python -m pip install --upgrade pip
```



d. Install the dependencies from the requirements file:

```
pip install -r requirements.txt
```



e. Verify the installed packages:

```
pip list
```

Confirm that the list includes:

```
fastapi
pydantic
uvicorn
sqlalchemy[asyncio]
psycopg[binary]
alembic
pyjwt
pwdlib[argon2]
pytest
black
ruff
```

Once verified, put the version numbers in the requirements.txt.

f. Verify that Git now sees the application directory and requirements file:

```
git status
```

Should see `sail-orbit/requirements.txt` as an untracked file. The `.sail-orbit-env` directory should **not** appear because it is ignored.

Step 5: Create the Application Project Structure

a. From inside `sail-orbit`, create the directories:

```
mkdir -p app/api app/models app/services app/schemas tests
```

b. Create the core application files:

```
touch app/__init__.py \
      app/config.py \
      app/extensions.py \
      run.py
```

c. Create the application README:

```
touch README.md
```

d. Create the Orbit API files:

```
touch app/api/auth.py \
      app/api/users.py \
      app/api/posts.py \
      app/api/comments.py \
      app/api/reactions.py \
      app/api/follows.py
```

e. Create the database model files:

```
touch app/models/user.py \
      app/models/post.py \
      app/models/comment.py \
      app/models/reaction.py \
      app/models/follow.py
```

f. Create the service files:

```
touch app/services/auth.py \
      app/services/feed.py
```

g. Create the Pydantic schema files:

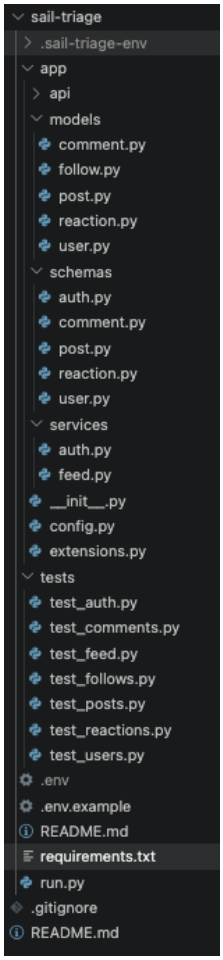
```
touch app/schemas/auth.py \
      app/schemas/user.py \
      app/schemas/post.py \
      app/schemas/comment.py \
      app/schemas/reaction.py
```

h. Create the test files:

```
touch tests/test_auth.py \
      tests/test_users.py \
      tests/test_posts.py \
      tests/test_comments.py \
      tests/test_reactions.py \
      tests/test_follows.py \
      tests/test_feed.py
```

i. Verify the structure: `find app tests -maxdepth 2 -type f | sort`

The structure should now be:



Step 6: Configure Application Settings and Environment Variables

a. Create an environment file for local development:

```
touch .env
```

b. Create a safe template that can be committed to Git:

```
touch .env.example
```

c. In `.env`, add the local application settings we know we will need:

```
FLASK_ENV=development
FLASK_DEBUG=1
```

d. Put the same variable names in `.env.example`, but without any sensitive values:

```
FLASK_ENV=development
FLASK_DEBUG=1
```

e. Confirm the root `.gitignore` includes:

```
.env
.env.*
!.env.example
```

f. In `app/config.py`, we will later load configuration from environment variables rather than hardcoding it into the application.

Step 7: Configure PostgreSQL for Development

a. First, check whether PostgreSQL is already installed on the computer:

```
psql --version
```

b. If `psql` is not found, install PostgreSQL with Homebrew:

```
brew install postgresql@17
```

```
Success. You can now start the database server using:

    '/opt/homebrew/Cellar/postgresql@17/17.11/bin/pg_ctl' -D '/opt/homebrew/var/postgresql@17' -l logfile start
```

c. Start PostgreSQL:

```
brew services start postgresql@17
```

```
(.sail-orbit-env) (base) user@MacBook-Pro sail-orbit % brew services start postgresql@17
==> Successfully started `postgresql@17` (label: homebrew.mxcl.postgresql@17)
```

d. Verify the server is running:

```
pg_isready
```

Response should be similar to:

```
/var/run/postgresql:5432 - accepting connections
```

```
(.sail-orbit-env) (base) user@MacBook-Pro sail-orbit % pg_isready
/tmp:5432 - accepting connections
```

e. Create a PostgreSQL database for Orbit:

```
createdb orbit
```

f. Verify that it exists:

```
psql -l
```

Should see `orbit` in the database list.

```
postgres=# \l
                                                List of databases
   Name    |   Owner    | Encoding | Locale Provider |   Collate   |    Ctype    | Locale | ICU Rules | Access privileges
-----------+------------+----------+-----------------+-------------+-------------+--------+-----------+-------------------
 orbit     | triage_app | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |        |           |
 postgres  | user       | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |        |           |
 template0 | user       | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |        |           | =c/user          +
           |            |          |                 |             |             |        |           | user=CTc/user
 template1 | user       | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |        |           | =c/user          +
           |            |          |                 |             |             |        |           | user=CTc/user
(4 rows)
```

g. Open the PostgreSQL prompt:

```
psql postgres
```

Should get a prompt similar to:

```
postgres=#
```

h. Create a Orbit database user. Replace `your_password_here` with a chosen password:

```
CREATE USER orbit_app WITH PASSWORD 'your_password_here';
```

i. Make `orbit_app` the owner of the `orbit` database:

```
ALTER DATABASE orbit OWNER TO orbit_app;
```

j. Connect to the `orbit` database:

```
\c orbit
```

k. Make `orbit_app` the owner of the `public` schema (Alembic to create/modify tables later):

```
ALTER SCHEMA public OWNER TO  orbit_app;
```

l. Exit PostgreSQL:

```
\q
```

m. Test the new application account:

```
psql -U orbit_app -d orbit -h localhost
```

It should ask for the password just created. After entering it, should reach:

```
orbit=>
```

Then run:

```
SELECT current_user, current_database();
```

```
orbit=> SELECT current_user, current_database();
 current_user | current_database
--------------+------------------
 orbit_app    | orbit
(1 row)
```

n. Now exit PostgreSQL:

```
\q
```

o. Then update `DATABASE_URL` in the `.env`:

```
DATABASE_URL=postgresql+psycopg://orbit_app:orbit_et@localhost:5432/orbit
```

Keep the password only in `.env`; `.env.example` should continue to have:
DATABASE_URL=

p. To shut down the database:

```
brew services stop postgresql@17
```

Verify it stopped:

```
pg_isready
```

Should get something like:

```
localhost:5432 - no response
```

Step 8: Configure SQLAlchemy and database connectivity.

a. Open:

```
app/extensions.py
```

Add:

```
"""Application extensions for Orbit."""

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from app.config import Config


engine = create_engine(
   Config.DATABASE_URL,
   pool_pre_ping=True,
)

SessionLocal = sessionmaker(
   bind=engine,
   autoflush=False,
   autocommit=False,
)
```

b. Create a small database test file:

```
touch app/database_test.py
```

c. Add:

```
"""Verify Orbit can connect to the configured database."""

from sqlalchemy import text

from app.extensions import engine


def test_database_connection():
   """Connect to PostgreSQL and return the current database
user."""

   with engine.connect() as connection:
       result = connection.execute(
           text("SELECT current_user, current_database();")
       )

       return result.fetchone()

if __name__ == "__main__":
   print(test_database_connection())
```

d. Run it from inside `sail-orbit` with the virtual environment active:

```
python -m app.database_test
```

Expected output should look similar to:

```
('orbit_app', 'orbit')
```

e. More the file to tests and rename to a better name for explanation

```
mv app/database_test.py tests/test_database_connection.py
```

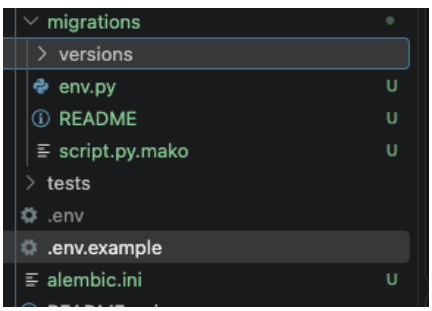
Step 9: Configure Alembic database migrations

a. From inside `sail-orbit`, initialize Alembic:

```
alembic init migrations
```

This creates:

```
alembic.ini
migrations/
├── env.py
├── script.py.mako
└── versions/
```



b. Open:

`alembic.ini`
Find:
`sqlalchemy.url =` -> Make it blank.
Want Alembic to use the same `DATABASE_URL` from `.env`

c. Open:
`migrations/env.py`
Near the top, add:
```
from app.config import Config
```
Then find:
```
config = context.config
```
Immediately after it, add:
```
config.set_main_option("sqlalchemy.url",
Config.DATABASE_URL)
```

d. For now, leave:
`target_metadata = None`
Haven't yet created the SQLAlchemy models. In the next step, replace this with the app's model metadata so Alembic can detect schema changes automatically.

e. Verify Alembic can connect to PostgreSQL:
`alembic current`

```
(.sail-triage-env) (base) user@MacBook-Pro sail-triage % alembic current
INFO  [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO  [alembic.runtime.migration] Will assume transactional DDL.
```

f. Verify Alembic itself is configured:
`alembic heads`

Alembic is initialized, is reading the Orbit database configuration, and can connect to PostgreSQL successfully.

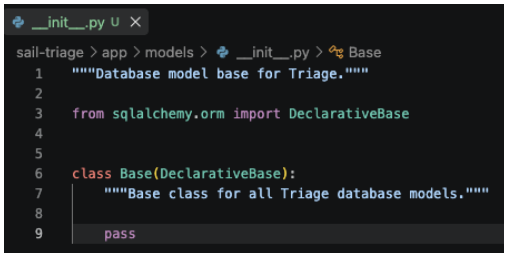
Step 10: Implement the user data model and database schema

Define the SQLAlchemy base, implement this model in `app/models/user.py`, expose its metadata to Alembic, generate the first migration, apply it, and verify the `users` table was actually created in PostgreSQL.

a. Create a shared SQLAlchemy base in `app/models/__init__.py`:
`touch app/models/__init__.py`

Add:
```
"""Database model base for Orbit."""
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
    """Base class for all Orbit database models."""
    pass
```



b. Open `app/models/user.py` and add:

```
"""User database model for Orbit."""
from datetime import datetime, timezone
from sqlalchemy import Boolean, DateTime, String, Text
from sqlalchemy.orm import Mapped, mapped_column
from app.models import Base

class User(Base):
    """Represent a Orbit user account."""
    # Name of the PostgreSQL table represented by this model.
    __tablename__ = "users"
    # Primary key used internally to uniquely identify each user.
    id: Mapped[int] = mapped_column(primary_key=True)
    # Account identity fields. Usernames and email addresses must be unique.
    username: Mapped[str] = mapped_column(
        String(50),
        unique=True,
        nullable=False,
    )
    email: Mapped[str] = mapped_column(
        String(255),
        unique=True,
        nullable=False,
    )
    # Store the password hash rather than the user's actual password.
    password_hash: Mapped[str] = mapped_column(
        String(255),
        nullable=False,
    )
    # Public profile information.
    display_name: Mapped[str] = mapped_column(
        String(100),
        nullable=False,
    )
    bio: Mapped[str | None] = mapped_column(
        Text,
        nullable=True,
    )
    # Control account permissions and whether the account can be used.
    role: Mapped[str] = mapped_column(
        String(20),
        default="user",
        nullable=False,
    )
    is_active: Mapped[bool] = mapped_column(
        Boolean,
        default=True,
        nullable=False,
    )
    # Record when the account is created and most recently updated.
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        default=lambda: datetime.now(timezone.utc),
        nullable=False,
    )
    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        default=lambda: datetime.now(timezone.utc),
        onupdate=lambda: datetime.now(timezone.utc),
        nullable=False,
    )
```

c. Wire Alembic to the Orbit model metadata.

Open `migrations/env.py`.

Add these imports near the top:

```
from app.models import Base
from app.models.user import User
```

Then replace:

```
target_metadata = None
```

with:

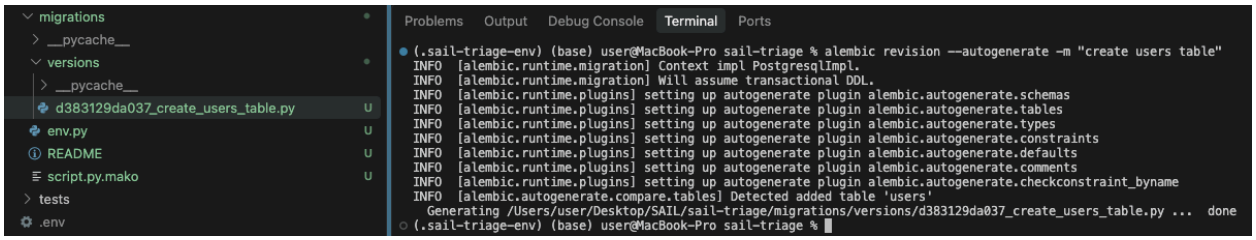
```
target_metadata = Base.metadata
```

The `User` import makes sure Alembic loads the model before inspecting `Base.metadata`.

d. Generate the first migration:

```
alembic revision --autogenerate -m "create users table"
```

Should see output indicating Alembic detected the new `users` table.



e. Inspect the migration file created under:

```
migrations/versions/
```

Confirm that its `upgrade()` function creates a `users` table with the fields from `User`.

```python
def upgrade() -> None:
    """Upgrade schema."""
    # ### commands auto generated by Alembic - please adjust! ###
    op.create_table('users',
    sa.Column('id', sa.Integer(), nullable=False),
    sa.Column('username', sa.String(length=50), nullable=False),
    sa.Column('email', sa.String(length=255), nullable=False),
    sa.Column('password_hash', sa.String(length=255), nullable=False),
    sa.Column('display_name', sa.String(length=100), nullable=False),
    sa.Column('bio', sa.Text(), nullable=True),
    sa.Column('role', sa.String(length=20), nullable=False),
    sa.Column('is_active', sa.Boolean(), nullable=False),
    sa.Column('created_at', sa.DateTime(timezone=True), nullable=False),
    sa.Column('updated_at', sa.DateTime(timezone=True), nullable=False),
    sa.PrimaryKeyConstraint('id'),
    sa.UniqueConstraint('email'),
    sa.UniqueConstraint('username')
    )
    # ### end Alembic commands ###
```

f. Apply the migration:

```
alembic upgrade head
```

```
(.sail-triage-env) (base) user@MacBook-Pro sail-triage % alembic upgrade head
INFO  [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO  [alembic.runtime.migration] Will assume transactional DDL.
INFO  [alembic.runtime.migration] Running upgrade  -> d383129da037, create users table
```

g. Verify the migration state:

```
alembic current
```

```
(.sail-triage-env) (base) user@MacBook-Pro sail-triage % alembic current

INFO  [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO  [alembic.runtime.migration] Will assume transactional DDL.
d383129da037 (head)
```

It should show the revision just created and indicate `(head)`.

h. Verify the table directly in PostgreSQL:

```
psql -U orbit_app -d orbit -h localhost
```

Then run:

```
\dt
```

Should see at least:

```
alembic_version
users
```

Inspect the table:

```
\d users
```

Confirm the columns include:

```
id
username
email
password_hash
display_name
bio
role
is_active
created_at
updated_at
```

```
orbit=> \dt
          List of relations
 Schema |      Name       | Type  |   Owner
--------+-----------------+-------+-----------
 public | alembic_version | table | orbit_app
 public | users           | table | orbit_app
(2 rows)
```

```
orbit=> \d users
                                      Table "public.users"
    Column     |           Type           | Collation | Nullable |              Default
---------------+--------------------------+-----------+----------+-----------------------------------
 id            | integer                  |           | not null | nextval('users_id_seq'::regclass)
 username      | character varying(50)    |           | not null |
 email         | character varying(255)   |           | not null |
 password_hash | character varying(255)   |           | not null |
 display_name  | character varying(100)   |           | not null |
 bio           | text                     |           |          |
 role          | character varying(20)    |           | not null |
 is_active     | boolean                  |           | not null |
 created_at    | timestamp with time zone |           | not null |
 updated_at    | timestamp with time zone |           | not null |
Indexes:
    "users_pkey" PRIMARY KEY, btree (id)
    "users_email_key" UNIQUE CONSTRAINT, btree (email)
    "users_username_key" UNIQUE CONSTRAINT, btree (username)
```

Exit:

```
\q
```

Now have:

- the `User` SQLAlchemy model
- Alembic wired to the model metadata
- the first migration generated and applied
- the `users` table created in PostgreSQL
- Alembic reporting the migration at `head`

Step 11: Implement user registration, authentication, and password security

a. Create password hashing utilities

```python
"""Authentication and password security services for Orbit."""

from pwdlib import PasswordHash


PASSWORD_HASH = PasswordHash.recommended()


def hash_password(password: str) -> str:
    """Hash a plaintext password for secure storage."""
    return PASSWORD_HASH.hash(password)

                                        (parameter) hashed_password: str
def verify_password(password: str, hashed_password: str) -> bool:
    """Verify a plaintext password against a stored password hash."""
    return PASSWORD_HASH.verify(password, hashed_password)
```

```
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit % python -c "from app.services.auth import hash_password, verify_password; h=hash_password
'OrbitTest123!'); print(h); print(verify_password('OrbitTest123!', h)); print(verify_password('wrong', h))"
$argon2id$v=19$m=65536,t=3,p=4$V8+TtTHpKvT999SZTLwIxg$O1u6sRfxbifU5Vh38yaH6XaqSDWHQFeXZYG4bmQwzPs
True
False
```

`auth.py` is a service module that provides Orbit's password hashing and verification functions. It was tested using `python -c` because service modules are imported and called rather than run directly.

```
tests/test_auth.py ...

============================================================== 3 passed in 0.20s ==
```

- Implemented hash_password() using pwdlib/Argon2.
- Implemented verify_password().
- Manually verified:
  - hashing works,
  - correct password → True,
  - incorrect password → False.

b. Generate a JWT secret and add `JWT_SECRET_KEY` to `.env`.

At command line:

- `python -c "import secrets; print(secrets.token_urlsafe(64))"`

paste into `.env` file

c. Create authentication/JWT utilities

Files involved:

- `app/config.py` — defines the JWT signing algorithm and token expiration settings.
- `app/services/auth.py` — creates, decodes, and validates JWT access tokens.
- `tests/test_auth.py` — tests token creation, decoding, invalid tokens, and expiration.

config.py

```python
"""Application configuration for Orbit."""

import os
from dotenv import load_dotenv

load_dotenv()

class Config:
    """Store configuration values loaded from environment variables."""

    APP_ENV = os.getenv("APP_ENV", "development")
    DEBUG = os.getenv("DEBUG", "false").lower() == "true"
    DATABASE_URL = os.getenv("DATABASE_URL")
    JWT_SECRET_KEY = os.getenv("JWT_SECRET_KEY")
    JWT_ALGORITHM = os.getenv("JWT_ALGORITHM", "HS256")
    JWT_ACCESS_TOKEN_EXPIRE_MINUTES = int(
        os.getenv("JWT_ACCESS_TOKEN_EXPIRE_MINUTES", "30")
    )
```

auth.py

```python
"""Authentication and password security services for Orbit."""

import jwt
from pwdlib import PasswordHash
from app.config import Config
from datetime import datetime, timedelta, timezone

PASSWORD_HASH = PasswordHash.recommended()

def hash_password(password: str) -> str:
    """Hash a plaintext password for secure storage."""
    return PASSWORD_HASH.hash(password)

def verify_password(password: str, hashed_password: str) -> bool:
    """Verify a plaintext password against a stored password hash."""
    return PASSWORD_HASH.verify(password, hashed_password)

def create_access_token(user_id: int) -> str:
    """Create a JWT access token for an authenticated user."""
    expires_at = datetime.now(timezone.utc) + timedelta(
        minutes=Config.JWT_ACCESS_TOKEN_EXPIRE_MINUTES
    )
    payload = {"sub": str(user_id), "exp": expires_at}

    return jwt.encode(
        payload,
        Config.JWT_SECRET_KEY,
        algorithm=Config.JWT_ALGORITHM,
    )

def decode_access_token(token: str) -> dict:
    """Decode and validate a JWT access token."""
    return jwt.decode(
        token,
        Config.JWT_SECRET_KEY,
        algorithms=[Config.JWT_ALGORITHM],
    )
```

Test run at command line uses a fake key placed in the .env file for JWT_SECRET_KEY. Once there can run:

```
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit % python -c "from app.services.auth import create_access_token, decode_access_token; token=
create_access_token(1); print(token); print(decode_access_token(token))"
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxIiwiZXhwIjoxNzg4OTI0ODc2fQ.-rFf4ZI_RKbytT-3F0l7lg-Zccql5Nk5T3fbxyh7npM
{'sub': '1', 'exp': 1788924876}
```

Added tests to test_auth.py:

```python

def test_create_and_decode_access_token():
    """Verify that a valid JWT contains the correct user ID."""
    token = create_access_token(user_id=1)
    payload = decode_access_token(token)

    assert payload["sub"] == "1"
    assert "exp" in payload


def test_decode_invalid_access_token():
    """Verify that an invalid JWT is rejected."""
    with pytest.raises(jwt.InvalidTokenError):
        decode_access_token("invalid.token.value")


def test_decode_expired_access_token():
    """Verify that an expired JWT is rejected."""
    payload = {
        "sub": "1",
        "exp": datetime.now(timezone.utc) - timedelta(minutes=1),
    }
    token = jwt.encode(
        payload,
        Config.JWT_SECRET_KEY,
        algorithm=Config.JWT_ALGORITHM,
    )

    with pytest.raises(jwt.ExpiredSignatureError):
        decode_access_token(token)
```

```
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit % python -m pytest tests/test_auth.py
============================= test session starts ==============================
platform darwin -- Python 3.12.2, pytest-9.1.1, pluggy-1.6.0
rootdir: /Users/user/Desktop/SAIL/sail-orbit
plugins: anyio-4.15.1
collected 6 items

tests/test_auth.py ......

============================== 6 passed in 0.23s ===============================
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit %
```

d. Add registration request handling and tests

Registration establishes the API path used to create a SAIL Orbit account. Incoming registration data is validated before a `User` object is created, the plaintext password is hashed, and the new account is committed to PostgreSQL.

Files involved:

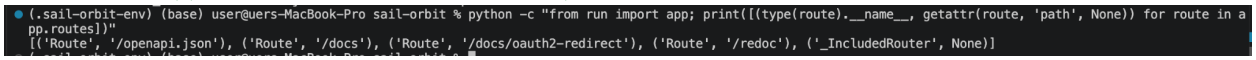
- `app/schemas/user.py` defines and validates registration fields.
- `app/extensions.py` provides a database session for the request and closes it afterward.
- `app/api/auth.py` implements `POST /auth/register` and account creation.

- `app/services/auth.py` provides password hashing.
- `app/models/user.py` defines the database representation of the account.
- `run.py` creates the FastAPI application and registers the authentication router, making `/auth/register` available through the application.

Request flow:

```
run.py registers auth router
        ↓
POST /auth/register
        ↓
UserRegistration validates request
        ↓
Password is hashed
        ↓
User model is created
        ↓
User is committed to PostgreSQL
        ↓
Non-sensitive account data returned
```

Run: `python -c "from run import app; print([(type(route).__name__, getattr(route, 'path', None)) for route in app.routes])"`



Shows app loads successfully.

Run: `python -c "from run import app; print(list(app.openapi()['paths'].keys()))"`



Shows FastAPI sees /auth/register.

Testing

Automated testing was added for the `POST /auth/register` endpoint using `httpx2.AsyncClient` and `ASGITransport`. This allows requests to be sent directly to the FastAPI application without starting a separate web server.

The test verified:

- A valid registration request returns HTTP `201 Created`.
- The response contains the expected username, email, display name, role, and account status.
- Neither the plaintext password nor `password_hash` is exposed in the API response.
- The new user is actually stored in PostgreSQL.
- The stored password is a hash rather than the submitted plaintext password.
- The stored hash successfully verifies against the original password.
- Test users are removed before and after testing so repeated test runs do not leave test data in the database.

The test uses `@pytest.mark.anyio` so pytest can execute the asynchronous HTTP requests. `httpx2` is used directly rather than Starlette's `TestClient`, avoiding the deprecated AnyIO interface encountered with `TestClient`.

Result: `1 passed in 0.39s` with no warnings.

```
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit % python -m pytest tests/test_users.py -v
============================= test session starts ==============================
platform darwin -- Python 3.12.2, pytest-9.1.1, pluggy-1.6.0 -- /Users/user/Desktop/SAIL/sail-orbit/.sail-orbit-env/bin/python
cachedir: .pytest_cache
rootdir: /Users/user/Desktop/SAIL/sail-orbit
plugins: anyio-4.15.1
collected 1 item

tests/test_users.py::test_register_user[asyncio] PASSED                  [100%]

============================== 1 passed in 0.39s ===============================
```

e. Add login request handling and tests

```
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit % python -m pytest tests/test_users.py::test_login_user -v
============================= test session starts ==============================
platform darwin -- Python 3.12.2, pytest-9.1.1, pluggy-1.6.0 -- /Users/user/Desktop/SAIL/sail-orbit/.sail-orbit-env/bin/python
cachedir: .pytest_cache
rootdir: /Users/user/Desktop/SAIL/sail-orbit
plugins: anyio-4.15.1
collected 1 item

tests/test_users.py::test_login_user[asyncio] PASSED                     [100%]

============================== 1 passed in 0.76s ===============================
```

f. Add current-user authentication dependency

```
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit % python -m pytest tests/test_auth.py -v
============================= test session starts ==============================
platform darwin -- Python 3.12.2, pytest-9.1.1, pluggy-1.6.0 -- /Users/user/Desktop/SAIL/sail-orbit/.sail-orbit-env/bin/python
cachedir: .pytest_cache
rootdir: /Users/user/Desktop/SAIL/sail-orbit
plugins: anyio-4.15.1
collected 8 items

tests/test_auth.py::test_hash_password_does_not_return_plaintext PASSED  [ 12%]
tests/test_auth.py::test_verify_password_accepts_correct_password PASSED [ 25%]
tests/test_auth.py::test_verify_password_rejects_incorrect_password PASSED [ 37%]
tests/test_auth.py::test_create_and_decode_access_token PASSED           [ 50%]
tests/test_auth.py::test_decode_invalid_access_token PASSED              [ 62%]
tests/test_auth.py::test_decode_expired_access_token PASSED              [ 75%]
tests/test_auth.py::test_get_current_user_accepts_valid_token PASSED     [ 87%]
tests/test_auth.py::test_get_current_user_rejects_invalid_token PASSED   [100%]

============================== 8 passed in 0.69s ===============================
```

g. Add duplicate username/email checks

```
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit % python -m pytest tests/test_users.py -v
============================= test session starts ==============================
platform darwin -- Python 3.12.2, pytest-9.1.1, pluggy-1.6.0 -- /Users/user/Desktop/SAIL/sail-orbit/.sail-orbit-env/bin/python
cachedir: .pytest_cache
rootdir: /Users/user/Desktop/SAIL/sail-orbit
plugins: anyio-4.15.1
collected 4 items

tests/test_users.py::test_register_user[asyncio] PASSED                  [ 25%]
tests/test_users.py::test_login_user[asyncio] PASSED                     [ 50%]
tests/test_users.py::test_register_rejects_duplicate_username[asyncio] PASSED [ 75%]
tests/test_users.py::test_register_rejects_duplicate_email[asyncio] PASSED [100%]

============================== 4 passed in 0.75s ===============================
```

h. Add password validation rules

```
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit % python -m pytest tests/test_users.py -v
============================= test session starts ==============================
platform darwin -- Python 3.12.2, pytest-9.1.1, pluggy-1.6.0 -- /Users/user/Desktop/SAIL/sail-orbit/.sail-orbit-env/bin/python
cachedir: .pytest_cache
rootdir: /Users/user/Desktop/SAIL/sail-orbit
plugins: anyio-4.15.1
collected 9 items

tests/test_users.py::test_register_user[asyncio] PASSED                  [ 11%]
tests/test_users.py::test_login_user[asyncio] PASSED                     [ 22%]
tests/test_users.py::test_register_rejects_duplicate_username[asyncio] PASSED [ 33%]
tests/test_users.py::test_register_rejects_duplicate_email[asyncio] PASSED [ 44%]
tests/test_users.py::test_register_rejects_short_password[asyncio] PASSED [ 55%]
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-orbittest123!] PASSED [ 66%]
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-ORBITTEST123!] PASSED [ 77%]
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-OrbitTestPass!] PASSED [ 88%]
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-OrbitTest1234] PASSED [100%]

============================== 9 passed in 0.72s ===============================
```

i. Complete registration testing

```
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit % python -m pytest tests/test_users.py -v
============================= test session starts ==============================
platform darwin -- Python 3.12.2, pytest-9.1.1, pluggy-1.6.0 -- /Users/user/Desktop/SAIL/sail-orbit/.sail-orbit-env/bin/python
cachedir: .pytest_cache
rootdir: /Users/user/Desktop/SAIL/sail-orbit
plugins: anyio-4.15.1
collected 10 items

tests/test_users.py::test_register_user[asyncio] PASSED                  [ 10%]
tests/test_users.py::test_login_user[asyncio] PASSED                     [ 20%]
tests/test_users.py::test_register_rejects_duplicate_username[asyncio] PASSED [ 30%]
tests/test_users.py::test_register_rejects_duplicate_email[asyncio] PASSED [ 40%]
tests/test_users.py::test_register_rejects_short_password[asyncio] PASSED [ 50%]
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-orbittest123!] PASSED [ 60%]
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-ORBITTEST123!] PASSED [ 70%]
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-OrbitTestPass!] PASSED [ 80%]
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-OrbitTest1234] PASSED [ 90%]
tests/test_users.py::test_register_rejects_invalid_email[asyncio] PASSED [100%]

============================== 10 passed in 0.76s ==============================
```

j. Complete login testing

```
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit % python -m pytest tests/test_users.py -v
============================= test session starts ==============================
platform darwin -- Python 3.12.2, pytest-9.1.1, pluggy-1.6.0 -- /Users/user/Desktop/SAIL/sail-orbit/.sail-orbit-env/bin/python
cachedir: .pytest_cache
rootdir: /Users/user/Desktop/SAIL/sail-orbit
plugins: anyio-4.15.1
collected 12 items

tests/test_users.py::test_register_user[asyncio] PASSED                  [  8%]
tests/test_users.py::test_login_user[asyncio] PASSED                     [ 16%]
tests/test_users.py::test_register_rejects_duplicate_username[asyncio] PASSED [ 25%]
tests/test_users.py::test_register_rejects_duplicate_email[asyncio] PASSED [ 33%]
tests/test_users.py::test_register_rejects_short_password[asyncio] PASSED [ 41%]
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-orbittest123!] PASSED [ 50%]
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-ORBITTEST123!] PASSED [ 58%]
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-OrbitTestPass!] PASSED [ 66%]
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-OrbitTest1234] PASSED [ 75%]
tests/test_users.py::test_register_rejects_invalid_email[asyncio] PASSED [ 83%]
tests/test_users.py::test_login_rejects_incorrect_password[asyncio] PASSED [ 91%]
tests/test_users.py::test_login_rejects_unknown_username[asyncio] PASSED [100%]

============================== 12 passed in 0.95s ==============================
```

k. Complete protected authentication testing

```
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit % python -m pytest tests/test_auth.py -v
============================= test session starts ==============================
platform darwin -- Python 3.12.2, pytest-9.1.1, pluggy-1.6.0 -- /Users/user/Desktop/SAIL/sail-orbit/.sail-orbit-env/bin/python
cachedir: .pytest_cache
rootdir: /Users/user/Desktop/SAIL/sail-orbit
plugins: anyio-4.15.1
collected 10 items

tests/test_auth.py::test_hash_password_does_not_return_plaintext PASSED  [ 10%]
tests/test_auth.py::test_verify_password_accepts_correct_password PASSED [ 20%]
tests/test_auth.py::test_verify_password_rejects_incorrect_password PASSED [ 30%]
tests/test_auth.py::test_create_and_decode_access_token PASSED           [ 40%]
tests/test_auth.py::test_decode_invalid_access_token PASSED              [ 50%]
tests/test_auth.py::test_decode_expired_access_token PASSED              [ 60%]
tests/test_auth.py::test_get_current_user_accepts_valid_token PASSED     [ 70%]
tests/test_auth.py::test_get_current_user_rejects_invalid_token PASSED   [ 80%]
tests/test_auth.py::test_get_current_user_rejects_expired_token PASSED   [ 90%]
tests/test_auth.py::test_get_current_user_rejects_unknown_user PASSED    [100%]

============================== 10 passed in 0.79s ==============================
```

l. Verify Step 11 end-to-end

```
tests/test_auth.py::test_hash_password_does_not_return_plaintext PASSED  [  4%]
tests/test_auth.py::test_verify_password_accepts_correct_password PASSED [  8%]
tests/test_auth.py::test_verify_password_rejects_incorrect_password PASSED [ 13%]
tests/test_auth.py::test_create_and_decode_access_token PASSED           [ 17%]
tests/test_auth.py::test_decode_invalid_access_token PASSED              [ 21%]
tests/test_auth.py::test_decode_expired_access_token PASSED              [ 26%]
tests/test_auth.py::test_get_current_user_accepts_valid_token PASSED     [ 30%]
tests/test_auth.py::test_get_current_user_rejects_invalid_token PASSED   [ 34%]
tests/test_auth.py::test_get_current_user_rejects_expired_token PASSED   [ 39%]
tests/test_auth.py::test_get_current_user_rejects_unknown_user PASSED    [ 43%]
tests/test_database_connection.py::test_database_connection PASSED       [ 47%]
tests/test_users.py::test_register_user[asyncio] PASSED                  [ 52%]
tests/test_users.py::test_login_user[asyncio] PASSED                     [ 56%]
tests/test_users.py::test_register_rejects_duplicate_username[asyncio] PASSED [ 60%]
tests/test_users.py::test_register_rejects_duplicate_email[asyncio] PASSED [ 65%]
tests/test_users.py::test_register_rejects_short_password[asyncio] PASSED [ 69%]
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-orbittest123!] PASSED [ 73%]
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-ORBITTEST123!] PASSED [ 78%]
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-OrbitTestPass!] PASSED [ 82%]
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-OrbitTest1234] PASSED [ 86%]
tests/test_users.py::test_register_rejects_invalid_email[asyncio] PASSED [ 91%]
tests/test_users.py::test_login_rejects_incorrect_password[asyncio] PASSED [ 95%]
tests/test_users.py::test_login_rejects_unknown_username[asyncio] PASSED [100%]

============================== 23 passed in 1.05s ==============================
```

```
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit % ruff check .
All checks passed!
```

```
All checks passed!
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit % black .
reformatted /Users/user/Desktop/SAIL/sail-orbit/app/config.py
reformatted /Users/user/Desktop/SAIL/sail-orbit/app/extensions.py
reformatted /Users/user/Desktop/SAIL/sail-orbit/app/models/__init__.py
reformatted                                                orbit/app/schemas/auth.py
reformatted                                                orbit/migrations/env.py
reformatted /Users/user/Desktop/SAIL/sail-orbit/app/models/user.py
reformatted /Users/user/Desktop/SAIL/sail-orbit/run.py
reformatted /Users/user/Desktop/SAIL/sail-orbit/app/services/auth.py
reformatted /Users/user/Desktop/SAIL/sail-orbit/app/schemas/user.py
reformatted /Users/user/Desktop/SAIL/sail-orbit/tests/test_database_connection.py
reformatted /Users/user/Desktop/SAIL/sail-orbit/app/api/auth.py
reformatted /Users/user/Desktop/SAIL/sail-orbit/migrations/versions/d383129da037_create_users_table.py
reformatted /Users/user/Desktop/SAIL/sail-orbit/tests/test_auth.py
reformatted /Users/user/Desktop/SAIL/sail-orbit/tests/test_users.py

All done! ✨ 🍰 ✨
14 files reformatted, 19 files left unchanged.
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit % black --check .
All done! ✨ 🍰 ✨
33 files would be left unchanged.
```

Open file in editor (cmd + click)

Step 12: Implement user profiles, roles, and permissions

a. Create and register authenticated user profile retrieval

Create the first user endpoint in app/api/users.py. This endpoint returns the complete account/profile information appropriate for the account owner. The authenticated user's identity comes from the JWT through the existing get_current_user dependency, so the client does not supply a user ID. Then register the users router with the FastAPI application. In run.py, you should already have the authentication router registered.

Then verify that FastAPI successfully loads both routers; use FastAPI's generated OpenAPI schema:

python -c "from run import app; print(list(app.openapi()['paths'].keys()))"

```
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit % python -c "from run import app; print(list(app.openapi()['paths'].keys()))"
['/auth/register', '/auth/login', '/users/me']
```

The three expected endpoints are exposed. The testing of its actual behavior next. Must verify three things:

1. A valid JWT allows access to `GET /users/me`.
2. The endpoint returns the authenticated user's expected profile/account fields.
3. The response does **not** expose `password_hash`.
4. Separately verify that access without a JWT is rejected

Also, now is a good time to refactor the test_users.py file. Include more reusable helpers and fixtures from pytest. **DONE**

1. Added 2 fixtures and 3 helpers currently
2. Tests still pass

Added `test_get_own_profile()` to test first 3, then for #4 added `test_get_own_profile_requires_authentication()`

```
tests/test_users.py::test_register_user[asyncio] PASSED
tests/test_users.py::test_login_user[asyncio] PASSED
tests/test_users.py::test_register_rejects_duplicate_username[asyncio] PASSED
tests/test_users.py::test_register_rejects_duplicate_email[asyncio] PASSED
tests/test_users.py::test_register_rejects_short_password[asyncio] PASSED
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-orbittest123!] PASSED
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-ORBITTEST123!] PASSED
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-OrbitTestPass!] PASSED
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-OrbitTest1234] PASSED
tests/test_users.py::test_register_rejects_invalid_email[asyncio] PASSED
tests/test_users.py::test_login_rejects_incorrect_password[asyncio] PASSED
tests/test_users.py::test_login_rejects_unknown_username[asyncio] PASSED
tests/test_users.py::test_get_own_profile[asyncio] PASSED
tests/test_users.py::test_get_own_profile_requires_authentication[asyncio] PASSED

============================================================== 14 passed in 0.93s ==============================================================
```

b. Add user profile update functionality

```
class UserProfileUpdate(BaseModel):
    """Define profile information an authenticated user may update."""

    # Both fields are optional so a user can update one profile field without
    # being required to resubmit the other.
    display_name: str | None = None
    bio: str | None = None
```

`UserProfileUpdate` deliberately creates an **allowlist** of user-editable profile fields. Even if a client submits values such as `role="admin"` or `is_active=False`, those fields are not part of this schema and therefore will not become part of the profile-update operation.

Now add the update endpoint to `app/api/users.py`.

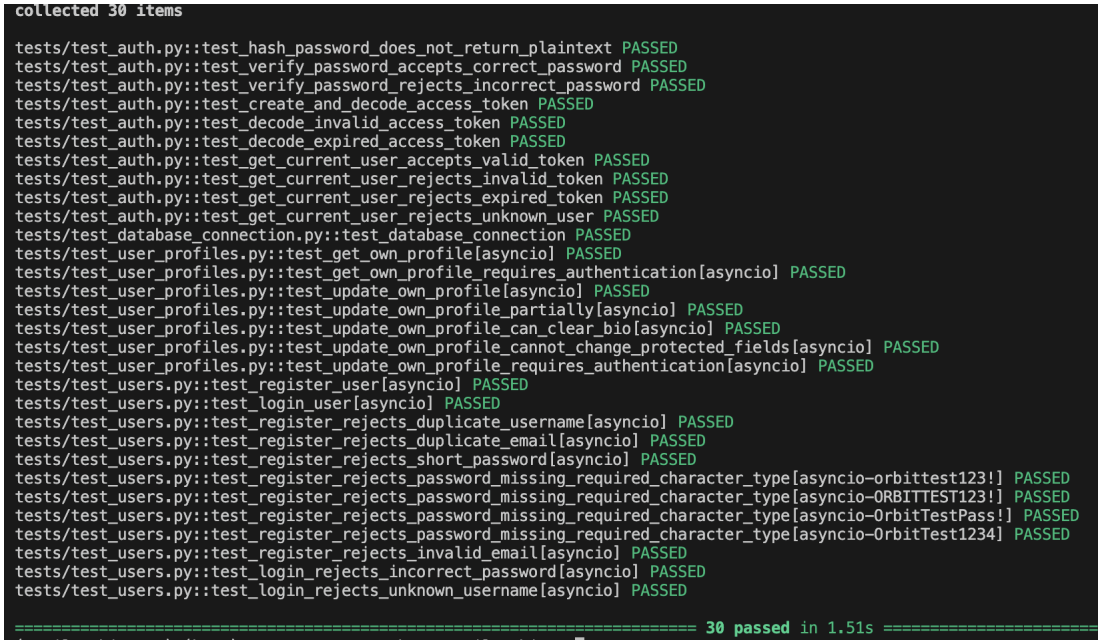
Why **`PATCH`:** this operation supports partial profile updates. A request containing only `bio`, for example, changes only `bio`; `display_name` remains untouched.

Why **`model_dump(exclude_unset=True)`:** it distinguishes a field the user deliberately supplied from one they omitted. That is important here because `bio=None` can intentionally clear a bio, while an omitted `bio` means leave it unchanged.

```
collected 15 items

tests/test_users.py::test_register_user[asyncio] PASSED
tests/test_users.py::test_login_user[asyncio] PASSED
tests/test_users.py::test_register_rejects_duplicate_username[asyncio] PASSED
tests/test_users.py::test_register_rejects_duplicate_email[asyncio] PASSED
tests/test_users.py::test_register_rejects_short_password[asyncio] PASSED
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-orbittest123!] PASSED
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-ORBITTEST123!] PASSED
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-OrbitTestPass!] PASSED
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-OrbitTest1234] PASSED
tests/test_users.py::test_register_rejects_invalid_email[asyncio] PASSED
tests/test_users.py::test_login_rejects_incorrect_password[asyncio] PASSED
tests/test_users.py::test_login_rejects_unknown_username[asyncio] PASSED
tests/test_user_profiles.py::test_get_own_profile[asyncio] PASSED
tests/test_user_profiles.py::test_get_own_profile_requires_authentication[asyncio] PASSED
tests/test_user_profiles.py::test_update_own_profile[asyncio] PASSED

============================================================== 15 passed in 1.02s ==============================================================
```

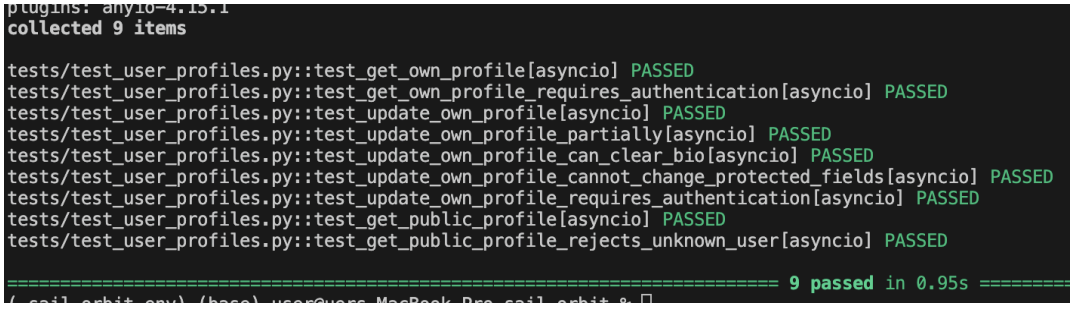


Implemented authenticated profile updates for `display_name` and `bio`, including partial updates and bio clearing. Profile tests were separated into `test_user_profiles.py` and verify retrieval, updates, authentication, protected fields, and sensitive-data protection; **all 7 tests pass.**

c. Add public user profile retrieval

This will allow Orbit users to view another user's public profile without exposing private account information such as email, role, account status, or password data.

Implemented public profile retrieval through `GET /users/{username}`. Public profiles expose only appropriate fields (`id`, `username`, `display_name`, `bio`, and `created_at`), exclude private account and authentication data, and return `404` for unknown users. **All 9 profile tests pass.**

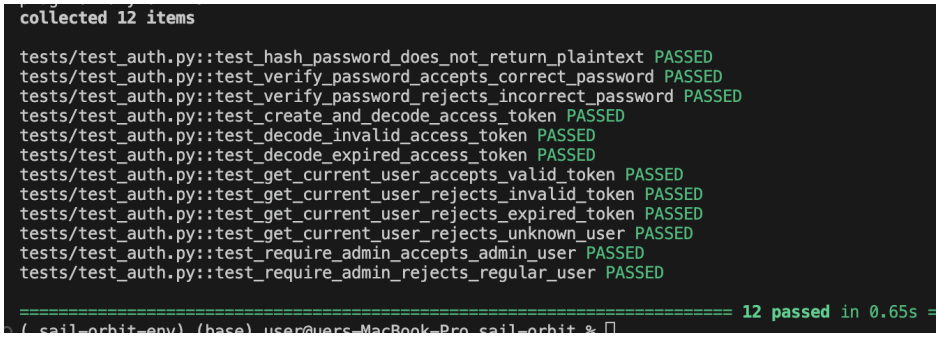


d. Implement role-based authorization

Role-based authorization will be implemented by adding a reusable authorization dependency to the existing authentication service, which will allow administrator-only endpoints to verify a user's role consistently.

This creates the reusable role check for administrator-only operations. A missing or invalid login remains a **401**, while an authenticated user without the required role receives **403 Forbidden**.

Next, verify the authorization dependency itself before using it for admin endpoints. 12d authorization testing is complete: admin users are permitted, regular users receive 403 Forbidden, and all 12 authentication tests pass.



e. Add administrator-only user management, including account activation/deactivation
Start with the request schema for changing account status. Add this below `UserProfileUpdate` in `app/schemas/user.py`:

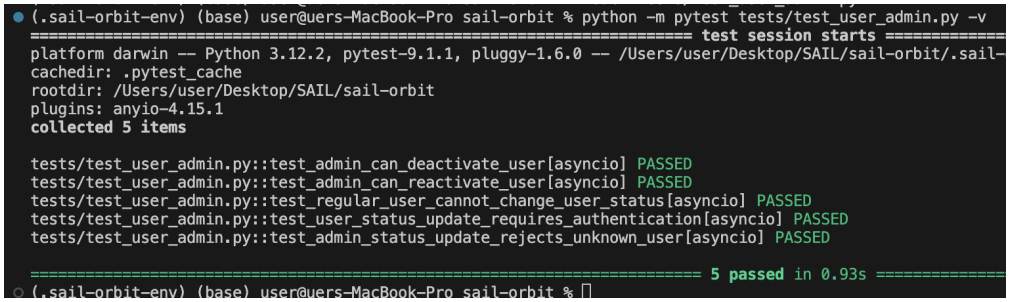
```
class UserStatusUpdate(BaseModel):
    """Define an administrator-requested change to a user's account status."""

    # Account activation is intentionally the only field accepted here.
    # Role changes and profile changes are separate operations with different
    # authorization and validation requirements.
    is_active: bool
```

Next, add the administrator endpoint that changes a user's active status. `admin_user` is intentionally present even though the function body does not use its value: resolving that dependency is what enforces administrator access before the endpoint runs.

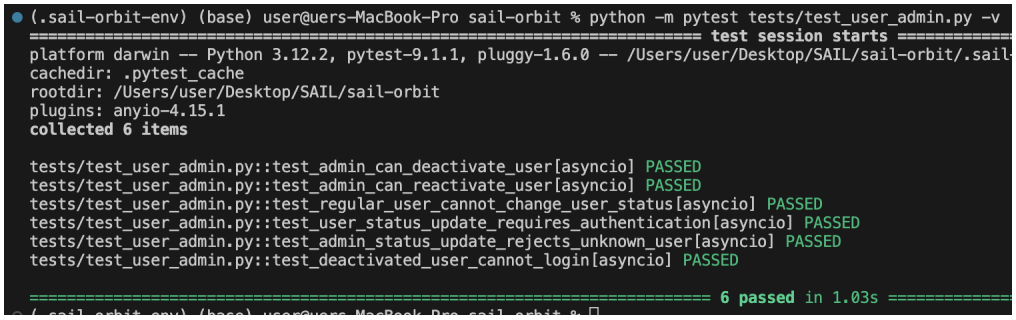
Next, create the administrator status endpoint. Since this is specifically user-management behavior, keep these tests in a new file: `touch tests/test_user_admin.py`

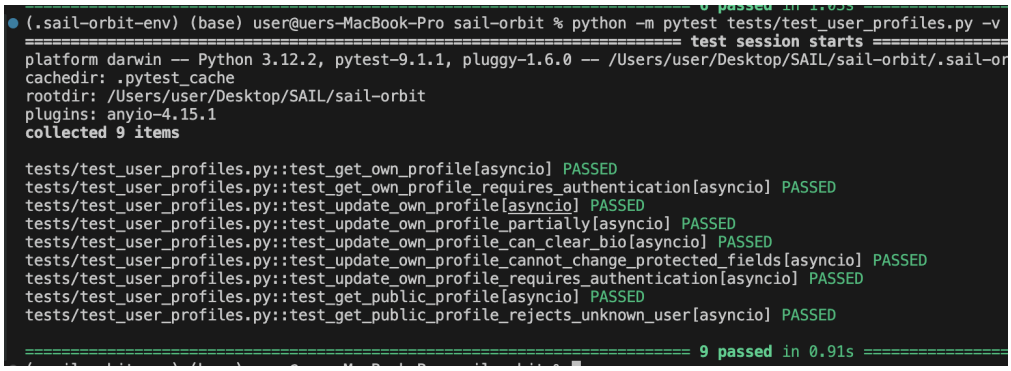
Created `tests/test_user_admin.py` to test administrator account-management functionality through the real API. The file creates isolated admin and standard-user accounts, authenticates the administrator through the normal login/JWT flow, and tests deactivation, reactivation, rejection of non-admin and unauthenticated requests, unknown-user handling, and database persistence of status changes. The tests verify that account activation and deactivation are limited to authenticated administrators and that status changes are correctly persisted.



f. Complete profile testing

Added inactive-account enforcement to `/auth/login` and a sixth admin test verifying that a deactivated user is rejected even with valid credentials. This makes account deactivation effective at authentication time instead of allowing a token to be issued and rejecting it only at protected endpoints.





g. Complete role and permission testing

Tested the `require_admin` authorization dependency and administrator account-management endpoint across permitted and rejected access paths. Verification covered admin access, `403` rejection of authenticated regular users, `401` rejection of unauthenticated requests and invalid/expired tokens, rejection of tokens for nonexistent users, administrator deactivation/reactivation of accounts, blocked login for deactivated accounts, and immediate rejection of previously issued JWTs after an account is deactivated; **all 19 tests pass.**

```
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit % python -m pytest tests/test_auth.py tests/test_user_admin.py -v
collected 19 items

tests/test_auth.py::test_hash_password_does_not_return_plaintext PASSED
tests/test_auth.py::test_verify_password_accepts_correct_password PASSED
tests/test_auth.py::test_verify_password_rejects_incorrect_password PASSED
tests/test_auth.py::test_create_and_decode_access_token PASSED
tests/test_auth.py::test_decode_invalid_access_token PASSED
tests/test_auth.py::test_decode_expired_access_token PASSED
tests/test_auth.py::test_get_current_user_accepts_valid_token PASSED
tests/test_auth.py::test_get_current_user_rejects_invalid_token PASSED
tests/test_auth.py::test_get_current_user_rejects_expired_token PASSED
tests/test_auth.py::test_get_current_user_rejects_unknown_user PASSED
tests/test_auth.py::test_require_admin_accepts_admin_user PASSED
tests/test_auth.py::test_require_admin_rejects_regular_user PASSED
tests/test_user_admin.py::test_admin_can_deactivate_user[asyncio] PASSED
tests/test_user_admin.py::test_admin_can_reactivate_user[asyncio] PASSED
tests/test_user_admin.py::test_regular_user_cannot_change_user_status[asyncio] PASSED
tests/test_user_admin.py::test_user_status_update_requires_authentication[asyncio] PASSED
tests/test_user_admin.py::test_admin_status_update_rejects_unknown_user[asyncio] PASSED
tests/test_user_admin.py::test_deactivated_user_cannot_login[asyncio] PASSED
tests/test_user_admin.py::test_deactivated_user_cannot_use_existing_token[asyncio] PASSED

============================================================== 19 passed in 1.46s ==============================
```

h. Verify Step 12 end-to-end

```
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit % python -m pytest -v
collected 41 items

tests/test_auth.py::test_hash_password_does_not_return_plaintext PASSED
tests/test_auth.py::test_verify_password_accepts_correct_password PASSED
tests/test_auth.py::test_verify_password_rejects_incorrect_password PASSED
tests/test_auth.py::test_create_and_decode_access_token PASSED
tests/test_auth.py::test_decode_invalid_access_token PASSED
tests/test_auth.py::test_decode_expired_access_token PASSED
tests/test_auth.py::test_get_current_user_accepts_valid_token PASSED
tests/test_auth.py::test_get_current_user_rejects_invalid_token PASSED
tests/test_auth.py::test_get_current_user_rejects_expired_token PASSED
tests/test_auth.py::test_get_current_user_rejects_unknown_user PASSED
tests/test_auth.py::test_require_admin_accepts_admin_user PASSED
tests/test_auth.py::test_require_admin_rejects_regular_user PASSED
tests/test_database_connection.py::test_database_connection PASSED
tests/test_user_admin.py::test_admin_can_deactivate_user[asyncio] PASSED
tests/test_user_admin.py::test_admin_can_reactivate_user[asyncio] PASSED
tests/test_user_admin.py::test_regular_user_cannot_change_user_status[asyncio] PASSED
tests/test_user_admin.py::test_user_status_update_requires_authentication[asyncio] PASSED
tests/test_user_admin.py::test_admin_status_update_rejects_unknown_user[asyncio] PASSED
tests/test_user_admin.py::test_deactivated_user_cannot_login[asyncio] PASSED
tests/test_user_admin.py::test_deactivated_user_cannot_use_existing_token[asyncio] PASSED
tests/test_user_profiles.py::test_get_own_profile[asyncio] PASSED
tests/test_user_profiles.py::test_get_own_profile_requires_authentication[asyncio] PASSED
tests/test_user_profiles.py::test_update_own_profile[asyncio] PASSED
tests/test_user_profiles.py::test_update_own_profile_partially[asyncio] PASSED
tests/test_user_profiles.py::test_update_own_profile_can_clear_bio[asyncio] PASSED
tests/test_user_profiles.py::test_update_own_profile_cannot_change_protected_fields[asyncio] PASSED
tests/test_user_profiles.py::test_update_own_profile_requires_authentication[asyncio] PASSED
tests/test_user_profiles.py::test_get_public_profile[asyncio] PASSED
tests/test_user_profiles.py::test_get_public_profile_rejects_unknown_user[asyncio] PASSED
tests/test_users.py::test_register_user[asyncio] PASSED
tests/test_users.py::test_login_user[asyncio] PASSED
tests/test_users.py::test_register_rejects_duplicate_username[asyncio] PASSED
tests/test_users.py::test_register_rejects_duplicate_email[asyncio] PASSED
tests/test_users.py::test_register_rejects_short_password[asyncio] PASSED
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-orbittest123!] PASSED
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-ORBITTEST123!] PASSED
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-OrbitTestPass!] PASSED
tests/test_users.py::test_register_rejects_password_missing_required_character_type[asyncio-OrbitTest1234] PASSED
tests/test_users.py::test_register_rejects_invalid_email[asyncio] PASSED
tests/test_users.py::test_login_rejects_incorrect_password[asyncio] PASSED
tests/test_users.py::test_login_rejects_unknown_username[asyncio] PASSED

============================================================================== 41 passed in 2.26s ==============================
```

```
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit % python -c "from run import app; print(list(app.openapi()['paths'].keys()))"
['/auth/register', '/auth/login', '/users/me', '/users/{username}', '/users/{username}/status']
```

Completed end-to-end verification of user profiles, roles, permissions, authentication, and administrator account management. The full test suite passes with **41 tests**, Ruff and Black checks pass, and all expected authentication and user API routes are registered.

Step 13: Run and manually verify the current backend locally.

a. Start the Orbit Application

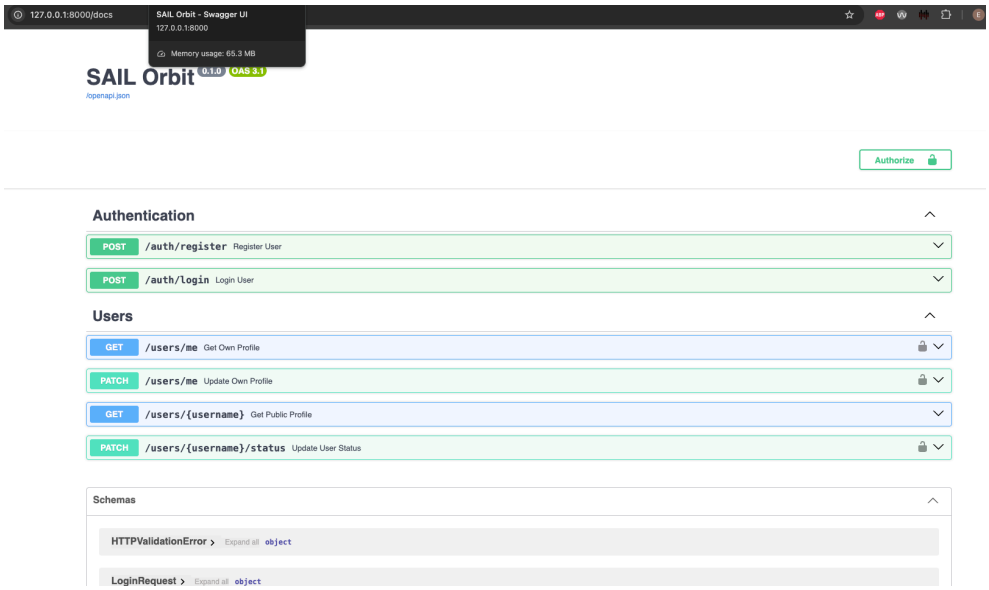
1. Start the FastAPI application: `uvicorn run:app --reload`

2. Verify the terminal reports that Uvicorn is running at `http://127.0.0.1:8000`.

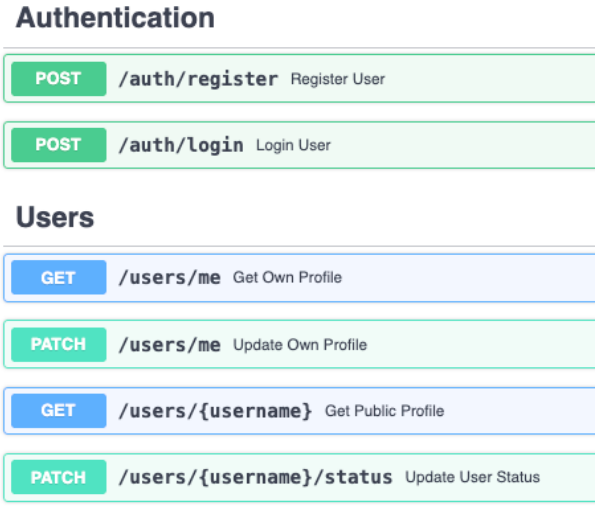
```
(.sail-orbit-env) (base) user@uers-MacBook-Pro sail-orbit % uvicorn run:app --reload
INFO:     Will watch for changes in these directories: ['/Users/user/Desktop/SAIL/sail-orbit']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

b. Open the FastAPI Browser Interface

1. Open a web browser.
2. Navigate to: `127.0.0.1:8000/docs`



3. Verify the interactive API documentation loads and displays these endpoints:
   - `POST /auth/register`
   - `POST /auth/login`
   - `GET /users/me`
   - `PATCH /users/me`
   - `GET /users/{username}`
   - `PATCH /users/{username}/status`



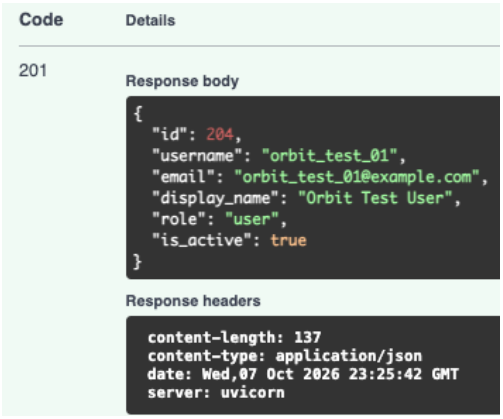
c. Verify Registration and Authentication

Use the interactive API documentation to register a test user, log in, and obtain an authentication token.
Verify that registration succeeds, login returns a token, and invalid credentials are rejected.

1. In Swagger at 127.0.0.1:8000/docs, click into auth/registration and click try it out.
2. Replace example values with:

```
{
  "username": "orbit_test_01",
  "email": "orbit_test_01@example.com",
  "password": "OrbitTest123!",
  "display_name": "Orbit Test User"
}
```

3. Click Execute



4. Repeat steps for login. Replace example code there with

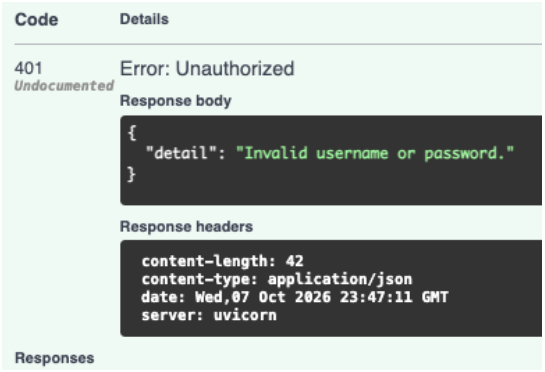
```
{
  "username": "orbit_test_01",
  "password": "OrbitTest123!"
}
```

5. Click Execute



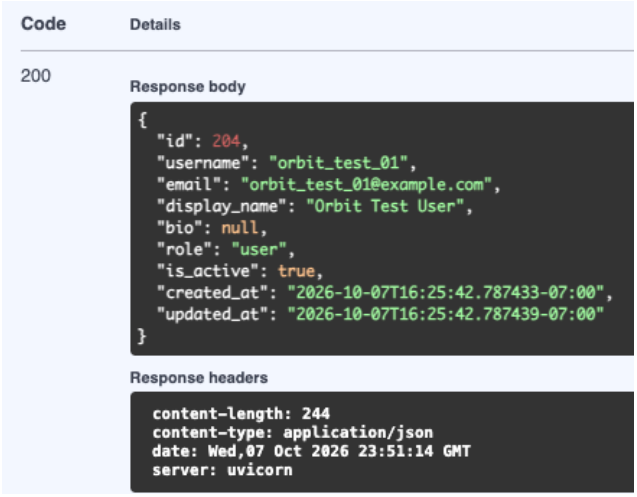
6. Repeat login Try it out steps again, using the below example:

```
{
  "username": "orbit_test_01",
  "password": "WrongPassword123!"
}
```



d. Verify User Profiles
Use the authenticated API endpoints to retrieve and update the test user's profile. Verify that changes persist in PostgreSQL and that public profile retrieval works.

1. In Swagger UI, click Authorize at the top right.
2. Enter the access token returned by your successful login.
3. Click Authorize, then Close.
4. Expand `GET /users/me`, click Try it out, then Execute.
5. Verify the response is `200 OK` and contains the profile for `orbit_test_01`.



6. Verify Profile Updates
   a. Expand `PATCH /users/me` in Swagger UI.
   b. Click Try it out.
   c. Enter:

```
{
  "display_name": "Orbit Updated User",
  "bio": "Testing Orbit profile updates."
}
```

   d. Click Execute and verify the response is `200 OK`.

Code | Details

200

Response body


```
{
  "id": 204,
  "username": "orbit_test_01",
  "email": "orbit_test_01@example.com",
  "display_name": "Orbit Updated User",
  "bio": "Testing Orbit profile updates.",
  "role": "user",
  "is_active": true,
  "created_at": "2026-10-07T16:25:42.787433-07:00",
  "updated_at": "2026-10-07T18:22:22.481091-07:00"
}
```


Response headers

```
content-length: 275
content-type: application/json
date: Thu,08 Oct 2026 01:22:21 GMT
server: uvicorn
```

e. Run `GET /users/me` again to verify both changes were saved.

Code | Details

200

Response body


```
{
  "id": 204,
  "username": "orbit_test_01",
  "email": "orbit_test_01@example.com",
  "display_name": "Orbit Updated User",
  "bio": "Testing Orbit profile updates.",
  "role": "user",
  "is_active": true,
  "created_at": "2026-10-07T16:25:42.787433-07:00",
  "updated_at": "2026-10-07T18:22:22.481091-07:00"
}
```


Response headers

```
content-length: 275
content-type: application/json
date: Thu,08 Oct 2026 01:23:00 GMT
server: uvicorn
```

f. Run `GET /users/username` with `orbit_test_01:`

Code | Details

200

Response body

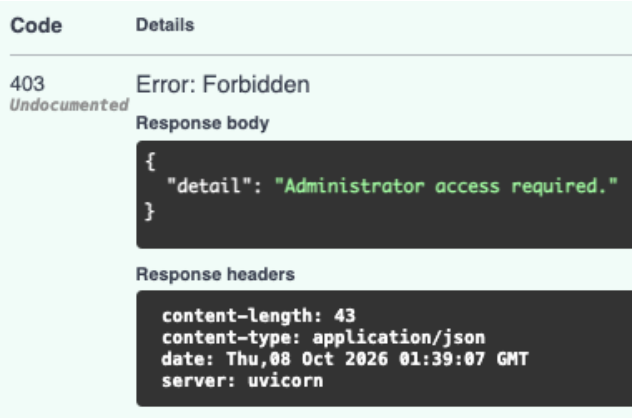

```
{
  "id": 204,
  "username": "orbit_test_01",
  "display_name": "Orbit Updated User",
  "bio": "Testing Orbit profile updates.",
  "created_at": "2026-10-07T16:25:42.787433-07:00"
}
```


Response headers

```
content-length: 160
content-type: application/json
date: Thu,08 Oct 2026 01:36:26 GMT
server: uvicorn
```

e. Verify Permissions and Account Status
Verify that regular users cannot access administrator-only operations.
Using an administrator account, test account deactivation and reactivation. Confirm that deactivated users cannot authenticate or access protected endpoints.

1. Regular user:
    a. Run `PATCH /users/{username}/status` with username `orbit_test_01` and body `{"is_active": false}` The false value tests an attempt to deactivate an account. The true value tests an attempt to reactivate one. For either, expect `403 Forbidden`.

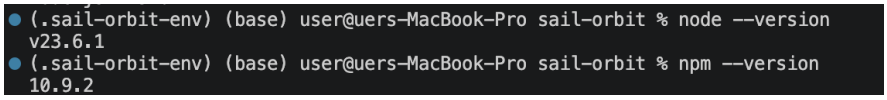


Step 14: Create the Orbit frontend structure and connect it to the FastAPI backend.

a. Create the Frontend Project

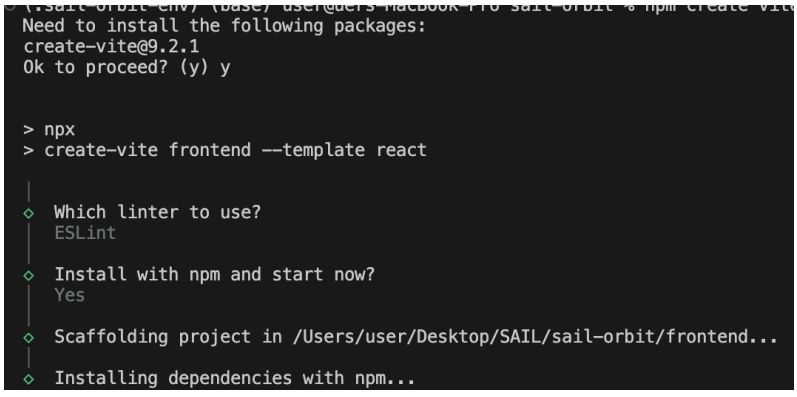
1. Open a second terminal in VS Code, separate from the terminal running Uvicorn. Navigate to `SAIL/sail-orbit`.
2. Verify Node.js and npm:

```
node --version
npm --version
```



3. Create the `frontend` directory and React/Vite project:

```
npm create vite@latest frontend -- --template react
```



b. Configure Frontend Styling and Components

1. Install Tailwind CSS and its Vite integration.
   From `SAIL/sail-orbit/frontend`, run:

```
npm install tailwindcss @tailwindcss/vite
```

2. Configure Tailwind in `frontend/vite.config.js`.

Replace the file contents with:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'
import path from 'node:path'

export default defineConfig({
  plugins: [react(), tailwindcss()],
  resolve: {
    alias: {
      '@': path.resolve(import.meta.dirname, './src'),
    },
  },
})
```

3. Configure the stylesheet.

Replace the contents of `frontend/src/index.css` with:

```
@import "tailwindcss";
```

4. Configure the import alias.

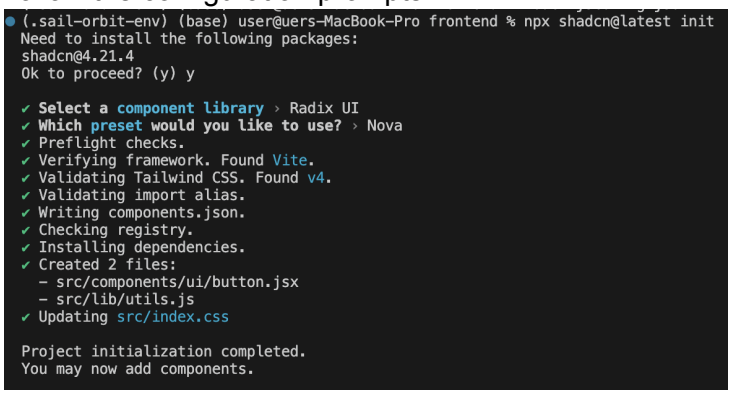
Create `frontend/jsconfig.json`:

```
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "@/*": ["./src/*"]
    }
  }
}
```

5. Initialize shadcn/ui.

From `frontend`, run: `npx shadcn@latest init`

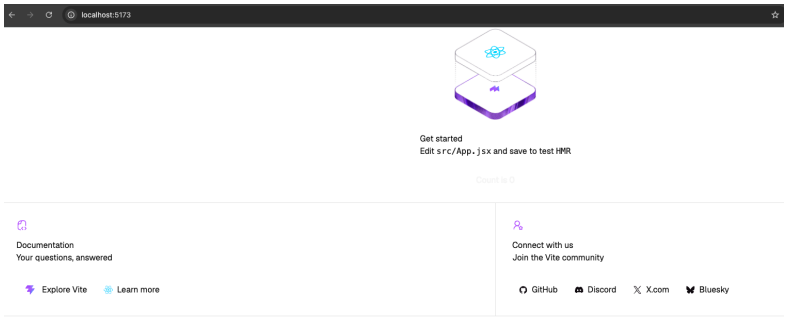
Follow the configuration prompts.



6. Verify the frontend starts.

```
npm run dev
```

Open the local URL displayed by Vite, typically `http://localhost:5173`, and confirm the application loads. The default Vite starter page is expected at this stage.



c. Connect React to the FastAPI Backend

Establishes communication between the React frontend and the existing FastAPI backend using the public user profile endpoint.

1. Configure the frontend API URL

Create this file: `SAIL/sail-orbit/frontend/.env`

Add: `VITE_API_BASE_URL=http://127.0.0.1:8000`

2. Enable CORS in FastAPI

Open `SAIL/sail-orbit/run.py`.

Add this import:

```
from fastapi.middleware.cors import CORSMiddleware
```

Immediately after:

```
app = FastAPI(title="SAIL Orbit")
```

Add:

```
app.add_middleware(
     CORSMiddleware,
     allow_origins=[
          "http://localhost:5173",
          "http://127.0.0.1:5173",
          ],
     allow_credentials=True,
     allow_methods=["*"],
     allow_headers=["*"],
)
```

Keep the existing router registrations unchanged. CORS allows the browser-based frontend on port 5173 to communicate with FastAPI on port 8000.

3. Create the frontend API client

Create:

`SAIL/sail-orbit/frontend/src/api.js`

Add:

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL

export async function getPublicProfile(username) {
  const response = await fetch(
    `API_BASE_URL/users/{encodeURIComponent(username)}`
  )

  if (!response.ok) {
```

```
    throw new Error(`Profile request failed: ${response.status}`)
  }

  return response.json()
}
```

4. Connect the React application

Open:

`SAIL/sail-orbit/frontend/src/App.jsx`

Replace its contents with:

```
import { useEffect, useState } from 'react'
import { getPublicProfile } from './api'

function App() {
  const [profile, setProfile] = useState(null)
  const [error, setError] = useState(null)

  useEffect(() => {
    getPublicProfile('orbit_test_01')
      .then(setProfile)
      .catch((err) => setError(err.message))
  }, [])

  return (
    <main className="mx-auto max-w-2xl p-8">
      <h1 className="mb-6 text-3xl font-bold">SAIL Orbit</h1>

      {error && <p className="text-red-600">{error}</p>}

      {!profile && !error && <p>Loading profile...</p>}

      {profile && (
        <section className="rounded-lg border p-6">
          <h2 className="text-xl font-semibold">
            {profile.display_name}
          </h2>
          <p className="text-sm text-gray-500">
            @{profile.username}
          </p>
          <p className="mt-4">{profile.bio}</p>
        </section>
      )}
    </main>
  )
}

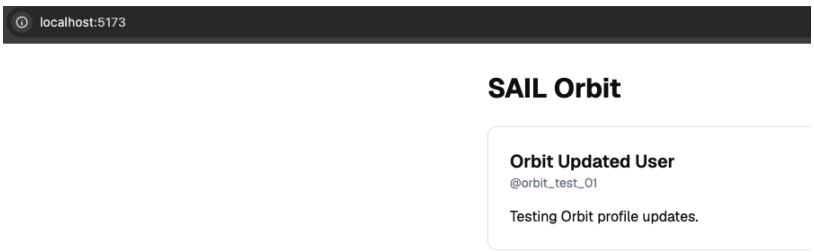
export default App
```

This is a temporary connection test. The actual profile interface belongs to Step 16.

5. Verify the connection

Confirm FastAPI is running at `http://127.0.0.1:8000`.
Restart Vite to load the new `.env` file: `npm run dev`
Open `http://localhost:5173`.
Expected result:
SAIL Orbit
Orbit Updated User
@orbit_test_01
Testing Orbit profile updates.

The profile data should come from FastAPI and PostgreSQL, not hardcoded frontend values.



The screenshot confirms React successfully retrieved and displayed the existing user profile through FastAPI.

Step 15: Build registration and login screens using the existing authentication API.
a. Confirm the authentication API contract
Before writing the React forms, we need to verify the exact request and response formats already implemented.

1. Keep FastAPI and Vite running.
2. Open `http://127.0.0.1:8000/docs`.
3. Expand these endpoints:
   - `POST /auth/register`
   - `POST /auth/login`
4. Confirm the Request body and Responses schemas for both endpoints correct

Endpoint	Request fields	Successful response
`POST /auth/register`	username, email, password, display_name	HTTP 201, string
`POST /auth/login`	username, password	HTTP 200, access_token, token_type

This makes sure the frontend sends the correct fields, handles authentication tokens correctly, and matches the existing backend.

b. Add authentication functions to `api.js`
Below `getPublicProfile()` function:
Add `registerUser()` and `loginUser()` to `frontend/src/api.js`.

Both functions use `fetch()` to send JSON requests to the existing FastAPI endpoints. They check whether the request succeeded, report errors, and return the server's response.

These functions provide a reusable connection between the React registration/login screens and the backend. Registration creates an account; login retrieves a JWT for authenticated requests.

c. Build the Registration and Login Screens

Create two React components using the authentication functions added previously.

1. Create `frontend/src/components/RegisterForm.jsx`

   Creates the registration form for username, email, display name, and password. React's useState manages form values, errors, and submission status. On submission, the component calls `registerUser()` in `api.js`. Successful registration returns the user to the login screen; errors are displayed in the form.

2. Create `frontend/src/components/LoginForm.jsx`

   Creates the login form for username and password. The component uses `useState` to manage inputs and calls `loginUser()` to authenticate through FastAPI. Successful authentication returns a JWT access token to `App.jsx`; failed requests display an error.

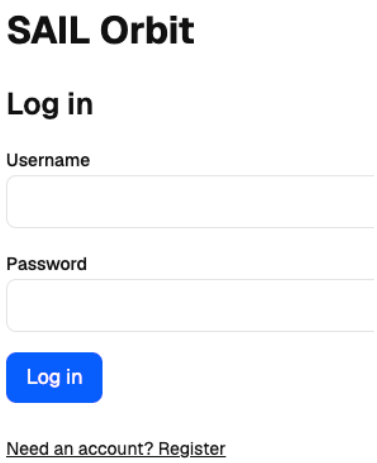
3. Update `frontend/src/App.jsx`

   Coordinates the registration and login components using React state. The `screen` variable controls which form appears, while `accessToken` stores the JWT after login. Logging out clears the token. The token is lost when the page refreshes.

4. Verify the screens

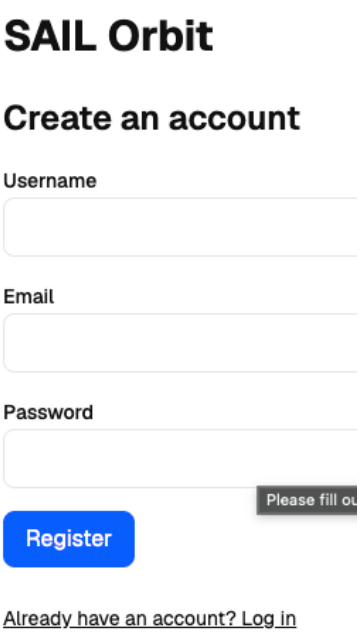
   With FastAPI and Vite running, open http://localhost:5173.

   a. Confirm the login form appears.

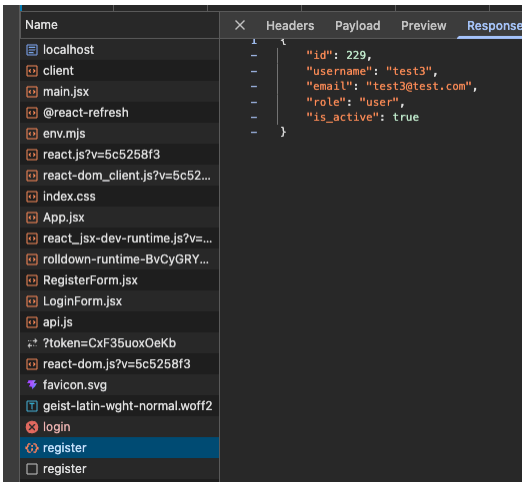


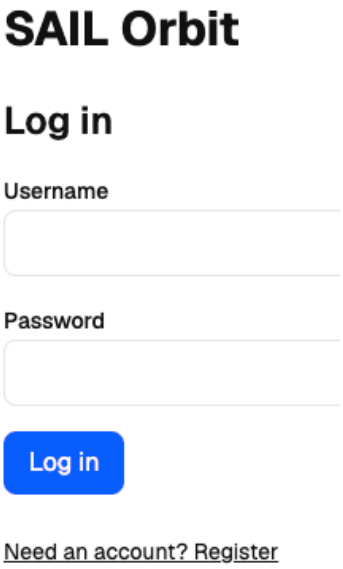
   b. Select Need an account? Register.

   At this step, a display name was recognized as being the same as a username and the display name field was removed. See the appendix, Section 8: SAIL Orbit: Removal of display_name, for details.

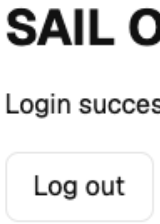


c. Register a new test user with a unique username and email and password. Confirm the application returns to the login screen.





d. Log in using the new credentials. Confirm Login successful appears.



e. Select Log out and confirm the login screen returns. **CONFIRMED**

Step 16: GitHub Actions for Black, Ruff, and frontend ESLint
a. Verify the Git repository configuration
From the SAIL repository root:
git branch --show-current
git status --short
git remote -v
Confirmed:

- Current branch: `develop`
- Working tree: clean before changes
- GitHub remote: `origin`
- GitLab remote: `gitlab`

b. Verify the existing Python and frontend configurations
cat sail-orbit/frontend/package.json
cat sail-orbit/ruff.toml
cat sail-orbit/requirements.txt
Confirmed:

- Black: `26.5.1`
- Ruff: `0.16.3`
- ESLint command: `npm run lint`

- Python application directory: `sail-orbit/`
- Frontend directory: `sail-orbit/frontend/`

c. Create the GitHub Actions workflow
From the SAIL repository root:
mkdir -p .github/workflows
Create `.github/workflows/ci.yml`:

```
name: SAIL Orbit CI

on:
  push:
    branches: [develop]
  pull_request:
    branches: [main]

jobs:
  python-checks:
    runs-on: ubuntu-latest
    continue-on-error: true
    defaults:
      run:
        working-directory: sail-orbit

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
          cache-dependency-path: sail-orbit/requirements.txt

      - name: Install Python dependencies
        run: python -m pip install -r requirements.txt

      - name: Check Black formatting
        run: black --check .

      - name: Run Ruff
        run: ruff check .

  frontend-checks:
    runs-on: ubuntu-latest
    continue-on-error: true
    defaults:
      run:
        working-directory: sail-orbit/frontend

    steps:
      - uses: actions/checkout@v4
```

```
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm
          cache-dependency-path: sail-orbit/frontend/package-lock.json

      - name: Install frontend dependencies
        run: npm ci

      - name: Run ESLint
        run: npm run lint
```

This runs on pushes to `develop` and pull requests targeting `main`.

d. Create the GitLab CI/CD configuration

From the SAIL repository root, create `.gitlab-ci.yml`:

```
stages:
  - lint

black:
  stage: lint
  image: python:3.12
  allow_failure: true
  script:
    - cd sail-orbit
    - pip install black==26.5.1
    - black --check .

ruff:
  stage: lint
  image: python:3.12
  allow_failure: true
  script:
    - cd sail-orbit
    - pip install ruff==0.16.3
    - ruff check .

eslint:
  stage: lint
  image: node:22
  allow_failure: true
  script:
    - cd sail-orbit/frontend
    - npm ci
    - npm run lint

workflow:
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"' - if: 'CI_COMMIT_BRANCH ==
"develop" && $CI_PIPELINE_SOURCE == "push"'
```

Runs on pushes to `develop` and merge requests.

e. Verify the frontend lockfile

ls sail-orbit/frontend/package-lock.json
Confirmed that `package-lock.json` exists. Both CI configurations use `npm ci` to install the dependencies recorded in this file.

f. Run and correct Python checks

From `sail-orbit`:
black --check .
ruff check .
Initial results:

- Black: 8 files require formatting.
- Ruff: 6 lint errors.

Run the automatic corrections:
ruff check . --fix
black .
Results:

- Ruff: 7 issues fixed, 0 remaining.
- Black: 7 files reformatted, 29 unchanged.

Verify:
ruff check .
black --check .
PASS

g. Run ESLint

From `sail-orbit/frontend`:
npm run lint
ESLint identified two errors in `src/components/ui/button.jsx`:

- Unused `React` import.
- `react-refresh/only-export-components` rule triggered by the `buttonVariants` export.

Defer these issues until the frontend components are developed further.

Step 17: Build user profile and account screens

a. Add API client functions for retrieving and updating the authenticated user's profile and activating or deactivating accounts.
b. Create the authenticated user's profile screen to display username, email, bio, role, and account status.
Create `frontend/src/components/UserProfile.jsx`.
This component will retrieve the logged-in user's profile using `getOwnProfile()` and display their username, email, biography, role, and account status.
c. Add profile editing functionality to update and save the user's bio.
Update `frontend/src/components/UserProfile.jsx` to support biography editing.

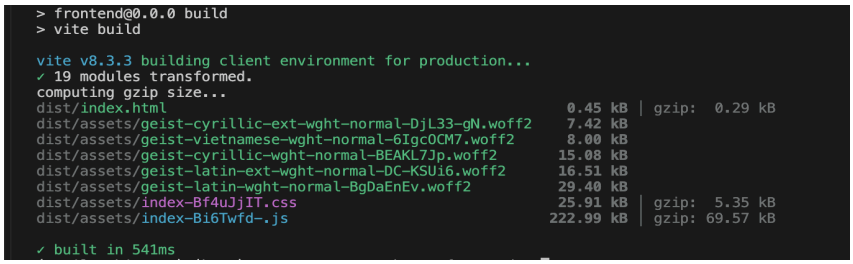
1. Added React state variables (`bio`, `editing`, `saving`, `saveError`, and `saveSuccess`) to manage the editor, API requests, and feedback messages.
2. Updated `loadProfile()` to initialize the biography editor with the existing profile data returned by `getOwnProfile()`.
3. Added the documented `handleSave()` function to submit biography changes through `updateOwnProfile()`, update the displayed profile with the saved response, and error handling.
4. Replaced the static biography display with a conditional interface containing a text area, Save button, and Cancel button.

5. Added loading protection against duplicate submissions, error messages, and confirmation of successful updates. Cancel restores the previously saved biography.
    a. `setSaving(true)` marks the save operation as in progress.
    b. `disabled={saving}` disables both Save and Cancel while the request runs.
    c. `setSaving(false)` re-enables the buttons when the request finishes.
6. Formatted `UserProfile.jsx` using Prettier:
   ```
   npx prettier --write sail-orbit/frontend/src/components/UserProfile.jsx
   ```

The component supports biography editing through the existing `PATCH /users/me` endpoint.

d. Create public profile screens to display usernames, biographies, and account creation dates.

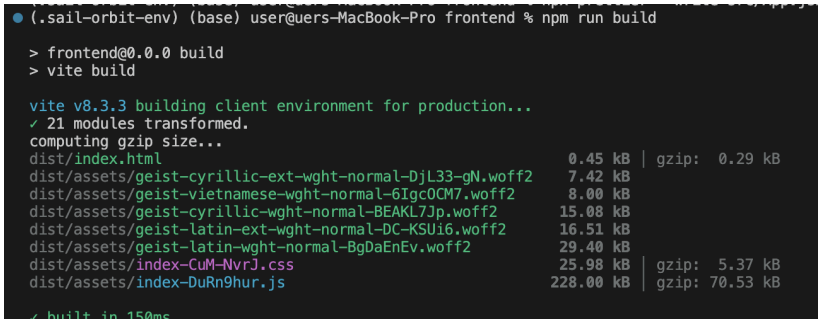
1. Create `frontend/src/components/PublicProfile.jsx`.
2. Import `useEffect`, `useState`, and the existing `getPublicProfile()` API function.
3. Add state variables for profile data, loading status, and errors.
4. Implement `loadProfile()` to retrieve public profile data using the supplied username.
5. Add cleanup logic to prevent outdated API responses from updating the component.
6. Display the username, biography, and account creation date.
7. Add loading, error, and unavailable-profile messages.
8. Format the file using Prettier:
   ```
   npx prettier --write sail-orbit/frontend/src/components/PublicProfile.jsx
   ```
9. Verify the frontend build:
   ```
   cd sail-orbit/frontend
   npm run build
   ```



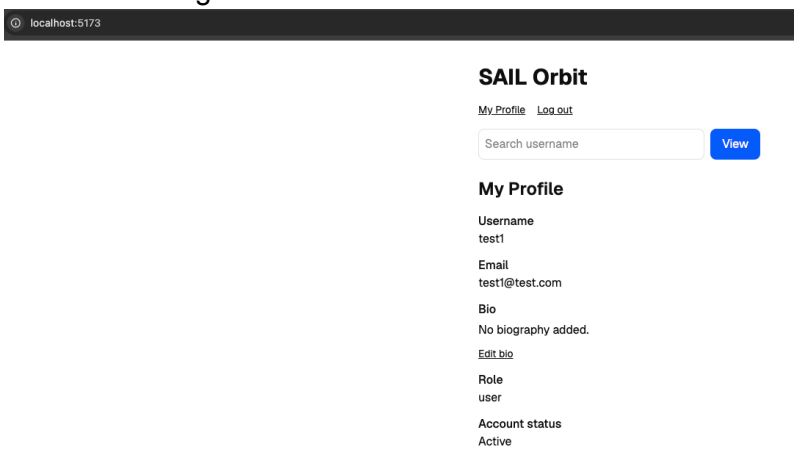
e. Add navigation between the authenticated user's profile and public profiles.

1. Update `frontend/src/App.jsx` to import `UserProfile` and `PublicProfile`.
2. Add state to track the active screen and the username of the selected public profile.
3. Add navigation controls for switching between the authenticated user's profile and public profiles.
4. Add a username input to allow users to look up public profiles.
5. Pass the JWT access token to `UserProfile` and the selected username to `PublicProfile`.
6. Add a Back to My Profile button on public profile screens.
7. Preserve the existing login, registration, and logout behavior when switching screens.

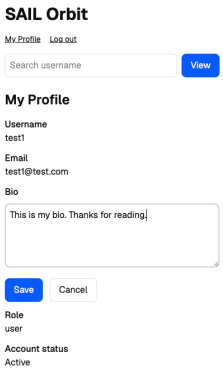
8. Format the updated file using Prettier.
9. Run `npm run build` to verify the frontend compiles.



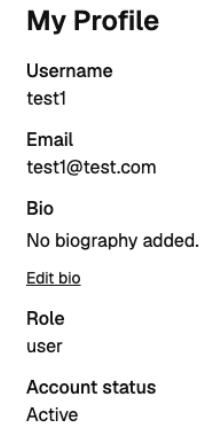
10. Test navigation between the authenticated user's profile and public profiles when the application is running locally.
    From `sail-orbit`, start FastAPI: `uvicorn app.run:app --reload`
    In a second terminal, from `sail-orbit/frontend`: `npm run dev`
    i. Successful login



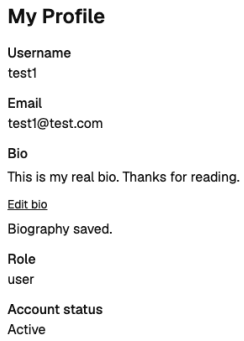
ii. Edit bio button works



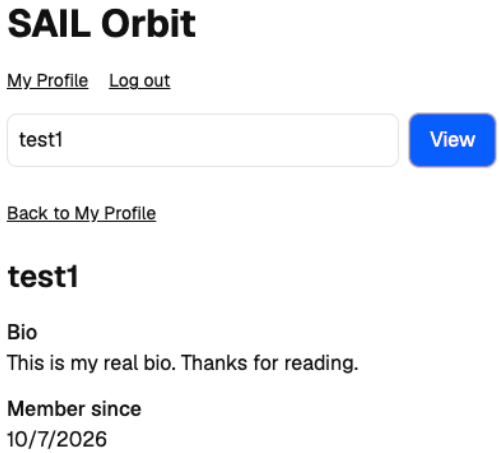
iii. Cancel closes edit without saving



iv. Edit and Save works



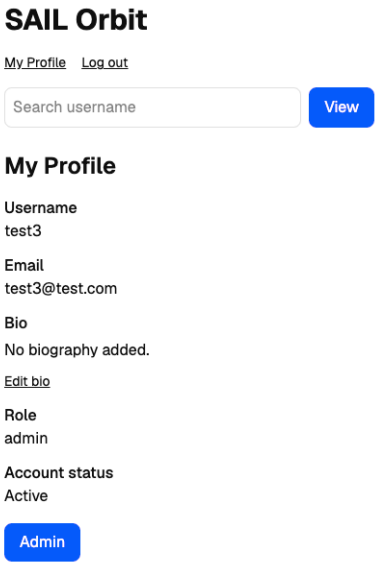
v. Search profile does return the updated profile

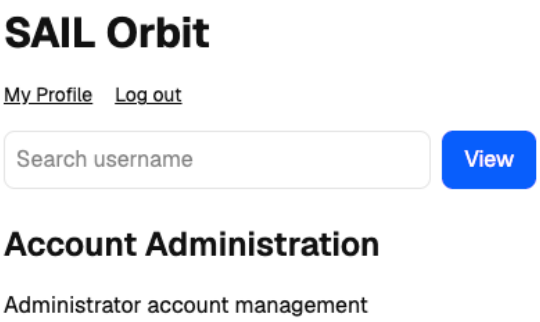


f. Implement role-based interface behavior to display administrator controls only for administrator accounts.

1. Use the existing `role` field returned by `getOwnProfile()` to determine whether the authenticated user is an administrator.
2. Update `UserProfile.jsx` to display an Admin control when `profile.role === "admin"`, using an `onOpenAdmin` callback for navigation.
3. Update `App.jsx` to support the administration screen alongside existing own-profile and public-profile screens, preserving profile lookup, My Profile, and logout.

4. Inspect the backend user model and database configuration, then create an administrator test account using the existing registration flow and assign its `admin` role in PostgreSQL.
5. Format the modified files with Prettier and run `npm run build`.
6. Test with regular and administrator accounts to verify role-based visibility, administration screen navigation, and existing profile and authentication functionality.





Updated `UserProfile.jsx` and `App.jsx` to support administrator-only navigation using the authenticated user's existing `role` field.
Configured the PostgreSQL test accounts:

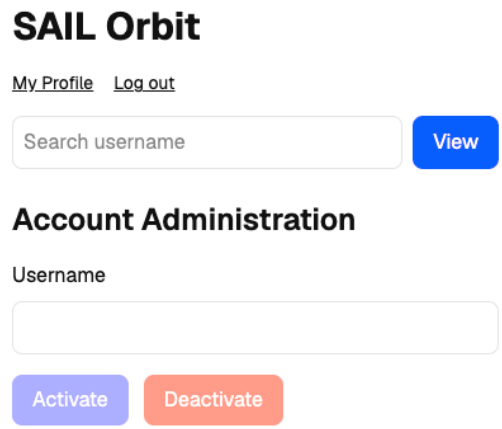
- `test3`: administrator (`admin`)
- `test1`, `test2`, `orbit_test_01`: regular users (`user`)

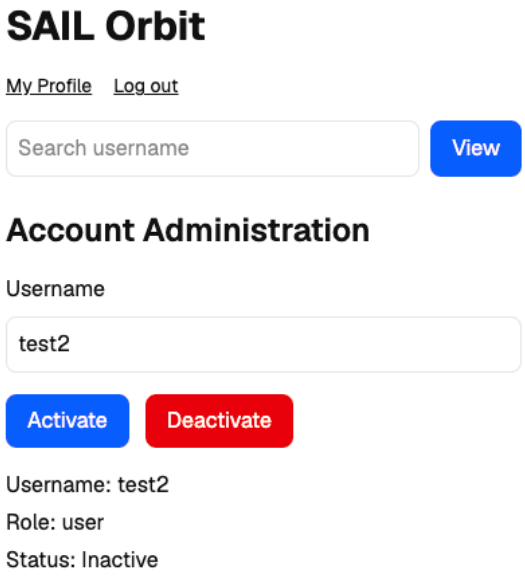
Verified that the Admin button appears for administrators, opens the Account Administration screen, and remains hidden for regular users. Existing profile navigation was also verified.
g. Create the administrator account management interface to activate and deactivate user accounts.

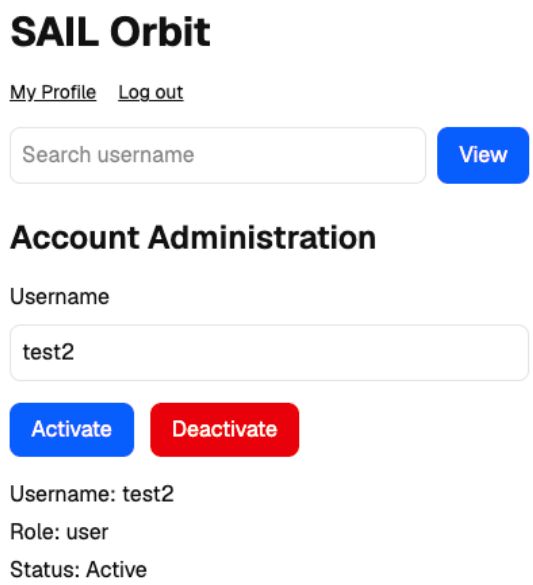
1. run: `grep -n -A 35 'export async function updateUserStatus' frontend/src/api.js` to inspect existing API function
2. create `frontend/src/components/AdminAccounts.jsx` and connect it to the Admin screen in `App.jsx`
    a. Inspect the existing backend endpoint: `grep -n -B 12 -A 55 'def update_user_status' app/api/users.py`

The backend confirms that administrators can activate or deactivate accounts, including currently inactive accounts. It also returns the username, role, and updated status. This interface uses the existing API to change an account's status. It displays the account's confirmed status after an update, rather than claiming to know its status beforehand.

a. Add this import immediately after the existing PublicProfile import:
```
import AdminAccounts from "./components/AdminAccounts";
```
b. Find this existing block in App.jsx:
```
      ) : profileScreen === "admin" ? (
        <section className="space-y-4">
          <h2 className="text-2xl font-semibold">Account Administration</h2>
          <p>Administrator account management</p>
        </section>
```
c. Replace it with:
```
      ) : profileScreen === "admin" ? (
        <AdminAccounts token={accessToken} />
```
d. Format and build from sail-orbit/frontend:
```
npx prettier --write src/App.jsx src/components/AdminAccounts.jsx
npm run build
```



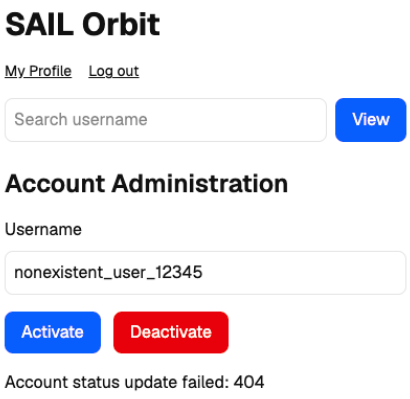


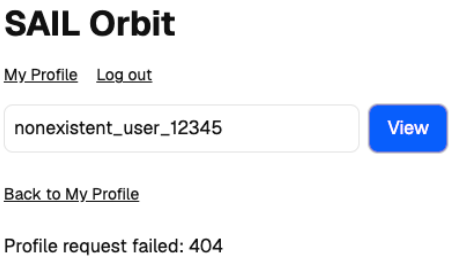


Deactivate: test2 successfully changed to Inactive.
Activate: test2 successfully returned to Active.
Administrator authorization: test3 successfully performed both operations.

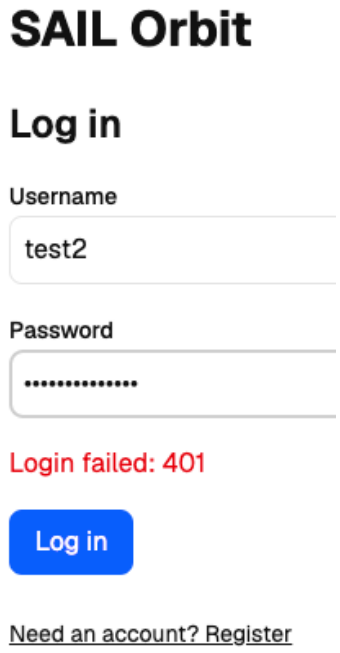
h. Add loading, success, and error handling for profile and account operations.

Entered a nonexistent username in the administrator screen and confirmed that deactivation returned a 404 error, then searched for a nonexistent public profile, confirmed another 404 error, and verified that Back to My Profile still worked.

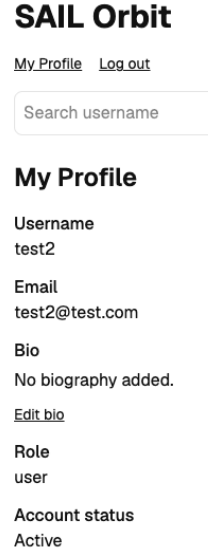




i. Verify authentication integration and complete local testing.
Logged in as admin (test3) and deactivated test2. Verified test2 cannot log in.



Reactivated and confirmed logged in as test2



NOTE: Some vulnerabilities found due to some depreciation of dependencies. See Appendix.

j. Frontend automated testing

The backend's 27 relevant tests already pass. Vitest and React Testing Library are installed, so we can finish by configuring the test runner and testing the Step 17 React components.

1. Configure Vitest

From `sail-orbit/frontend`, create `vitest.config.js`:

```
import { defineConfig } from "vitest/config";
import react from "@vitejs/plugin-react";

export default defineConfig({
```

```
  plugins: [react()],
  esbuild: {
    jsx: "automatic",
  },
  test: {
    environment: "jsdom",
    globals: true,
    clearMocks: true,
  },
});
```

2. Add the test command

In `package.json`, add `"test": "vitest run"` to the existing `scripts` section:

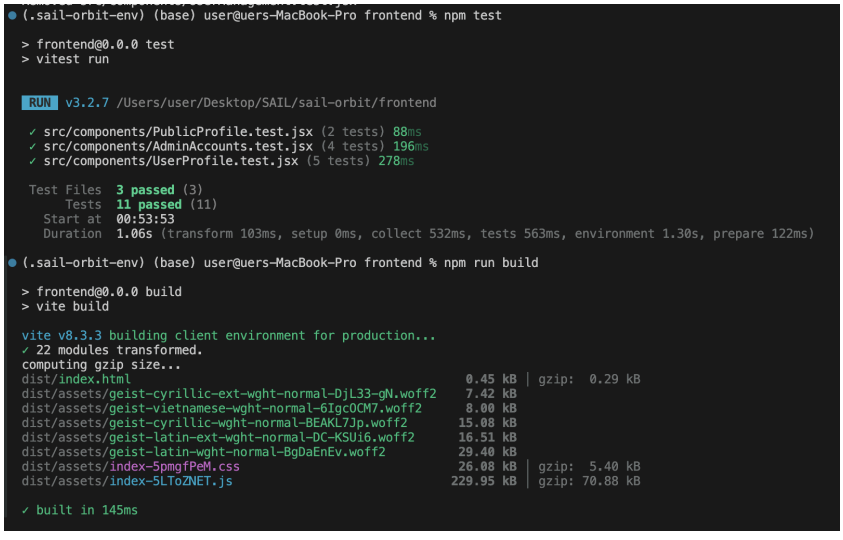
```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "lint": "eslint .",
  "preview": "vite preview",
  "test": "vitest run"
}
```

3. Get the component code

To write tests that match the actual components rather than guessing their markup, run: cat src/components/UserProfile.jsx src/components/PublicProfile.jsx src/components/AdminAccounts.jsx

4. Initially all 11 tests were put into UserProfile.test.jsx, but needed to split into more files since it got too long. See docs -> python_scripts ->test_separation_script.py for script run in CLI at frontend dir

Tests passed



Step 18: Build posts and associated API endpoints, schemas, frontend functionality, and tests.
Step 19: Add comments and reactions, i.e. data models, API endpoints, schemas, frontend functionality, and tests.
Add follow and unfollow relationships, i.e. data models, API endpoints, schemas, frontend functionality, and tests.
Build the chronological feed, including backend feed logic, API endpoints, schemas, frontend functionality, and tests.
Complete frontend navigation and remaining user-interface behavior.
Add application and security-relevant logging.
Add health endpoints and application error handling.
Complete remaining automated and integration testing.
Run Black and Ruff against the complete application.
Run and test the complete application locally, i.e. frontend, FastAPI, and PostgreSQL.
Verify FastAPI validation and OpenAPI documentation.
Generate sustained automated user activity and verify Orbit under a variable workload.
Containerize the completed application.
Deploy Orbit and PostgreSQL to the k3s cluster.
Verify the deployed application is reachable, persistent, and functioning correctly.

Section 9: Monitoring, Logging, & Observability

Overview

At this juncture, SAIL establishes centralized monitoring and observability for the Raspberry Pi nodes, Kubernetes cluster, Orbit application, and PostgreSQL database. Prometheus, Grafana, Loki, and OpenTelemetry provide metrics, logs, and traces to monitor system health, analyze performance, investigate failures, and identify abnormal activity.

The implementation begins with cluster verification and resource assessment, followed by deployment and configuration of monitoring components using Helm. Grafana provides centralized access to operational data, while Alertmanager handles notifications. Persistent storage, retention policies, and operating baselines support ongoing troubleshooting and the investigation workflows in Section 12.

Tools

- **Prometheus:** Collects and stores infrastructure, Kubernetes, application, and database metrics.
- **Grafana:** Provides dashboards and tools for exploring metrics, logs, and traces.
- **Node Exporter:** Exposes Raspberry Pi CPU, memory, disk, and network metrics.
- **kube-state-metrics:** Exposes Kubernetes resource and workload metrics.
- **Loki:** Stores and queries centralized logs.
- **Log collector:** Forwards Kubernetes container and system logs to Loki.
- **OpenTelemetry:** Instruments applications and collects and exports traces.
- **Trace backend:** Stores and retrieves application traces.

- **PostgreSQL exporter:** Exposes database performance and operational metrics.
- **Alertmanager:** Processes alerts and delivers notifications.
- **Helm:** Deploys and manages monitoring components in Kubernetes.

Implementation

Step 1: Power on the SAIL cluster and verify network connectivity
a. Confirm the Samsung T7 SSD is connected to pi-node-1.
b. Verify all three Raspberry Pis are connected to the network switch using Ethernet.
c. Power on the SAIL tower and allow all three nodes to boot.
d. Verify network connectivity to `10.0.0.20`, `10.0.0.21`, and `10.0.0.22` using `ping`.
Step 2: Verify SSH access, k3s services, and Kubernetes connectivity
a. Connect to each Raspberry Pi using SSH with the `sailadmin` username and password.
b. Verify the `k3s` service is active on pi-node-1.
c. Verify the `k3s-agent` service is active on pi-node-2 and pi-node-3.
d. Verify the Kubernetes API server is accessible using `kubectl cluster-info`.
e. Confirm all three nodes report `Ready` using `kubectl get nodes`.
Step 3: Verify SSD, NFS, and Kubernetes storage
a. Verify the external SSD is mounted at `/mnt/sail-storage` on pi-node-1.
b. Check available SSD capacity using `df -h`.
c. Confirm the NFS server is active and `/mnt/sail-storage/kubernetes` is exported.
d. Verify the `sail-nfs` Kubernetes StorageClass exists.
e. Review existing persistent volumes and persistent volume claims for errors.
Step 4: Verify internet access and install or verify Helm
a. Confirm internet connectivity from the administration computer.
b. Verify internet connectivity from pi-node-1 and the worker nodes.
c. Check whether Helm is installed using `helm version`.
d. Install Helm if necessary and verify the installation.
e. Review existing Helm releases using `helm list -A`.
Step 5: Inspect cluster resources, Metrics Server, and existing workloads
a. List running Kubernetes pods, deployments, and services.
b. Verify Metrics Server is available in the cluster.
c. Review CPU and memory utilization using `kubectl top nodes` and `kubectl top pods -A`.
d. Inspect node capacity, allocatable resources, and current resource requests.
e. Identify existing workloads and available resources for monitoring components.
Step 6: Plan monitoring deployment, storage, and resource allocations
a. Determine the Kubernetes namespace for monitoring components.
b. Select compatible Helm charts and ARM64 container images.
c. Establish CPU and memory allocations for Prometheus, Grafana, Loki, and other components.
d. Determine storage requirements for each monitoring component, including whether the existing NFS-backed storage is appropriate.
e. Select initial metrics, log, and trace retention settings.
f. Determine how monitoring interfaces will be accessed.
g. Establish Grafana authentication, credential management, and access restrictions for monitoring interfaces.
Step 7: Deploy Prometheus and configure metrics collection
a. Add the selected Prometheus Helm repository.
b. Configure Prometheus resource requests and limits.

c. Configure persistent metrics storage using a storage arrangement appropriate for Prometheus.
d. Set the initial metrics retention period.
e. Deploy Prometheus to Kubernetes.
f. Verify Prometheus pods and services are running.
g. Access the Prometheus interface and confirm metrics are being collected.

Step 8: Configure Raspberry Pi and Kubernetes metrics

a. Deploy or enable Node Exporter across the three Raspberry Pis.
b. Deploy or enable kube-state-metrics.
c. Configure Prometheus to collect Kubernetes component metrics.
d. Verify CPU, memory, filesystem, and network metrics from all three nodes.
e. Verify Kubernetes node, pod, deployment, and workload metrics.

Step 9: Deploy Grafana and create monitoring dashboards

a. Deploy Grafana using a compatible Helm chart.
b. Configure persistent storage for Grafana settings and dashboards.
c. Configure browser access to Grafana.
d. Add Prometheus as a Grafana data source.
e. Create or import dashboards for Raspberry Pi resource utilization.
f. Create or import Kubernetes dashboards for nodes, pods, and workloads.
g. Include shared storage measurements where available.
h. Verify dashboard readings against actual cluster measurements.

Step 10: Deploy Loki and centralized log collection

a. Select a Loki configuration appropriate for the Raspberry Pi cluster.
b. Configure Loki resource allocations and persistent log storage.
c. Set the initial log retention period.
d. Deploy Loki to Kubernetes.
e. Deploy a compatible log collector.
f. Configure collection of Kubernetes container logs and relevant system logs.
g. Add Loki as a Grafana data source.
h. Verify logs can be searched by node, namespace, and workload.

Step 11: Review monitoring storage, retention, and persistence

a. Review the storage configurations established for Prometheus, Grafana, and Loki.
b. Verify that each component uses its intended persistent storage.
c. Review configured metrics and log retention periods.
d. Verify retained data survives relevant pod restarts.
e. Configure monitoring of storage consumption.
f. Identify any storage configuration or retention adjustments needed before adding tracing.
g. Configure backups for Grafana settings and dashboards.
h. Verify recovery of critical monitoring configurations from backup.

Step 12: Deploy OpenTelemetry and a tracing backend

a. Select a compatible trace storage backend.
b. Configure resource allocations and appropriate persistent trace storage.
c. Establish the initial trace retention period.
d. Deploy an OpenTelemetry Collector if required.
e. Configure telemetry collection and export.
f. Connect the tracing backend to Grafana.
g. Generate test traces and verify they can be retrieved.

Step 13: Instrument Orbit and PostgreSQL

a. Add FastAPI instrumentation for request counts, response times, and errors.
b. Configure structured application logging.

c. Add OpenTelemetry tracing to selected application and database operations.
d. Deploy and configure PostgreSQL metrics collection.
e. Configure Prometheus to collect Orbit and PostgreSQL metrics.
f. Create Grafana dashboards for application and database monitoring.
g. Generate application activity and verify metrics, logs, and traces are collected.

Step 14: Configure monitoring alerts

a. Configure Prometheus alerting rules and Alertmanager.
b. Create alerts for unavailable nodes and services.
c. Configure CPU, memory, and storage utilization alerts.
d. Configure Orbit error-rate and PostgreSQL availability alerts.
e. Configure alert notification delivery.
f. Test alert activation and resolution using controlled conditions.

Step 15: Establish normal operating baselines

a. Record normal CPU and memory utilization across all three Raspberry Pis.
b. Record filesystem and network activity.
c. Record Kubernetes workload resource consumption and pod behavior.
d. Review typical log activity and volume.
e. Record Orbit response times, error rates, and PostgreSQL activity.
f. Document observation periods and operating conditions for future comparison.

Step 16: Verify monitoring end-to-end

a. Confirm metrics are collected from all three Raspberry Pi nodes.
b. Verify Kubernetes, Orbit, and PostgreSQL dashboards.
c. Generate and retrieve centralized container logs.
d. Generate and retrieve application traces.
e. Verify monitoring data persistence after relevant pod restarts.
f. Trigger controlled alert conditions and verify notifications.
g. Restore normal conditions and confirm alerts resolve.
h. Record verification results and remaining monitoring gaps.

End State: Grafana provides centralized visibility into SAIL infrastructure, Kubernetes, Orbit, and PostgreSQL through metrics, logs, traces, and alerts. Normal operating baselines are available for comparison during Section 12.

Section 10: Security & Infrastructure Automation

Purpose: Add security visibility and make configuration/security work reproducible rather than manually administered.

Include:

- Falco: runtime behavior detection
- Trivy: image/configuration/vulnerability scanning
- Ansible: system/configuration automation
- Terraform/OpenTofu: IaC
- Bash: appropriate system automation
- Python: more complex automation

- Security scanning incorporated into the application lifecycle
- Establish a **known security baseline**

End state: We can identify vulnerable configurations/images and suspicious runtime activity, while demonstrating repeatable infrastructure/configuration management.

Section 11: AI-Assisted Operations & Deployment Automation

This is where several pieces we've discussed come together.

Purpose: Automate application delivery and give local AI controlled access to SAIL operational information.

Include:

- GitHub Actions: CI/CD
- Helm: Kubernetes application packaging/configuration
- Argo CD: GitOps deployment
- Ollama: local model runtime
- Open WebUI: human AI interface
- **MCP:** controlled AI access to selected SAIL data/tools
- **Deterministic verification:** verify important AI claims against actual system data
- **AI provenance:** retain evidence supplied to the AI and resulting analysis
- **Human approval gates:** AI can recommend consequential remediation; you authorize it
- **Agent observability:** record AI request success/failure and latency

End state: A code change can travel through a professional automated deployment process, while the local AI can perform evidence-based analysis of the environment without being given unrestricted control.

Section 12: Event Scenarios and Investigation Workflows

The purpose of this section is to document the operational and security scenarios used to validate the SAIL environment. Each scenario will include event generation, detection mechanisms, collected evidence, AI-assisted analysis, and remediation actions.

Planned scenarios include:

Scenario	Category
Worker node failure	Infrastructure

Pod crash loop	Infrastructure
Storage utilization threshold exceeded	Infrastructure
Unauthorized shell access to a container	Security
Deployment of a vulnerable container image	Security
Excessive resource consumption by a workload	Security / Operations
Unexpected process execution within a container	Security
Service outage and recovery validation	Operations

Each scenario will document:

1. Event description
2. Event generation method
3. Detection mechanism
4. Logs and metrics collected
5. AI-generated analysis
6. Investigation process
7. Remediation actions
8. Lessons learned

Appendix:

Section 5 Step 7: `sail-nfs-client-setup.sh`

```
#!/usr/bin/env bash

################################################################################
# SAIL - NFS Client Setup and Verification
#
# Description:
#   Configures NFS client support across the three-node SAIL Kubernetes cluster
#   and verifies that the worker nodes can access the shared NFS storage hosted
#   by pi-node-1.
#
#   The script:
#     1. Installs nfs-common on all three Raspberry Pi nodes.
#     2. Mounts the SAIL NFS share temporarily from pi-node-2.
#     3. Verifies pi-node-2 can write to the shared storage.
#     4. Mounts the same share temporarily from pi-node-3.
#     5. Verifies pi-node-3 can see data written by pi-node-2.
#     6. Verifies pi-node-3 can write to the shared storage.
#     7. Unmounts temporary test mounts when testing is complete.
#
# Requirements:
#   - Run from the SAIL administration MacBook.
#   - SSH access to all three Raspberry Pi nodes must already be configured.
#   - pi-node-1 must already be exporting the NFS share.
#   - The SAIL nodes must be reachable at the addresses defined below.
#
# Usage:
#   chmod +x sail-nfs-client-setup.sh
#   ./sail-nfs-client-setup.sh
#
################################################################################

set -e

# ------------------------------------------------------------------------------
# Configuration
# ------------------------------------------------------------------------------

SSH_USER="sailadmin"

NODE1="10.0.0.20"
NODE2="10.0.0.21"
NODE3="10.0.0.22"

NFS_SERVER="$NODE1"
NFS_SHARE="/mnt/sail-storage/kubernetes"
TEST_MOUNT="/mnt/sail-nfs-test"


# ------------------------------------------------------------------------------
# Functions
# ------------------------------------------------------------------------------

# Display a section heading in the terminal output.
```

```
section() {
   echo
   echo "========================================"
   echo "$1"
   echo "========================================"
}


# Install the NFS client utilities on a SAIL node.
install_nfs_client() {
   local node="$1"

   echo "Configuring ${node}..."

   ssh -t "${SSH_USER}@${node}" \
       "sudo apt update && sudo apt install -y nfs-common"

   echo "NFS client configuration completed on ${node}."
}


# ------------------------------------------------------------------------------
# NFS Client Installation
# ------------------------------------------------------------------------------

section "Installing NFS client utilities"

for node in "$NODE1" "$NODE2" "$NODE3"; do
   install_nfs_client "$node"
done


# ------------------------------------------------------------------------------
# pi-node-2 NFS Verification
# ------------------------------------------------------------------------------

section "Testing NFS access from pi-node-2"

ssh -t "${SSH_USER}@${NODE2}" "
   set -e

   # Create a temporary mount point for the NFS verification.
   sudo mkdir -p ${TEST_MOUNT}

   # Mount the SSD-backed NFS share hosted by pi-node-1.
   sudo mount -t nfs -o vers=4.1 \
       ${NFS_SERVER}:${NFS_SHARE} \
       ${TEST_MOUNT}

   echo 'NFS share mounted on pi-node-2.'

   # Display the active mount for verification.
   findmnt ${TEST_MOUNT}

   # Write a file that will later be verified from pi-node-3.
   touch ${TEST_MOUNT}/pi-node-2-test
```

```
   if [ ! -f ${TEST_MOUNT}/pi-node-2-test ]; then
       echo 'ERROR: pi-node-2 test file was not created.'
       exit 1
   fi

   echo 'pi-node-2 test file created successfully.'

   ls -l ${TEST_MOUNT}

   # The mount is required only for this verification.
   sudo umount ${TEST_MOUNT}

   echo 'pi-node-2 NFS verification completed successfully.'
"


# ------------------------------------------------------------------------------
# pi-node-3 NFS Verification
# ------------------------------------------------------------------------------

section "Testing NFS access from pi-node-3"

ssh -t "${SSH_USER}@${NODE3}" "
   set -e

   # Create a temporary mount point for the NFS verification.
   sudo mkdir -p ${TEST_MOUNT}

   # Mount the same SSD-backed NFS share.
   sudo mount -t nfs -o vers=4.1 \
       ${NFS_SERVER}:${NFS_SHARE} \
       ${TEST_MOUNT}

   echo 'NFS share mounted on pi-node-3.'

   findmnt ${TEST_MOUNT}

   # Confirm that data written from pi-node-2 is visible from pi-node-3.
   if [ ! -f ${TEST_MOUNT}/pi-node-2-test ]; then
       echo 'ERROR: pi-node-3 cannot see the file created by pi-node-2.'
       exit 1
   fi

   echo 'pi-node-3 can see pi-node-2-test.'

   # Confirm that pi-node-3 can also write to the shared storage.
   touch ${TEST_MOUNT}/pi-node-3-test

   if [ ! -f ${TEST_MOUNT}/pi-node-3-test ]; then
       echo 'ERROR: pi-node-3 test file was not created.'
       exit 1
   fi

   echo 'pi-node-3 test file created successfully.'

   ls -l ${TEST_MOUNT}
```

```
   # Remove the temporary mount after verification.
   sudo umount ${TEST_MOUNT}

   echo 'pi-node-3 NFS verification completed successfully.'
"


# ------------------------------------------------------------------------------
# Results
# ------------------------------------------------------------------------------

section "NFS client setup and verification complete"

echo "Verified:"
echo "  - nfs-common installed on all three nodes"
echo "  - pi-node-2 can mount the NFS share"
echo "  - pi-node-2 can write to shared storage"
echo "  - pi-node-3 can mount the NFS share"
echo "  - pi-node-3 can read data written by pi-node-2"
echo "  - pi-node-3 can write to shared storage"
```

Section 6 Step 2: `sail-k3s-prerequisites.sh`

```
#!/usr/bin/env bash

################################################################################
# SAIL - k3s Prerequisite Configuration
#
# Description:
#   Verifies and configures the Raspberry Pi OS cgroup settings required by
#   k3s across all three SAIL Kubernetes nodes.
#
#   The script:
#     1. Connects to each Raspberry Pi over SSH.
#     2. Checks /boot/firmware/cmdline.txt for the required memory cgroup
#        parameters.
#     3. Adds only parameters that are missing.
#     4. Preserves cmdline.txt as a single logical line.
#     5. Verifies the resulting configuration.
#     6. Reports which nodes require a reboot before k3s installation.
#
# Requirements:
#   - Run from the SAIL administration MacBook.
#   - SSH access to all three Raspberry Pi nodes must already work.
#   - Raspberry Pi OS must already be installed on the nodes.
#
# Usage:
#   chmod +x sail-k3s-prerequisites.sh
#   ./sail-k3s-prerequisites.sh
#
################################################################################

set -uo pipefail

# ------------------------------------------------------------------------------
# Configuration
# ------------------------------------------------------------------------------
```

```
SSH_USER="sailadmin"

NODES=(
   "10.0.0.20"
   "10.0.0.21"
   "10.0.0.22"
)

CMDLINE_FILE="/boot/firmware/cmdline.txt"

MODIFIED_NODES=()


# ------------------------------------------------------------------------------
# Functions
# ------------------------------------------------------------------------------

# Display a section heading in terminal output.
section() {
   echo
   echo "========================================"
   echo "$1"
   echo "========================================"
}


# Verify and configure one Raspberry Pi node in a single SSH session.
configure_node() {
   local node="$1"
   local status

   section "Checking ${node}"

   ssh -t "${SSH_USER}@${node}" \
       "CMDLINE_FILE='${CMDLINE_FILE}' bash -s" <<'REMOTE_SCRIPT'

set -e

REQUIRED_CGROUP_MEMORY="cgroup_memory=1"
REQUIRED_CGROUP_ENABLE="cgroup_enable=memory"

cmdline=(cat"CMDLINE_FILE")

echo "Current configuration:"
echo "$cmdline"
echo

missing=()

# Check each required parameter independently.
if [[ " $cmdline " != *" $REQUIRED_CGROUP_MEMORY "* ]]; then
   missing+=("$REQUIRED_CGROUP_MEMORY")
fi

if [[ " $cmdline " != *" $REQUIRED_CGROUP_ENABLE "* ]]; then
   missing+=("$REQUIRED_CGROUP_ENABLE")
fi

# No modification is necessary when both parameters are already present.
if [[ ${#missing[@]} -eq 0 ]]; then
   echo "PASS: Required cgroup parameters are already present."
   exit 0
fi
```

```
echo "Missing parameter(s): ${missing[*]}"
echo "Updating $CMDLINE_FILE..."

additions="${missing[*]}"

# Create a backup before changing the boot configuration.
sudo cp "CMDLINE_FILE""{CMDLINE_FILE}.sail-backup"

# Append only the missing parameters to the existing command line.
sudo sed -i "1 s|[[:space:]]*$| additions|""CMDLINE_FILE"

# Verify that cmdline.txt still contains exactly one logical line.
line_count=(awk'ENDprintNR'"CMDLINE_FILE")

if [[ "$line_count" -ne 1 ]]; then
   echo "ERROR: cmdline.txt no longer contains exactly one logical line."
   exit 1
fi

# Verify both required parameters are now present.
updated_cmdline=(cat"CMDLINE_FILE")

if [[ " $updated_cmdline " != *" $REQUIRED_CGROUP_MEMORY "* ]] ||
  [[ " $updated_cmdline " != *" $REQUIRED_CGROUP_ENABLE "* ]]; then
   echo "ERROR: Required cgroup parameters were not successfully configured."
   exit 1
fi

echo
echo "PASS: Required cgroup parameters are now configured."
echo "Updated configuration:"
echo "$updated_cmdline"

# Exit code 10 tells the Mac-side script that this node was modified.
exit 10

REMOTE_SCRIPT

   status=$?

   case "$status" in
       0)
           # Node was already configured.
           ;;
       10)
           MODIFIED_NODES+=("$node")
           ;;
       *)
           echo
           echo "ERROR: Prerequisite configuration failed on ${node}."
           exit "$status"
           ;;
   esac
}


# ------------------------------------------------------------------------------
# Main
# ------------------------------------------------------------------------------

section "SAIL k3s prerequisite configuration"
```

```
for node in "${NODES[@]}"; do
   configure_node "$node"
done


# ------------------------------------------------------------------------------
# Results
# ------------------------------------------------------------------------------

section "k3s prerequisite verification complete"

if [[ ${#MODIFIED_NODES[@]} -eq 0 ]]; then
   echo "All three nodes are configured correctly."
   echo "No reboot is required."
else
   echo "The following node(s) were modified and must be rebooted:"
   printf '  - %s\n' "${MODIFIED_NODES[@]}"

   echo
   echo "Reboot these nodes before installing k3s."
   echo "After they return, run this script again to confirm all nodes pass."
fi
```

Section 6 Step 3: `sail-k3s-cluster-setup.sh`

```
#!/usr/bin/env bash

################################################################################
# SAIL - k3s Cluster Setup
#
# Installs and verifies the three-node SAIL k3s cluster.
#
#   pi-node-1  10.0.0.20  k3s server/control plane
#   pi-node-2  10.0.0.21  k3s agent
#   pi-node-3  10.0.0.22  k3s agent
#
# SSH connections are reused during the script to avoid repeated SSH
# password prompts.
################################################################################

set -euo pipefail

SSH_USER="sailadmin"
SERVER="10.0.0.20"
NODE2="10.0.0.21"
NODE3="10.0.0.22"

# Reuse SSH connections for 10 minutes.
SSH_OPTS="-o ControlMaster=auto -o ControlPersist=10m -o
ControlPath=/tmp/sail-ssh-%r@%h:%p"


# ------------------------------------------------------------------------------
# Cleanup
# ------------------------------------------------------------------------------

cleanup() {
   for node in "$SERVER" "$NODE2" "$NODE3"; do
       ssh $SSH_OPTS -O exit "${SSH_USER}@${node}" 2>/dev/null || true
   done
}
```

```
trap cleanup EXIT


# ------------------------------------------------------------------------------
# Establish reusable SSH connections
# ------------------------------------------------------------------------------

echo
echo "========================================"
echo "Opening SSH connections"
echo "========================================"

for node in "$SERVER" "$NODE2" "$NODE3"; do
   echo "Connecting to ${node}..."
   ssh $SSH_OPTS -Nf "${SSH_USER}@${node}"
done


# ------------------------------------------------------------------------------
# Install k3s server on pi-node-1
# ------------------------------------------------------------------------------

echo
echo "========================================"
echo "Installing k3s on pi-node-1"
echo "========================================"

ssh -t $SSH_OPTS "${SSH_USER}@${SERVER}" \
   "curl -sfL https://get.k3s.io | sh -"


# ------------------------------------------------------------------------------
# Wait for control plane
# ------------------------------------------------------------------------------

echo
echo "Waiting for pi-node-1 to become Ready..."

for attempt in {1..36}; do

   if ssh $SSH_OPTS "${SSH_USER}@${SERVER}" \
       "sudo k3s kubectl get node pi-node-1 \
       -o jsonpath='{.status.conditions[?(@.type==\"Ready\")].status}'" \
       2>/dev/null | grep -q True; then

       echo "pi-node-1 is Ready."
       break
   fi

   if [[ "$attempt" -eq 36 ]]; then
       echo "ERROR: pi-node-1 did not become Ready."
       exit 1
   fi

   sleep 5
done


# ------------------------------------------------------------------------------
# Retrieve cluster token
# ------------------------------------------------------------------------------

echo
```

```
echo "========================================="
echo "Retrieving cluster join token"
echo "========================================="

K3S_TOKEN=$(
   ssh $SSH_OPTS "${SSH_USER}@${SERVER}" \
       "sudo cat /var/lib/rancher/k3s/server/node-token"
)

if [[ -z "$K3S_TOKEN" ]]; then
   echo "ERROR: Could not retrieve the cluster join token."
   exit 1
fi

echo "Cluster join token retrieved."


# ------------------------------------------------------------------------------
# Install worker nodes
# ------------------------------------------------------------------------------

echo
echo "========================================="
echo "Installing k3s on pi-node-2"
echo "========================================="

ssh -t $SSH_OPTS "${SSH_USER}@${NODE2}" \
   "curl -sfL https://get.k3s.io | \
   K3S_URL='https://${SERVER}:6443' \
   K3S_TOKEN='${K3S_TOKEN}' \
   sh -"


echo
echo "========================================="
echo "Installing k3s on pi-node-3"
echo "========================================="

ssh -t $SSH_OPTS "${SSH_USER}@${NODE3}" \
   "curl -sfL https://get.k3s.io | \
   K3S_URL='https://${SERVER}:6443' \
   K3S_TOKEN='${K3S_TOKEN}' \
   sh -"


# ------------------------------------------------------------------------------
# Verify cluster
# ------------------------------------------------------------------------------

echo
echo "========================================="
echo "Verifying cluster"
echo "========================================="

for attempt in {1..36}; do

   READY_COUNT=$(
       ssh $SSH_OPTS "${SSH_USER}@${SERVER}" \
           "sudo k3s kubectl get nodes --no-headers | \
           awk '\$2 == \"Ready\" {count++} END {print count+0}'"
   )

   if [[ "$READY_COUNT" -eq 3 ]]; then
```

```
        break
    fi

    echo "Ready nodes: ${READY_COUNT}/3"

    if [[ "$attempt" -eq 36 ]]; then
        echo "ERROR: Not all nodes became Ready."
        exit 1
    fi

    sleep 5
done


# ------------------------------------------------------------------------------
# Final result
# ------------------------------------------------------------------------------

ssh $SSH_OPTS "${SSH_USER}@${SERVER}" \
    "sudo k3s kubectl get nodes"

echo
echo "========================================"
echo "k3s cluster setup complete"
echo "========================================"

echo "Verified:"
echo "  - k3s server installed on pi-node-1"
echo "  - k3s agent installed on pi-node-2"
echo "  - k3s agent installed on pi-node-3"
echo "  - all three nodes joined the cluster"
echo "  - all three nodes report Ready"
```

Removal of Display_name from files and database

Section 8: SAIL Orbit: Removal of display_name

Status: Backend changes and database migration completed
Migration: `62ce8ce120c2`
Test results: 40 passed
1. Identify References to `display_name`
From the `sail-orbit` project root, ran:

```
grep -Rn --exclude-dir=node_modules --exclude-dir=.git
--exclude-dir=.sail-orbit-env "display_name" .
```

The command recursively searched for `display_name`, excluding the Node.js dependencies, Git directory, and Python virtual environment.

The search identified references in:

- `app/models/user.py`
- `app/schemas/user.py`
- `app/api/auth.py`
- `app/api/users.py`
- `frontend/src/components/RegisterForm.jsx`
- `frontend/src/api.js`
- `tests/test_auth.py`

- `tests/test_users.py`
- `tests/test_user_profiles.py`
- `tests/test_user_admin.py`
- `migrations/versions/d383129da037_create_users_table.py`

2. Remove `display_name` From Application Code
Updated the affected files:

File	Changes
`app/models/user.py`	Removed the SQLAlchemy display_name column
`app/schemas/user.py`	Removed display_name from UserRegistration and UserProfileUpdate
`app/api/auth.py`	Removed display_name from registration and its response
`app/api/users.py`	Removed display_name from profile and account responses
`frontend/src/components/RegisterForm.jsx`	Removed the display name registration field
`frontend/src/api.js`	Removed the obsolete display_name reference in the API documentation
`tests/test_auth.py`	Removed display_name references
`tests/test_users.py`	Removed display_name from registration requests and response assertions
`tests/test_user_profiles.py`	Removed display_name references and deleted test_update_own_profile_partially
`tests/test_user_admin.py`	Removed display_name references

The `username` field remains the unique login identifier and is included in public profile API responses. The `bio` field was retained.
The original migration, `d383129da037_create_users_table.py`, was left unchanged.

3. Generate the Alembic Migration
Ran:

```
alembic revision --autogenerate -m "remove display_name from users"
```

Alembic generated:

```
migrations/versions/62ce8ce120c2_remove_display_name_from_users.py
```

The output included:

```
Detected removed table 'users'
```

The generated `upgrade()` contained:

```
op.drop_table('users')
```

This would have removed the entire `users` table, so was corrected before execution.

4. Edit the Migration File

The migration file was opened in TextEdit:

```
open -e migrations/versions/62ce8ce120c2_remove_display_name_from_users.py
```

The autogenerated table removal and recreation operations were replaced.

Corrected upgrade:

```
def upgrade() -> None:
    """Remove the display_name column from the users table."""
    op.drop_column("users", "display_name")
```

Corrected downgrade:

```
def downgrade() -> None:
    """Restore the display_name column to the users table."""
    op.add_column(
        "users",
        sa.Column(
            "display_name",
            sa.String(length=100),
            nullable=False,
            server_default="",
        ),
    )
    op.alter_column(
        "users",
        "display_name",
        server_default=None,
    )
```

The downgrade uses a temporary default when restoring the non-nullable column to a table containing existing records.

5. Verify and Apply the Migration

Checked the current database revision: `alembic current`

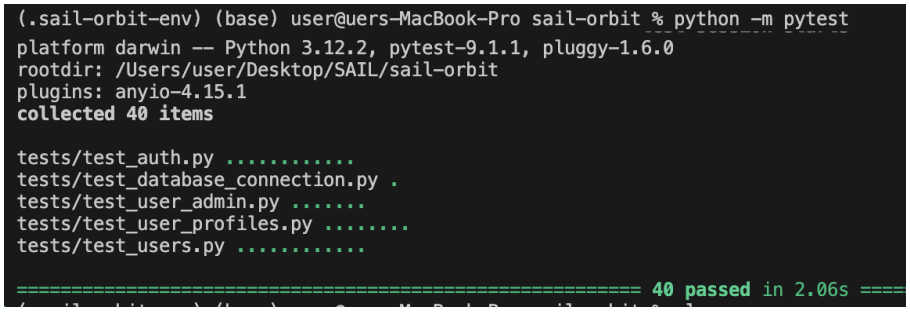
Output: `d383129da037`

Applied the corrected migration: `alembic upgrade head`

The terminal reported the upgrade from `d383129da037` to `62ce8ce120c2`.

6. Run Backend Tests

```
python -m pytest
```



7. Final Status

- Removed `display_name` references from the application code and tests.
- Preserved `username` and `bio`.
- Retained the original database migration.

- Corrected the autogenerated Alembic migration to drop only the `display_name` column.
- Successfully applied the migration.
- Verified that all 40 backend tests pass.

Section 8 Step 17i: Deferred Issues: Frontend Dependencies and Security

Identified: October 14, 2026
Status: Deferred until the application is ready for deployment.

1. Node.js compatibility

- Current version: Node.js 23.6.1.
- ESLint 10 requires Node.js 20.19+, 22.13+, or 24+.
- npm reported `EBADENGINE` warnings during installation.
- Recommended resolution: Move to a supported Node.js LTS version.

2. Dependency vulnerabilities

- npm initially reported 10 vulnerabilities: 1 moderate, 7 high, and 2 critical.
- A production-only audit (`npm audit --omit=dev`) reported 7 high-severity vulnerabilities.
- The production findings originate from the `shadcn` dependency chain, including `braces`, `micromatch`, and `fast-glob`.
- The reported `braces` vulnerability involves potential denial of service through deeply nested patterns.
- npm suggested a potentially breaking downgrade to `shadcn@1.0.0` using `npm audit fix --force`.
- No corrective changes were made.

3. Dependency classification

- `shadcn` is currently listed under production dependencies.
- A search found no direct imports of `shadcn` in the frontend's `src` directory.
- Moving `shadcn` to development dependencies may reduce production exposure, but this has not been implemented or verified.

Decision

Defer remediation while Orbit remains under active development. The vulnerabilities have not been demonstrated to affect Step 17 functionality.

Before deployment: Reassess dependency vulnerabilities, confirm which packages are required at runtime, upgrade Node.js to a supported version, apply appropriate fixes, and rerun security audits and application tests.